Type systems
(course notes)

Ambrus Kaposi

Eötvös Loránd University, Budapest, Hungary

akaposi@inf.elte.hu

December 13, 2023

Contents

Introduction **3**

I What is a programming language? **4**

1 Levels of abstraction **5**
1.1 String . . . 8
1.2 Sequence of lexical elements . . . 8
1.3 Abstract syntax tree . . . 9
1.3.1 Iteration . . . 11
1.3.2 Induction . . . 15
1.3.3 Another example of iteration and induction: natural numbers . . . 18
1.4 Well-typed syntax . . . 20
1.4.1 Type inference . . . 21
1.4.2 Standard interpretation . . . 23
1.4.3 Typing relation . . . 23
1.5 Well-typed syntax with equations . . . 24
1.5.1 Standard model and equational consistency . . . 25
1.5.2 Normalisation . . . 26
1.5.3 Transition relation . . . 29
1.5.4 Another example of normalisation: integers . . . 29

2 Bindings **33**
2.1 Making bindings first order via contexts . . . 34
2.1.1 Abstract binding trees . . . 34
2.1.2 Well-typed syntax . . . 38
2.1.3 Well-typed syntax with equations . . . 39
2.1.3.1 Standard model and inequalities in the empty context . . . 42
2.1.3.2 Type inference . . . 43
2.1.3.3 Normalisation . . . 44
2.1.3.4 A sort of first order functions . . . 52
2.2 Making bindings first order via combinators . . . 52

II Type systems **56**

3 Higher order function space **57**
3.1 Standard model . . . 59
3.2 Normalisation . . . 59

4 Product and sum types **61**
4.1 Applications . . . 63
4.2 Algebraic properties of types . . . 66
4.3 Standard model . . . 67

5 Inductive types **68**
5.1 No uniqueness rules 72
5.2 Standard model 72
5.3 Definable functions on natural numbers 73
5.4 Normalisation 75

6 Coinductive types **77**
6.1 Standard model 79

7 Summary **81**

Bibliography **83**

Introduction

These are the notes for the course on type systems for programming languages given at Eötvös Loránd University (Autumn semesters 2020–2023, Computer Science MSc course). Previous versions of this course used Robert Harper's book [11] (Autumns 2016–2019) and István Csörnyei's books [9, 8] (before 2016). Other excellent textbooks on the topic are [19, 20, 24].

In these notes we define programming languages as algebraic theories (see Chapter 1 and [13]). Most of the material is formalised in Agda, it uses an equality type in strict Prop [10] with a postulated strong tranport rule and postulated quotient inductive-inductive types [14]. Their computation rules are added to Agda using rewrite rules [7].

These notes are compiled from literate Agda files and are available at https://bitbucket.org/akaposi/typesystems. They are under continuous development.

Special thanks to Bálint Kocsis and Márk Széles who developed the first version of the formalisation during a summer internship in 2020. Thanks to István Donkó, Viktor Bense and Szumi Xie who developed materials for the tutorials and contributed significantly to the formalisation. Thanks to the following students who fixed errors and typos: András Kovács, Kálmán Kostenszky, Álmos Sőnfeld, Erik Sulcz, András Vadász, Barnabás Zahorán. Thanks to the students of the course for their excellent questions.

Part I

What is a programming language?

Chapter 1

Levels of abstraction

Learning goals of this chapter

- Levels of abstraction when defining a programming language: strings, sequences of lexical elements, abstract syntax trees, well-typed syntax, well-typed syntax with equations
- Meta language, object language
- Model, syntax, defining functions by iteration
- Dependent model, proving properties by induction
- Disjointness and injectivity of syntactic operators, finiteness of elements of the syntax
- Trivial model, C-style standard model
- Type inference
- Standard model and the standard interpreter
- Derivable and admissible operators and equations
- Equational consistency
- Normalisation, its completeness and stability

In this chapter, we study a very simple programming language called Razor (as an homàge to Hutton's razor [12]) which contains e.g. the following program.

```
if isZero (num 0 + num 1) then false else isZero (num 0)
```

A Razor program is either a numeric or a boolean expression. Numbers can be formed using num i where i is a natural number. Booleans are true or false. We have the usual if-then-else operator, addition and an isZero operator which says whether a number is 0. The above program evaluates in the following steps (each new line is a step).

```
if isZero (num 0 + num 1) then false else isZero (num 0)
if isZero (num 1) then false else isZero (num 0)
if false then false else isZero (num 0)
isZero (num 0)
true
```

We will describe this language in several iterations. Each iteration will be a more precise but also more abstract description of the language: strings, sequences of lexical elements, abstract syntax trees, well-typed syntax and well-typed syntax with equations.

See Figures 1.1, 1.2.

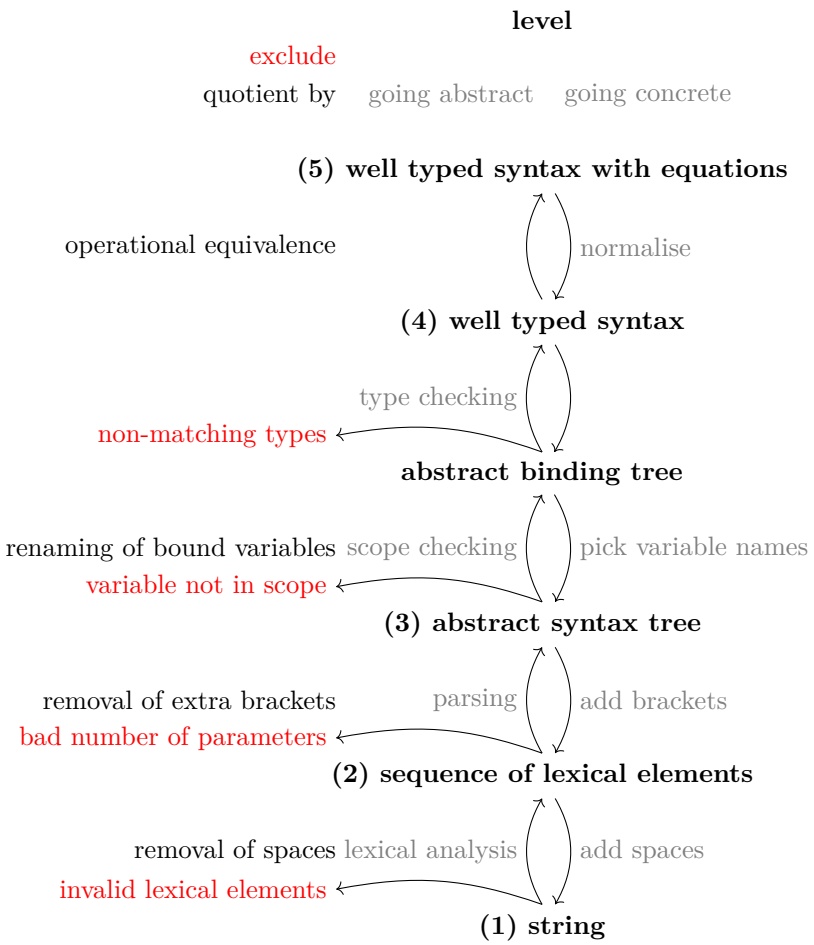
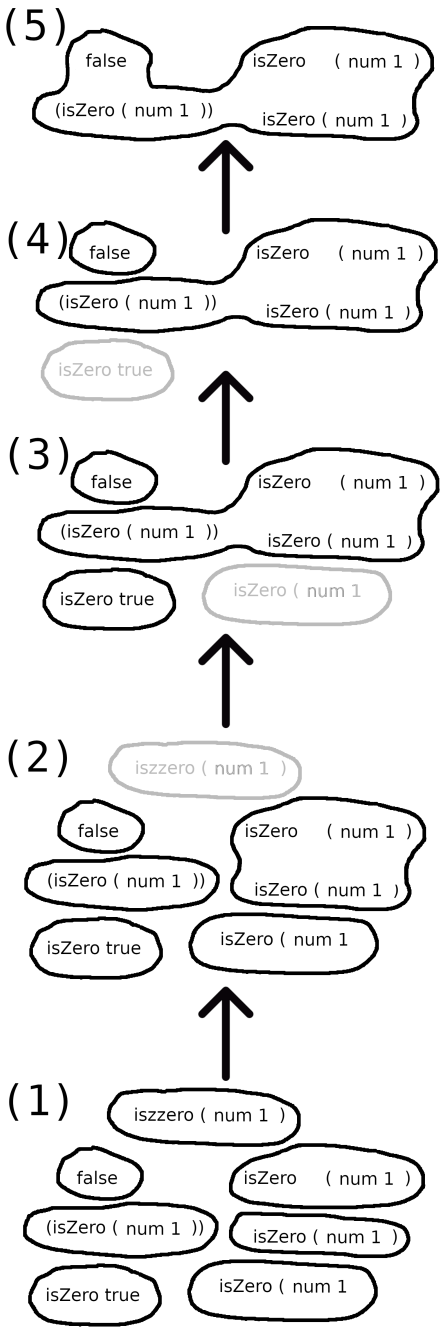


Figure 1.1: Different levels of abstraction when defining a programming language and transformations between levels. At more abstract levels, certain programs are excluded and others identified. Abstract binding trees are sometimes called well-scoped syntax trees, see chapter **??** (they are not relevant for the Razor language).



(5) well-typed syntax with equations

Ty	: Set
Tm	: Ty → Set
Bool	: Ty
Nat	: Ty
true	: Tm Bool
false	: Tm Bool
ite	: Tm Bool → Tm A → Tm A → Tm A
num	: ℕ → Tm Nat
isZero	: Tm Nat → Tm Bool
_+_	: Tm Nat → Tm Nat → Tm Nat
$\text{ite}\beta_1$	: ite true u v = u
$\text{ite}\beta_2$	: ite false u v = v
$\text{isZero}\beta_1$	: isZero (num 0) = true
$\text{isZero}\beta_2$	: isZero (num (1+n)) = false
$+\beta$	: num m + num n = num (m + n)

(4) well-typed syntax

Ty	: Set
Tm	: Ty → Set
Bool	: Ty
Nat	: Ty
true	: Tm Bool
false	: Tm Bool
ite	: Tm Bool → Tm A → Tm A → Tm A
num	: ℕ → Tm Nat
isZero	: Tm Nat → Tm Bool
_+_	: Tm Nat → Tm Nat → Tm Nat

(3) abstract syntax tree

Tm	: Set
true	: Tm
false	: Tm
ite	: Tm → Tm → Tm → Tm
num	: ℕ → Tm
isZero	: Tm → Tm
_+_	: Tm → Tm → Tm

(2) list of the following lexical elements:

(,), true, false, if, then, else, num, isZero, +, 0, 1, 2, 3, ...

(1) any string

Figure 1.2: Left: example Razor programs at different levels of abstraction. Each bubble represents a separate program. Right: description of the Razor expression language at levels (1)–(5).

1.1 String

As a first approximation, we say that a program is a string, that is, a sequence of (ASCII) characters. This is how we write programs on a computer. Any string is a program.

Many strings do not correspond to meaningful programs in our language such as num 3 - num 2 as we don't have subtraction. Also, there are different strings which represent the same program. For example, isZero (num 1) and isZero (num 1) are different as strings but should be the same programs as the extra spaces after isZero shouldn't matter.

Instead of describing which strings are meaningful programs and defining an equivalence relation for identifying strings that represent the same program, we will describe programs using a more abstract structure.

1.2 Sequence of lexical elements

We describe Razor by the following lexical elements.

$$(,), \text{true}, \text{false}, \text{if}, \text{then}, \text{else}, \text{num}, \text{isZero}, +, 0, 1, 2, ...$$

The ... part means that any natural number is a lexical element. A program is an arbitrary sequence made of these.

Our example program

if isZero (num 0 + num 1) then false else isZero (num 0)

is the following sequence.

$$[\text{if}, \text{isZero}, (, \text{num}, 0, +, \text{num}, 0,), \text{then}, \text{false}, \text{else}, \text{isZero}, (, \text{num}, 0,)]$$

Now we have much fewer programs and num 3 - num 2 is not a program anymore because there is no lexical element for -. Any two programs given as strings which differ only in the number of spaces will end up as the same program at this level: isZero (num 1) and isZero (num 1) are both given by the sequence [isZero, (, num, 1,)].

However, we still have meaningless programs, e.g. [(, true] (there is no closing parenthesis) or [num, 1, +] (+ needs two arguments), and so on. Also, there are programs which could be identified, e.g. [(, true,)] and [true] (the parentheses are redundant in the former).

Again, to solve these issues, we move to a higher-level representation of programs.

Note that we can always obtain a string from a sequence of lexical elements by simply printing the sequence. In the other direction, we can implement a lexical analyser (lexer) which turns a string into a sequence of lexical elements or returns an error.

For convenience, we usually write strings instead of lists of lexical elements. In this case, we specify that we are interpreting the strings as lists of lexical elements.

Exercise 1.1 (compulsory)**.** *Which equations of lists of lexical elements hold?*

num 3 $\stackrel{?}{=}$ num(3)
isZero 0 $\stackrel{?}{=}$ true
if 3 then (false) $\stackrel{?}{=}$ if 3 then (false)
(if 3 then (false)) $\stackrel{?}{=}$ if 3 then (false)
then then if $\stackrel{?}{=}$ then then if

Exercise 1.2 (compulsory)**.** *Mark those strings for which the lexical analyser outputs an error. For those strings that are accepted, write down the list of lexical elements it outputs.*

if true then true else num 0
if true then num 0 else num 0
if num 0 then num 0 else num 0
if if then num 0 else num 0
true + zero
true + num 1
true + isZero
isZero + 0

Exercise 1.3. *Write a lexical analyser for Razor: the input is a string, the output is either a sequence of lexical elements or an error.*

1.3 Abstract syntax tree

We define our language by the following BNF grammar (where N means a natural number):

T ::= true | false | if T then T else T | num N | isZero T | T + T

In Haskell, we would write this as follows (using Int for natural numbers).

```
data Tm = True | False | Ite Tm Tm Tm | Num Int | IsZero Tm | Tm :+: Tm
```

We will use the following Agda notation for the same definition. Here every operator is followed by its *arity*: the number of arguments it takes. We call elements of Tm terms. We write ite instead of if-then-else and +o instead of + for convenience.

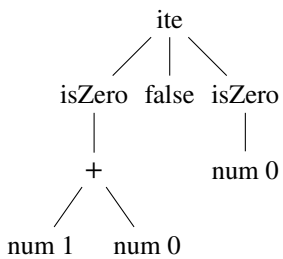
```
data Tm : Set where
  true   : Tm
  false  : Tm
  ite    : Tm → Tm → Tm → Tm
  num    : ℕ → Tm
  isZero : Tm → Tm
  _+o_   : Tm → Tm → Tm
```

In this description, we added the information that if-then-else has three, isZero one and addition two Tm arguments while num has a natural number argument. Programs are now trees which have true, false or num i at their leaves and they can have ternary branching with ite at the branching node, unary branching with isZero at the node or binary branching with + at the node.

Our example program

if isZero (num 0 + num 1) then false else isZero (num 0)

is depicted as follows.

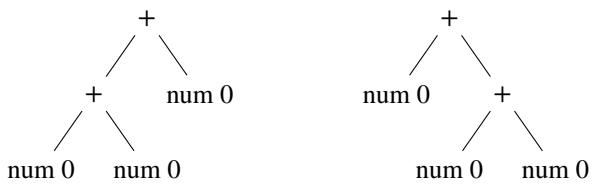


Instead of drawing trees we usually use linear notation to save space:

```
ite (isZero (num 0 +o num 1)) false (isZero (num 0))
```

Programs such as (true and num 1 + which were valid as sequences of lexical elements do not correspond to any tree, and the programs (true) and true correspond to the same tree.

Note that (num 0 + num 0) + num 0 and num 0 + (num 0 + num 0) are different trees.



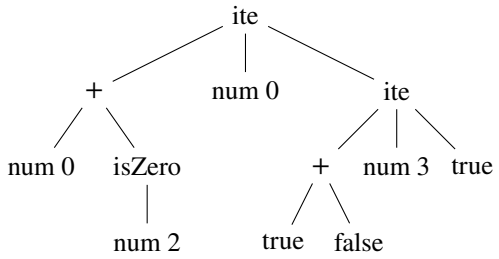
Even if the intuitive meaning of both is the number zero, they are different as trees. An example of parentheses which even changes the intuitive meaning of an expression is $1 + (2 * 3) = 7 \neq 9 = (1 + 2) * 3$ (this example is not in the Razor language as it has multiplication).

The *meta theory* (meta language) is the language that we use to speak about the *object theory* (object language). Our meta language is Agda (and sometimes English but we can translate all of our English arguments to Agda), our object language is Razor.

Exercise 1.4 (compulsory)**.** *Draw the trees depicting the following programs:*

(true + true) + true
((true + true) + true) + true
(true + (true + true)) + true
true + ((true + true) + true)
ite (true + true) ((true + true) + true) (true + (true + true))

Exercise 1.5 (compulsory)**.** *Write down the following tree with linear notation:*



Exercise 1.6 (compulsory)**.** *Here are some lists of lexical elements (written as strings). Which ones are accepted by the parser for Razor?*

isZero (num 1) + (num 1)
isZero (num 1 + num 1)
isZero (num 2) 3
num 1 + isZero
1 + num 1
true + 1

Exercise 1.7 (compulsory)**.** *Write down the following BNF notations in Agda using* data *(as we did for* Tm*).* N *means natural numbers, as before.*

T ::= op0 | op1 T | op2 T T | op3 T T T | op4 T T T T

A ::= a | fb B
B ::= fa A

V ::= vzero | vsuc V
E ::= num N | E < E | E = E | var V
C ::= V := E | while E S | if E then S else S
S ::= empty | C colon S

Exercise 1.8. *Write a parser for Razor: a program which given a sequence of lexical elements, outputs an element of* Tm *or an error.*

Exercise 1.9 (compulsory)**.** *Which of the following lists of lexical elements are accepted by a parser and produce a* Tm*?*

if true then true else num 0
if true then num 0 else num 0
if num 0 then num 0 else num 0
if num 0 then (num 0 else num 0)
if if then num 0 else num 0
true + false
true + (num 1))
true + (num 1)
(true + isZero)
isZero + num 0

Exercise 1.10 (compulsory)**.** *Which of the following abstract syntax trees are equal to* num 0 + (num 1 + num 1)*?*

num 0 + ((num 1) + (num 1))
(num 0 + ((num 1) + (num 1)))
(num 0 + num 1) + num 1
((num 0 + num 1) + num 1)
num 1 + num 1
num 2
(num 0 + num 0) + (num 1 + num 1)
ite true (num 0 + (num 1 + num 1)) (num 0)
ite true (num 0 + (num 1 + num 1)) (num 0 + (num 1 + num 1))
ite true (num 0 + (num 1 + num 1)) false

1.3.1 Iteration

We call a set A with two elements, a ternary operation on A (an A → A → A → A function), an infinite sequence of As (a ℕ → A function), an endofunction (an A → A function) and a binary operation on A (an A → A → A function) a *model* of *RazorAST* (sometimes called a RazorAST algebra). We fix a notation for this. A RazorAST model (or simply model) is a record with the following fields.

```
record Model {ℓ} : Set (lsuc ℓ) where
  field
    Tm     : Set ℓ
    true   : Tm
    false  : Tm
    ite    : Tm → Tm → Tm → Tm
    num    : ℕ → Tm
    isZero : Tm → Tm
    _+o_   : Tm → Tm → Tm
```

Note that we used the same names as for the previously defined trees. We call the above defined Razor programs (trees) the *syntax* of Razor. The syntax also forms a model, we denote it by I, its components are I.Tm, I.true, I.false, and so on. The syntax has the property that it can be interpreted into any other model (it is an initial model, hence the name I): given a model M, there is a homomorphism (a function that commutes with all operators) from I to M which we denote M.⟦_⟧. This function is called the *iterator* (it is sometimes called recursor, fold, catamorphism, nondependent eliminator, interpreter, evaluator).

```
⟦_⟧ : I.Tm → Tm
⟦ I.true        ⟧ = true
⟦ I.false       ⟧ = false
⟦ I.ite t t' t" ⟧ = ite ⟦ t ⟧ ⟦ t' ⟧ ⟦ t" ⟧
⟦ I.num n       ⟧ = num n
⟦ I.isZero t    ⟧ = isZero ⟦ t ⟧
⟦ t I.+o t'     ⟧ = ⟦ t ⟧ +o ⟦ t' ⟧
```

The syntax is sometimes called the *free model* over the empty set: we start with the empty set, and then we freely add elements that are generated from this set by the operators. In the beginning, the only generated new elements are I.true, I.false, I.num i (for all is), because these don't require any preexisting elements. Now we already have some elements in the set, so we can apply the other operators (such as ite) to those, which in turn generate even more elements, then we can apply the operators to those elements as well, and so on. All the terms in the syntax are generated this way.

An example of another RazorAST model is h (height) defined as follows.

```
H : Model
H = record
```

```
{ Tm     = ℕ
; true   = 0
; false  = 0
; ite    = λ n n' n" → 1 + max n (max n' n")
; num    = λ n → 0
; isZero = λ n → 1 + n
; _+o_   = λ n n' → 1 + max n n'
}
```

Note that we distinguish two addition operations: +o is the addition operator in a model (H in this case), while + is just addition of (metatheoretic) natural numbers.

If our example term ite (isZero (zero + suc zero)) false (isZero zero) is interpreted in the model H, we obtain the height of the tree.

```
H.ite (H.isZero (H.num 0 H.+o H.num 1))              H.false (H.isZero (H.num 0))  ≡≡
1 + max (H.isZero (H.num 0 H.+o H.num 1))   (max H.false (H.isZero (H.num 0))) ≡≡
1 + max (1 + (H.num 0 H.+o H.num 1))        (max H.false (H.isZero (H.num 0))) ≡≡
1 + max (1 + (1 + max (H.num 0) (H.num 1))) (max H.false (H.isZero (H.num 0))) ≡≡
1 + max (1 + (1 + max 0 0))                 (max H.false (H.isZero (H.num 0))) ≡≡
1 + max (1 + (1 + 0))                       (max H.false (H.isZero (H.num 0))) ≡≡
1 + max 2                                   (max H.false (H.isZero (H.num 0))) ≡≡
1 + max 2                                   (max 0 (H.isZero (H.num 0)))       ≡≡
1 + max 2                                   (max 0 (1 + (H.num 0)))            ≡≡
1 + max 2                                   (max 0 (1 + 1))                    ≡≡
1 + max 2                                   (max 0 2)                          ≡≡
1 + max 2                                   2                                  ≡≡
1 + 2                                                                          ≡≡
3                                                                              ∎
```

The metatheoretic set of booleans 2 has two elements tt and ff. We denote logical disjunction by ∨.

We define another model T where terms are elements of 2 and a term is tt if and only if it contains true.

```
T : Model
T = record
  { Tm     = 2
  ; true   = tt
  ; false  = ff
  ; ite    = λ t t' t" → t ∨ t' ∨ t"
  ; num    = λ n → ff
  ; isZero = λ t → t
  ; _+o_   = λ t t' → t ∨ t'
  }
```

Now

```
T.ite (T.isZero (T.num 0 T.+o T.num 1))    T.false    (T.isZero (T.num 0)) ≡≡
      (T.isZero (T.num 0 T.+o T.num 1)) ∨ T.false ∨ (T.isZero (T.num 0)) ≡≡
      (T.num 0 T.+o T.num 1)            ∨ ff      ∨ (T.num 0)            ≡≡
      (T.num 0 ∨     T.num 1)           ∨ ff      ∨ ff                   ≡≡
      (ff      ∨     ff)                ∨ ff      ∨ ff                   ≡≡
      ff                                                                 ∎
```

and

```
T.ite T.true (T.num 0) T.false ≡≡ T.true ∨ (T.num 0) ∨ T.false ≡≡ tt ∨ ff ∨ ff ≡≡ tt
```

We have that for any model M

```
M.⟦ I.ite   (I.isZero   (I.num 0   I.+o I.num     1)) I.false   (I.isZero   (I.num 0)) ⟧ ≡
      M.ite (M.isZero (M.num 0 M.+o M.num 1)) M.false (M.isZero (M.num 0))
```

and similarly for any other syntactic term.

In functional languages like Haskell or Agda, iterating into a model is a special case of a recursive definition using pattern matching on the syntax. For example, H.⟦_⟧ corresponds to a height function defined by pattern matching as follows.

```
height : I.Tm → ℕ
height I.true          = 0
height I.false         = 0
height (I.ite t t' t") = 1 + max (height t) (max (height t') (height t"))
height (I.num n)       = 0
height (I.isZero t)    = 1 + height t
height (t I.+o t')     = 1 + max (height t) (height t')
```

Iterating into h satisfies the same equations:

```
H.⟦_⟧ : I.Tm → ℕ

H.⟦ I.true ⟧          ≡ 0
H.⟦ I.false ⟧         ≡ 0
H.⟦ I.ite t t' t" ⟧ ≡ 1 + max H.⟦ t ⟧ (max H.⟦ t' ⟧ H.⟦ t" ⟧)
H.⟦ I.num n ⟧         ≡ 0
H.⟦ I.isZero t ⟧      ≡ 1 + H.⟦ t ⟧
H.⟦ t I.+o t' ⟧       ≡ 1 + max H.⟦ t ⟧ H.⟦ t' ⟧
```

Some Haskell functions defined by pattern matching are not expressible using iteration into a model. Haskell allows nonterminating recursion or recursion where some cases are not handled, as in the following examples.

```
f : I.Tm → ℕ
f t = f t

g : I.Tm → ℕ
g I.true  = 3
g I.false = 1
```

In Agda, all functions defined by pattern matching are also expressible using the iterator (or using induction, see below). For details see [16, 1].

The syntactic operators are *disjoint*: for example, I.true is not equal to I.false. We can show this using a model where terms are metatheoretic booleans, true is metatheoretic true, false is metatheoretic false. The other components don't matter, here we just set them to constant false.

```
TF : Model
TF = record
  { Tm     = 2
  ; true   = tt
  ; false  = ff
  ; ite    = λ _ _ _ → ff
  ; num    = λ _ → ff
  ; isZero = λ _ → ff
  ; _+o_   = λ _ _ → ff
  }
```

Now we can use the iterator and congruence of equality to obtain a contradiction from I.true ≡ I.false:

```
true≠false : ¬ (I.true ≡ I.false)
true≠false e = tt≠ff (cong TF.⟦_⟧ e)
```

However, there are models where true = false, e.g. the trivial model where terms are given by the set with one element tt and all operators return this element.

```
Triv : Model
Triv = record
  { Tm     = Lift ⊤
  ; true   = _
  ; false  = _
  ; ite    = _
  ; num    = _
  ; isZero = _
  ; _+o_   = _
  }
```

Every syntactic term is *finite*, e.g. there is no syntactic term of the form I.isZero (I.isZero (I.isZero ⋯)) (an infinitely deep tree with I.isZero at each node) because it would be mapped to an infinite natural number by H.⟦_⟧ which does not exist. This is not true for arbitrary models: there can be models where Tm has infinite elements, for example, $\mathsf{Tm} = \mathbb{N} \to \mathbb{2}$ (a term is an infinite sequence of booleans).

Exercise 1.11 (compulsory)**.** *Define a model where a term is a natural number and if we interpret a syntactic term in the model, we get the number of* true*s in the tree. For example,* false +o num 3 = 0*,* true +o num 3 = 1*,* true +o true = 2*.*

Exercise 1.12 (compulsory)**.** *Define a model which calculates the number of nodes in a tree, e.g.* num 1 = 1*,* isZero (num 1) = 2*,* isZero (num 1) + true = 4*.*

Exercise 1.13 (compulsory)**.** *Prove that* I.num 0 I.+o I.num 0 *is not equal* I.num 0*.*

Exercise 1.14 (compulsory)**.** *Prove that* I.isZero (I.num 100) *is not equal* I.isZero I.true*.*

Exercise 1.15 (compulsory)**.** *Prove that* I.isZero (I.num 100) *is not equal* I.isZero (I.num 101)*.*

Exercise 1.16. *Create a model where* true = false*, but* true ≠ num 0*.*

Exercise 1.17. *Define a RazorAST model where terms are* $\mathbb{N} \to \mathbb{2}$ *sequences, any sequence that starts with* tt *is interpreted as* true*, any sequence that startsf with* ff *is* false*,* num i *is a sequence with* i *many* tt*s in it. Addition is a sequence where the number of* tt*s is the sum of the number of* tt*s in the two sequences.*

Exercise 1.18. *Define a RazorAST model where terms are natural numbers, booleans are interpreted in C-style: false is* 0*, true is* 1 *and for if-then-else, anything that is not* 0 *is interpreted as true. We can call this model the degenerate standard model.*

Exercise 1.19 (recommended)**.** *Define a nonstandard C-style model the interpretation into which is not onto. What is the simplest such model?*

Exercise 1.20 (compulsory)**.** *Is there a model where* Tm = Lift ⊥*?*

Exercise 1.21 (compulsory)**.** *Show that for any two models* M *and* N *there is a model where* Tm = M.Tm × N.Tm*.*

Exercise 1.22. *Create a model where* Tm = $\mathbb{2}$*,* num 0 = tt *and all other operators are* ff*. With the help of this, create a function* isZero : I.Tm → $\mathbb{2}$*. Then create an optimisation* I.Tm → I.Tm *which maps* I.num 0 + t *to* t *and* t + I.num 0 *to* t*.*

Exercise 1.23 (compulsory)**.** *Consider a subset of RazorAST where we only have one type:*

```
Tm : Set
zero : Tm
suc : Tm → Tm
_+_ : Tm → Tm → Tm
```

Define a model St *such that* St.⟦_⟧ : I.Tm → ℕ *is an interpreter for this language (where* I *is the syntax for this language). E.g.* St.⟦num 1 + num 3⟧ = 4*.*

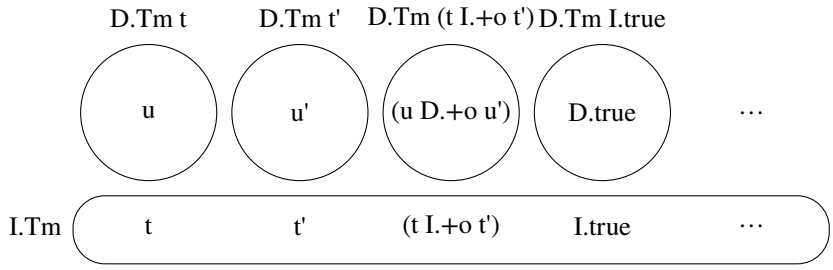


Figure 1.3: Depiction of a dependent model D. There is a separate set D.Tm t for every syntactic term t.

1.3.2 Induction

In addition to iteration, the syntax supports induction which is a generalisation (dependent variant) of iteration. Induction can be stated by saying that for any *dependent model* D there is a *dependent homomorphism* from I to D. A *dependent model* (dependent algebra, displayed algebra/model) is given by the following components.

```
record DepModel {ℓ} : Set (lsuc ℓ) where
  field
    Tm•     : I.Tm → Set ℓ
    true•   : Tm• I.true
    false•  : Tm• I.false
    ite•    : {t : I.Tm} → Tm• t → {u : I.Tm} → Tm• u → {v : I.Tm} → Tm• v →
              Tm• (I.ite t u v)
    num•    : (n : ℕ) → Tm• (I.num n)
    isZero• : {t : I.Tm} → Tm• t → Tm• (I.isZero t)
    _+o•_   : {u : I.Tm} → Tm• u → {v : I.Tm} → Tm• v → Tm• (u I.+o v)
```

In a dependent model, instead of a set of terms, there is a family of sets indexed over I.Tm, that is, there is a set Tm• t for each element t : I.Tm (See Figure 1.3). Then we have elements of these families at each syntactic operator as index. So, we have a Tm• I.true, a Tm• I.false, ..., a Tm• (t I.+o t'). For operators with parameters, we also have induction hypotheses, for example, we have Tm• t and Tm• t' as inputs for _+o_.

The dependent homomorphism from the syntax is called *induction* (induction principle, eliminator). This is a dependent function which for an input t : I.Tm outputs a Tm• t. Moreover, as in the non-dependent case, it maps each syntactic operator to its counterpart in the dependent model.

```
⟦_⟧ : (t : I.Tm) → Tm• t
⟦ I.true        ⟧ = true•
⟦ I.false       ⟧ = false•
⟦ I.ite t t' t" ⟧ = ite• ⟦ t ⟧ ⟦ t' ⟧ ⟦ t" ⟧
⟦ I.num n       ⟧ = num• n
⟦ I.isZero t    ⟧ = isZero• ⟦ t ⟧
⟦ t I.+o t'     ⟧ = ⟦ t ⟧ +o• ⟦ t' ⟧
```

A special case of the dependent model is when Tm• t = Lift (P t) for some P : I.Tm → Prop, that is, Tm is a predicate (unary relation) on syntactic terms. In this case, the other components say that each syntactic operator respects the predicate. For example, if the predicate holds for t and t', then it also holds for t I.+o t'. C.f. natural number models (a set with an element and an endofunction) and induction on natural numbers, see Section 1.3.3. All inductively defined sets come with a notion of model, iteration, dependent model and induction [14]. In addition to natural numbers, other well-known examples are lists, binary trees.

Consider the following model which describes the number of leaves.

```
L1 : Model
L1 = record
  { Tm     = ℕ
  ; true   = 1
  ; false  = 1
  ; ite    = λ t t' t" → t + t' + t"
  ; num    = λ _ → 1
  ; isZero = λ t → t
  ; _+o_   = _+_
  }
module L1 = Model L1
```

Here is another model for describing the number of leaves using a different logic: At a binary branching, the number of leaves increases by one, at a ternary branching it increases by two.

```
L2 : Model
L2 = record
  { Tm     = ℕ
  ; true   = 0
  ; false  = 0
  ; ite    = λ t t' t" → 2 + t + t' + t"
  ; num    = λ _ → 0
  ; isZero = λ t → t
  ; _+o_   = λ t t' → 1 + t + t'
  }
module L2 = Model L2
```

Now we prove that counting the leaves of a syntactic term using L1 is equal to counting the leaves using L2 and adding one. To this end, we define the following dependent model

```
D : DepModel
D = record
  { Tm• = λ t → Lift (L1.⟦ t ⟧ ≡ 1 + L2.⟦ t ⟧)
  ; true• = mk refl
  ; false• = mk refl
  ; ite• = λ { {t}(mk t•){u}(mk u•){v}(mk v•) → mk (
    L1.⟦ t I.+o u I.+o v ⟧
                                        ≡⟨ refl ⟩
    L1.⟦ t ⟧ + L1.⟦ u ⟧ + L1.⟦ v ⟧
                                        ≡⟨ cong₃ (λ x y z → x + y + z) t• u• v• ⟩
    (1 + L2.⟦ t ⟧) + (1 + L2.⟦ u ⟧) + (1 + L2.⟦ v ⟧)
                                        ≡⟨ ass+ {1 + L2.⟦ t ⟧} ⟩
    (1 + L2.⟦ t ⟧) + ((1 + L2.⟦ u ⟧) + (1 + L2.⟦ v ⟧))
                                        ≡⟨ cong ((1 + L2.⟦ t ⟧) +_) (ass+ {1 + L2.⟦ u ⟧} ⁻¹) ⟩
    (1 + L2.⟦ t ⟧) + ((1 + L2.⟦ u ⟧ + 1) + L2.⟦ v ⟧)
                                        ≡⟨ cong (λ z → (1 + L2.⟦ t ⟧) + (z + L2.⟦ v ⟧))
                                                (+comm {1 + L2.⟦ u ⟧}) ⟩
    (1 + L2.⟦ t ⟧) + (2 + L2.⟦ u ⟧ + L2.⟦ v ⟧)
                                        ≡⟨ refl ⟩
    (1 + L2.⟦ t ⟧) + (2 + (L2.⟦ u ⟧ + L2.⟦ v ⟧))
                                        ≡⟨ ass+ {1 + L2.⟦ t ⟧} ⁻¹ ⟩
    ((1 + L2.⟦ t ⟧) + 2) + (L2.⟦ u ⟧ + L2.⟦ v ⟧)
                                        ≡⟨ cong (_+ (L2.⟦ u ⟧ + L2.⟦ v ⟧))
                                                (+comm {1 + L2.⟦ t ⟧}{2}) ⟩
    3 + L2.⟦ t ⟧ + (L2.⟦ u ⟧ + L2.⟦ v ⟧)
                                        ≡⟨ ass+ {3 + L2.⟦ t ⟧} ⁻¹ ⟩
    3 + L2.⟦ t ⟧ + L2.⟦ u ⟧ + L2.⟦ v ⟧
```

```
                                                  ≡⟨ refl ⟩
    1 + L2.⟦ I.ite t u v ⟧
                                                  ∎)}
  ; num• = λ _ → mk refl
  ; isZero• = λ t• → t•
  ; _+o•_ = λ { {u}(mk u•){v}(mk v•) → mk (
    L1.⟦ u I.+o v ⟧
                                                  ≡⟨ refl ⟩
    L1.⟦ u ⟧ + L1.⟦ v ⟧
                                                  ≡⟨ cong (_+ L1.⟦ v ⟧) u• ⟩
    1 + L2.⟦ u ⟧ + L1.⟦ v ⟧
                                                  ≡⟨ cong (λ z → 1 + L2.⟦ u ⟧ + z) v• ⟩
    (1 + L2.⟦ u ⟧) + (1 + L2.⟦ v ⟧)
                                                  ≡⟨ ass+ {1 + L2.⟦ u ⟧} ⁻¹ ⟩
    ((1 + L2.⟦ u ⟧) + 1) + L2.⟦ v ⟧
                                                  ≡⟨ cong (_+ L2.⟦ v ⟧) (ass+ {1}{_}{1} ⁻¹) ⟩
    (1 + (L2.⟦ u ⟧ + 1)) + L2.⟦ v ⟧
                                                  ≡⟨ cong (λ z → 1 + z + L2.⟦ v ⟧)
                                                        (+comm {L2.⟦ u ⟧}{1}) ⟩
    (1 + (1 + L2.⟦ u ⟧)) + L2.⟦ v ⟧
                                                  ≡⟨ refl ⟩
    2 + (L2.⟦ u ⟧ + L2.⟦ v ⟧)
                                                  ≡⟨ refl ⟩
    1 + (L2.⟦ u I.+o v ⟧)
                                                  ∎)}
  }
module D = DepModel D
```

Interpreting into the dependent model D gives our proof:

```
L1=L2 : ∀ t → L1.⟦ t ⟧ ≡ 1 + L2.⟦ t ⟧
L1=L2 t = un D.⟦ t ⟧
```

The operators of the syntax are *injective*. For example, if I.isZero t ≡ I.isZero t', then t ≡ t'. To show this, we use a trivial dependent model which is only used to peel off the I.isZero from any term.

```
module inj where
  D' : DepModel
  D' = record
    { Tm• = λ _ → I.Tm
    ; true• = I.true
    ; false• = I.true
    ; ite• = λ _ _ _ → I.true
    ; num• = λ _ → I.true
    ; isZero• = λ {t} _ → t
    ; _+o•_ = λ _ _ → I.true
    }
  module D' = DepModel D'
  isZeroInj : ∀{t t'} → I.isZero t ≡ I.isZero t' → t ≡ t'
  isZeroInj e = cong D'.⟦_⟧ e
```

Note that there are models in which isZero is not injective.

Exercise 1.24 (recommended)**.** *Show that the number of* true*s in a syntactic term is less than or equal to* 3 *to the power the height of the tree. For this, define a dependent model* D *where* D.Tm t *has an inhabitant exactly when* N.⟦t⟧ *is less than or equal to* 3 ^ (H.⟦t⟧)*. Here* N *is the model defined in Exercise 1.11 and* H *is the model defined for calculating the height of a term.*

D.Tm t := N.⟦ t ⟧ ≤ 3 ^ (H.⟦ t ⟧)
D.true : N.⟦ I.true ⟧ = 1 ≤ 1 = 3 ^ 0 = 3 ^ (H.⟦ I.true ⟧)
D.false : N.⟦ I.false ⟧ = 0 ≤ 1 = 3 ^ 0 = 3 ^ (H.⟦ I.false ⟧)
D.ite t^D $t^{D'}$ $t^{D''}$:
N.⟦ I.ite t t' t'' ⟧ =
N.⟦t⟧ + N.⟦t'⟧ + N.⟦t''⟧ ≤(t^D,$t^{D'}$,$t^{D''}$)
3 ^ (H.⟦t⟧) + 3 ^ (H.⟦t'⟧) + 3 ^ (H.⟦t''⟧) ≤
3 ^ (H.⟦t⟧) + 3 ^ (H.⟦t'⟧) + 3 ^ (H.⟦t''⟧) ≤
3 * 3 ^ (max H.⟦t⟧ (max H.⟦t'⟧ H.⟦t''⟧)) =
3 ^ (1 + max H.⟦t⟧ (max H.⟦t'⟧ H.⟦t''⟧)) =
3 ^ (H.⟦ I.ite t t' t'' ⟧)
D.num n : N.⟦ I.num n ⟧ = 0 ≤ 1 = 3 ^ 0 = 3 ^ (H.⟦ I.num n ⟧)
D.isZero t^D : N.⟦ I.isZero t ⟧ = N.⟦ t ⟧ ≤(t^D) 3 ^ (H.⟦ t ⟧) ≤ 3 ^ (1 + H.⟦ t ⟧) = 3 ^ (H.⟦ I.isZero t ⟧)
t^D D.+o $t^{D'}$:
N.⟦ t I.+o t' ⟧ =
N.⟦ t ⟧ + N.⟦ t' ⟧ ≤(t^D,$t^{D'}$)
3 ^ (H.⟦ t ⟧) + 3 ^ (H.⟦ t' ⟧) ≤
2 * 3 ^ (max H.⟦ t ⟧ H.⟦ t' ⟧) ≤
3 * 3 ^ (max H.⟦ t ⟧ H.⟦ t' ⟧) ≤
3 ^ (1 + max H.⟦ t ⟧ H.⟦ t' ⟧) =
3 ^ (H.⟦ t I.+o t' ⟧)

Exercise 1.25. *Show that every model can be turned into a dependent model, and derive the iterator using the induction principle.*

Exercise 1.26 (recommended)**.** *Prove that* H.⟦ t ⟧ ≤ S.⟦ t ⟧ *where* S *is the model defined in exercise 1.12.*

Exercise 1.27 (recommended)**.** *Prove that* I._+o_ *is injective: if* u I.+o v ≡ u' I.+o v', *then* u ≡ u' *and* v = v'. *Define a model* M *where* M._+o_ *is not injective.*

Exercise 1.28. *Define a model which counts the number of* true*s and* false*s in a term. Define a model which counts the number of* true*s in a term. Show that interpreting a syntactic term in the first one always gives a greater or equal number than interpreting a term in the second one.*

1.3.3 Another example of iteration and induction: natural numbers

A model of naturals is a set Nat with an element Zero and an endofunction Suc.

```
record Model {ℓ} : Set (lsuc ℓ) where
  field
    Nat  : Set ℓ
    Zero : Nat
    Suc  : Nat → Nat
```

There is a model I given by actual natural numbers.

```
I : Model
I = record { Nat = ℕ ; Zero = 0 ; Suc = 1 +_ }
```

For any other model M, we have a function from I.Nat to M.Nat which respects the two operations.

M.⟦_⟧ : I.Nat → M.Nat
M.⟦ I.Zero ⟧ = M.Zero
M.⟦ I.Suc n ⟧ = M.Suc M.⟦ n ⟧

We define the following model where Nat is syntactic natural numbers.

```
M : Model
M = record { Nat = I.Nat ; Zero = I.Suc I.Zero ; Suc = λ n → I.Suc (I.Suc n) }
```

Interpretation into M is the function $n \mapsto 2 * n + 1$:

```
testM0 : M.⟦ 0 ⟧ ≡ 1
testM1 : M.⟦ 1 ⟧ ≡ 3
testM2 : M.⟦ 2 ⟧ ≡ 5
```

Now we define a model where Nat is endofunctions on I.Nat, Zero is the identity function, and Suc is post-composition with I.Suc.

```
A : Model
A = record { Nat = I.Nat → I.Nat ; Zero = λ n → n ; Suc = λ f → I.Suc ∘ f }
```

Interpretation into A is the function that maps n into the function which adds n to a number:

```
testA0 : A.⟦ 0 ⟧ ≡ λ n → n
testA1 : A.⟦ 1 ⟧ ≡ I.Suc
testA2 : A.⟦ 2 ⟧ ≡ I.Suc ∘ I.Suc
testA3 : A.⟦ 3 ⟧ ≡ I.Suc ∘ I.Suc ∘ I.Suc
```

Thus we can define addition of natural numbers as follows.

```
_+'_ : I.Nat → I.Nat → I.Nat
_+'_ = A.⟦_⟧

test1+3 : 1 +' 3 ≡ 4
test3+2 : 3 +' 2 ≡ 5
```

A dependent model is the data for induction on natural numbers (we use only Prop-valued families for convenience, so this is not the mose general version of DepModel).

```
record DepModel {ℓ} : Set (lsuc ℓ) where
  field
    Nat•  : I.Nat → Prop ℓ
    Zero• : Nat• I.Zero
    Suc•  : {n : I.Nat} → Nat• n → Nat• (I.Suc n)
```

For example, we prove associativity of the above addition by the following dependent model. The Nat component says what we want to prove for each number, the Zero component is the base case, the Suc components is the inductive case.

```
Ass : (n o : I.Nat) → DepModel
Ass n o = record
  { Nat•  = λ m → (m +' n) +' o ≡ m +' (n +' o)
  ; Zero• = refl
  ; Suc•  = cong suc
  }
```

The base case holds by reflexivity, in the inductive case we simply use the induction hypothesis. Now we obtain the proof of associativity by interpreting into the dependent model Ass.

```
ass : (m n o : I.Nat) → (m +' n) +' o ≡ m +' (n +' o)
ass m n o = Assno.⟦_⟧ m
  where
    module Assno = DepModel (Ass n o)
```

Exercise 1.29 (recommended)**.** *Show that* 0 *is right unit for addition using another dependent model.*

Exercise 1.30 (recommended)**.** *Show that* + *is commutative. You will need two separate dependent models.*

Exercise 1.31 (recommended)**.** *Show that the operators of the syntax are disjoint:* I.Suc i ≠ I.Zero.

Exercise 1.32 (recommended)**.** *Show that* I.Suc *is injective.*

1.4 Well-typed syntax

The abstract syntax tree description of Razor still contains meaningless programs, e.g. isZero true or true + num 2. The problem is that isZero expects a natural number and not a boolean and + expects two natural numbers. The solution is to add the type information to the terms, that is, whether a term is a boolean or a natural number.

The language RazorWT (well-typed) contains two sorts: Ty and Tm, and the latter sort is indexed by the first one. A RazorWT model is a record with the following components.

```
record Model {ℓ ℓ'} : Set (lsuc (ℓ ⊔ ℓ')) where
  field
    Ty     : Set ℓ
    Tm     : Ty → Set ℓ'
    Nat    : Ty
    Bool   : Ty
    true   : Tm Bool
    false  : Tm Bool
    ite    : {A : Ty} → Tm Bool → Tm A → Tm A → Tm A
    num    : ℕ → Tm Nat
    isZero : Tm Nat → Tm Bool
    _+o_   : Tm Nat → Tm Nat → Tm Nat
```

There is no set of terms anymore, but there is a set of Nat-terms and a set of Bool-terms, separately. Each operator expects a term of the correct type as input and returns a term of the appropriate type. For example, the first argument of ite has to be a boolean and the second and third arguments have to be of the same type. The type of ite t u v is the same as the type of the second and third arguments. This type is an *implicit argument* of ite which we omit for readability (the full type would be (A : Ty) → Tm Bool → Tm A → Tm A → Tm A, now it is {A : Ty} → Tm Bool → Tm A → Tm A → Tm A).

We could have used two sorts, TmBool and TmNat instead however that would not generalise to other type formers such as lists where we would need infinitely many sets of terms.

The syntax of RazorWT is still called I and its iterator consists of two functions (one for each sort, denoted by ⟦_⟧T and ⟦_⟧t) which preserve all operations.

```
⟦_⟧T : I.Ty → Ty
⟦_⟧t  : ∀{A} → I.Tm A → Tm ⟦ A ⟧T
⟦ I.Nat      ⟧T = Nat
⟦ I.Bool     ⟧T = Bool
⟦ I.true     ⟧t = true
⟦ I.false    ⟧t = false
⟦ I.ite t u v ⟧t = ite ⟦ t ⟧t ⟦ u ⟧t ⟦ v ⟧t
⟦ I.num n    ⟧t = num n
⟦ I.isZero t ⟧t = isZero ⟦ t ⟧t
⟦ u I.+o v   ⟧t = ⟦ u ⟧t +o ⟦ v ⟧t
```

We also have a notion of dependent model and induction just as for RazorAST.

```
record DepModel {ℓ ℓ'} : Set (lsuc (ℓ ⊔ ℓ')) where
  field
    Ty•     : I.Ty → Set ℓ
    Tm•     : ∀{A} → Ty• A → I.Tm A → Set ℓ'
    Nat•    : Ty• I.Nat
    Bool•   : Ty• I.Bool
    true•   : Tm• Bool• I.true
    false•  : Tm• Bool• I.false
    ite•    : ∀{A t u v}{A• : Ty• A} → Tm• Bool• t → Tm• A• u → Tm• A• v →
              Tm• A• (I.ite t u v)
```

```
  num•    : (n : ℕ) → Tm• Nat• (I.num n)
  isZero• : ∀{t} → Tm• Nat• t → Tm• Bool• (I.isZero t)
  _+o•_   : ∀{u v} → Tm• Nat• u → Tm• Nat• v → Tm• Nat• (u I.+o v)
⟦_⟧T : (A : I.Ty) → Ty• A
⟦_⟧t  : ∀{A}(t : I.Tm A) → Tm• ⟦ A ⟧T t
⟦ I.Nat       ⟧T = Nat•
⟦ I.Bool      ⟧T = Bool•
⟦ I.true      ⟧t = true•
⟦ I.false     ⟧t = false•
⟦ I.ite t u v ⟧t = ite• ⟦ t ⟧t ⟦ u ⟧t ⟦ v ⟧t
⟦ I.num n     ⟧t = num• n
⟦ I.isZero t  ⟧t = isZero• ⟦ t ⟧t
⟦ u I.+o v    ⟧t = ⟦ u ⟧t +o• ⟦ v ⟧t
```

An alternative notation for the syntax is the *derivation rule* style. Instead of

ite : {A : Ty} → Tm Bool → Tm A → Tm A → Tm A

or even

ite : {A : Ty}(t : Tm Bool)(u : Tm A)(v : Tm A) → Tm A

we write

$$\frac{\text{t : Tm Bool} \qquad \text{u : Tm A} \qquad \text{v : Tm A}}{\text{ite t u v : Tm A}}$$

and similarly for the other rules. The horizontal line simply denotes function space. Now we can derive RazorWT terms by building *derivation trees* in a bottom-up way. At each step we check whether the types match – the tree has to be built in a well-typed way. The derivation of our example term

ite {Bool} (isZero (num 0 + num 1)) false (isZero (num 0))

is depicted as follows.

$$\frac{\dfrac{\dfrac{\dfrac{}{\text{num 0 : Tm Nat}} \qquad \dfrac{}{\text{num 1 : Tm Nat}}}{\text{num 0 + num 1 : Tm Nat}}}{\text{isZero (num 0 + num 1) : Tm Bool}} \qquad \dfrac{}{\text{false : Tm Bool}} \qquad \dfrac{\dfrac{}{\text{num 0 : Tm Nat}}}{\text{isZero (num 0) : Tm Bool}}}{\text{ite (isZero (num 0 + num 1)) false (isZero (num 0)) : Tm Bool}}$$

Exercise 1.33 (compulsory)**.** *Draw the derivation trees of the following terms.*

isZero (num 1 + ite true (num 2) (num 3))
isZero (num 1 + (num 2 + num 3))
isZero ((num 1 + num 2) + num 3)
ite (isZero (num 0)) (num 1 + num 2) (num 3)

Exercise 1.34 (recommended)**.** *Implement the two different leaf count functions and using a dependent model show that they give the same result. See Section 1.3.2.*

1.4.1 Type inference

The goal of type inference is to turn a syntactic RazorAST term into a well-typed (RazorWT) term. The depiction of this process is that we try to build a derivation tree, but we might fail at some point.

Precisely, we will use the RazorAST-iterator on a RazorAST model built out of RazorWT terms. This is not really possible because e.g. RazorAST requires us to add any two terms using _+_, but the addition that we have only works for terms of type Nat. Terms in our RazorAST algebra are either a pair of a RazorWT type and a term of that type or a failure. We use Maybe

to express possible failure: just (A, t) and nothing are both in the set Maybe (Σ I.Ty I.Tm), the latter expresses failure. Failure propagates through the operations ite, isZero and _+_.

```
Inf : RazorAST.Model
Inf = record
  { Tm     = Maybe (Σ I.Ty I.Tm)
  ; true   = just (I.Bool , I.true)
  ; false  = just (I.Bool , I.false)
  ; ite    = ite
  ; num    = λ n → just (I.Nat , I.num n)
  ; isZero = isZero
  ; _+o_   = _+o_
  }
  where
    ite : Maybe (Σ I.Ty I.Tm) → Maybe (Σ I.Ty I.Tm) → Maybe (Σ I.Ty I.Tm) → Maybe (Σ I.Ty I.Tm)
    ite (just (I.Bool , t)) (just (I.Nat  , u)) (just (I.Nat  , v)) = just (I.Nat , I.ite t u v)
    ite (just (I.Bool , t)) (just (I.Bool , u)) (just (I.Bool , v)) = just (I.Bool , I.ite t u v)
    ite _                   _                   _                   = nothing

    isZero : Maybe (Σ I.Ty I.Tm) → Maybe (Σ I.Ty I.Tm)
    isZero (just (I.Nat , t)) = just (I.Bool , I.isZero t)
    isZero _                  = nothing

    _+o_ : Maybe (Σ I.Ty I.Tm) → Maybe (Σ I.Ty I.Tm) → Maybe (Σ I.Ty I.Tm)
    just (I.Nat , t) +o just (I.Nat , t') = just (I.Nat , (t I.+o t'))
    _                +o _                 = nothing
module Inf = RazorAST.Model Inf
```

Interpretation into this model gives us type inference.

```
infer : RazorAST.I.Tm → Maybe (Σ I.Ty I.Tm)
infer = Inf.⟦_⟧
```

Exercise 1.35 (compulsory)**.** *Here are some lists of lexical elements (written as strings). They are all accepted by the parser. Which ones are rejected by type inference? For the ones which are accepted, write down the RazorWT terms which are produced.*

if true then true else num 0
if true then num 0 else num 0
if num 0 then num 0 else num 0
if num 0 then num 0 else true
true + zero
true + num 1
true + isZero false
true + isZero (num 0)

Exercise 1.36 (compulsory)**.** *Here are some RazorAST terms. Which ones are rejected by type inference? For the ones which are accepted, write down the RazorWT terms which are produced.*

ite true true (num 0)
ite true (num 0) (num 0)
ite (num 0) (num 0) (num 0)
true +o zero
true +o num 1
true +o isZero (num 1)
isZero (num 1) +o num 0

1.4.2 Standard interpretation

We define the standard model (set model, denotational semantics). The idea is that all operators in our object language are given by their metatheoretic counterparts.

```
St : Model
St = record
  { Ty     = Set
  ; Tm     = λ T → T
  ; Nat    = ℕ
  ; Bool   = 2
  ; true   = tt
  ; false  = ff
  ; ite    = if_then_else_
  ; num    = λ n → n
  ; isZero = λ { zero → tt ; _ → ff }
  ; _+o_   = _+_
  }
module St = Model St
```

Using the iterator we get an interpreter (the metacircular interpreter, evaluator, normaliser) for our language:

```
norm : ∀{A} → I.Tm A → St.⟦ A ⟧T
norm = St.⟦_⟧t
```

We were not able to define the standard interpreter for RazorAST because there we had meaningless programs. For example, we would have had to interpret both Bool and Nat by the same set (c.f. Exercise 1.18).

1.4.3 Typing relation

The traditional way of defining the well-typed syntax is by defining a binary relation between syntactic types and syntactic RazorAST terms (see e.g. [11]). This relation is defined inductively as follows.

```
module I' = RazorAST.I
data _⦂_ : I'.Tm → I.Ty → Prop where
  true   :
                        ----------------------
                        I'.true ⦂ I.Bool
  false  :
                        ----------------------
                        I'.false ⦂ I.Bool
  ite    : ∀{t u v A} →
                        t ⦂ I.Bool →
                        u ⦂ A      →
                        v ⦂ A      →
                        ----------------------
                        I'.ite t u v ⦂ A
  num    : ∀{n} →
                        ----------------------
                        I'.num n ⦂ I.Nat
  isZero : ∀{t} →
                        t ⦂ I.Nat  →
                        ----------------------
                        I'.isZero t ⦂ I.Bool
```

```
_+o_  : ∀{u v} →
                u ⦂ I.Nat  →
                v ⦂ I.Nat  →
                ------------------------
                (u I'.+o v) ⦂ I.Nat
```

Exercise 1.37. *Show that there is an isomorphism between* I.Tm A *and* Σ I'.Tm (_⦂ A).

We defined well-typed terms directly because we do not want to talk about non-well-typed terms in the same way as we are not interested in programs with non-matching brackets.

1.5 Well-typed syntax with equations

In the well-typed syntax, we still had too many terms in a sense: for example, the terms I.true and I.isZero (I.num 0) were different, even if they should be equal in a sensible semantics. The information which terms should be the same (usually this is called operational semantics) was missing. Now we add this information in the form of equalities.

A Razor model (without any qualifiers) consists of the following components. The only difference from RazorWT is the addition of the five equations.

```
record Model {ℓ ℓ'} : Set (lsuc (ℓ ⊔ ℓ')) where
  field
    Ty       : Set ℓ
    Tm       : Ty → Set ℓ'
    Nat      : Ty
    Bool     : Ty
    true     : Tm Bool
    false    : Tm Bool
    ite      : {A : Ty} → Tm Bool → Tm A → Tm A → Tm A
    num      : ℕ → Tm Nat
    isZero   : Tm Nat → Tm Bool
    _+o_     : Tm Nat → Tm Nat → Tm Nat
    iteβ₁    : ∀{A}{u v : Tm A} → ite true u v ≡ u
    iteβ₂    : ∀{A}{u v : Tm A} → ite false u v ≡ v
    isZeroβ₁ : isZero (num 0) ≡ true
    isZeroβ₂ : {n : ℕ} → isZero (num (1 + n)) ≡ false
    +β       : {m n : ℕ} → num m +o num n ≡ num (m + n)
```

The iterator is given as before.

```
⟦_⟧T        : I.Ty → Ty
⟦ I.Nat ⟧T  = Nat
⟦ I.Bool ⟧T = Bool

postulate
  ⟦_⟧t      : {A : I.Ty} → I.Tm A → Tm ⟦ A ⟧T
  ⟦true⟧    :          ⟦ I.true          ⟧t ≈ true
  ⟦false⟧   :          ⟦ I.false         ⟧t ≈ false
  ⟦ite⟧     : {A : I.Ty}{t : I.Tm I.Bool}{u v : I.Tm A} →
                       ⟦ I.ite t u v ⟧t ≈ ite ⟦ t ⟧t ⟦ u ⟧t ⟦ v ⟧t
  ⟦num⟧     : {n : ℕ} → ⟦ I.num n   ⟧t ≈ num n
  ⟦isZero⟧  : ∀{t}    → ⟦ I.isZero t ⟧t ≈ isZero ⟦ t ⟧t
  ⟦+o⟧      : ∀{u v}  → ⟦ u I.+o v  ⟧t ≈ ⟦ u ⟧t +o ⟦ v ⟧t
```

We also have dependent models and an eliminator.

Now we can reproduce the evaluation of the example program from the beginning of Chapter 1 as an equality between terms.

```
ite (isZero (num 0 +o num 1)) false (isZero (num 0))
  ≡⟨ cong (λ x → ite (isZero x) false (isZero (num 0))) +β ⟩
ite (isZero (num 1))          false (isZero (num 0))
  ≡⟨ cong (λ x → ite x false (isZero (num 0))) isZeroβ₂ ⟩
ite false                     false (isZero (num 0))
  ≡⟨ iteβ₂ ⟩
isZero (num 0)
  ≡⟨ isZeroβ₁ ⟩
true
  ∎
```

Two terms which result in the same boolean or number are equal.

The type inference algorithm given for RazorWT also works for Razor. We can't calculate the height of a term anymore as before because of the equalities. Every function has to preserve the equalities. For example, we need that height (isZero (num 0)) = height true. If we view these terms as syntax trees, their height is 1 and 0, respectively.

1.5.1 Standard model and equational consistency

The term ite (isZero (num 0 +o num 1)) false (isZero (num 0)) was built using the operators of our language. Every model supports these operations, hence the term can be built in any model. The equality ite (isZero (num 0 +o num 1)) false (isZero (num 0)) = true was built using the equations in our language which hold in every model. Hence the equality also holds in every model. Such a term or equation is called *derivable*. A derivable term can be constructed in any model, a derivable equation holds in every model. In contrast, there are properties that we can only prove using induction: these hold in the syntax, but might not hold for every model. For example, the fact that for any t : Tm Bool we have t = true or t = false holds in the syntax (this is a consequence of normalisation, see below) but not in an arbitrary model. Such properties are called *admissible*.

Another property of the syntax is that true ≠ false. We can show this using the standard model.

The operators in the standard model are defined exactly as for RazorWT, but we also have to prove the five equalities iteβ₁, ..., +β: they all hold by definition.

```
St : Model
St = record
  { Ty        = Set
  ; Tm        = λ A → A
  ; Nat       = ℕ
  ; Bool      = 2̃
  ; true      = tt
  ; false     = ff
  ; ite       = if_then_else_
  ; num       = λ n → n
  ; isZero    = iteℕ tt (λ _ → ff)
  ; _+o_      = _+_
  ; iteβ₁     = refl
  ; iteβ₂     = refl
  ; isZeroβ₁  = refl
  ; isZeroβ₂  = refl
  ; +β        = refl
  }
module St = Model St
```

From the standard model we obtain *equational consistency* of our language, that is, there are terms which are not equal. This means that we didn't add too many equalities when defining the language. In the syntax, if true was equal to false, then their standard interpretations would also be equal, but then tt would be equal to ff.

```
true≠false : ¬ (I.true ≡ I.false)
true≠false e = tt≠ff (cong St.⟦_⟧t e)
```

If we are not careful when defining the equations of the language, it is easy to lose equational consistency, and then our language becomes useless, as any two terms will be equal.

As for any language, we can define a trivial model.

```
Triv : Model
Triv = record
  { Ty       = Lift ⊤
  ; Tm       = λ _ → Lift ⊤
  ; Nat      = mk triv
  ; Bool     = mk triv
  ; true     = mk triv
  ; false    = _
  ; ite      = _
  ; num      = _
  ; isZero   = _
  ; _+o_     = _
  ; isZeroβ₁ = refl
  ; isZeroβ₂ = refl
  ; +β       = refl
  ; iteβ₁    = refl
  ; iteβ₂    = refl
  }
module Triv = Model Triv
```

In the trivial model, any two terms are equal, hence all the equalities that can be described in the language, hold. For example, in this model we not only have true = false, but also Nat = Bool and true = num 5 (this equality only makes sense because we know that Nat = Bool). We also have that for any t : Tm Bool, t = true or t = false, trivially.

Exercise 1.38 (recommended)**.** *Construct a model in which it does not hold that for a* t : Tm Bool *we have* t = true *or* t = false*.*

Exercise 1.39 (compulsory)**.** *Show that if* true = false *in a model, then any two* u, v : Tm Nat *are equal in that model. A consequence is that if a compiler compiles true and false to the same code, then it compiles all natural numbers to the same code.*

1.5.2 Normalisation

Normalisation essentially says that the language can also be defined without equations. That is, there is a subset of syntactic terms called *normal forms* which are pairwise distinct and every syntactic term is equal to a normal form. Syntactic terms are quotiented by the equations of the language and normalisation implies that there is a function which selects a representative from each equivalence class. For Razor, there are two normal forms of type Bool (true and false) and countably many normal forms of type Nat (num n for each n : ℕ). Thus we simply define normal forms of type A by interpreting the A in the standard model, together with an injection ⌜_⌝ (called quotation) into syntactic terms.

```
Nf : I.Ty → Set
Nf A = St.⟦ A ⟧T

⌜_⌝ : ∀ {A} → St.⟦ A ⟧T → I.Tm A
⌜_⌝ {I.Bool} tt = I.true
⌜_⌝ {I.Bool} ff = I.false
⌜_⌝ {I.Nat} = I.num
```

The normalisation function is simply interpretation into the standard model:

```
norm : {A : I.Ty} → I.Tm A → Nf A
norm = St.⟦_⟧t
```

Completeness of normalisation says that every term is equal to its normal form (followed by quote). In our case, this means that every boolean term (element of I.Tm I.Bool) is either true or false and every natural number term is a finite application of I.sucs on I.zero. We first show that quote commutes with the operations ite, isZero and _+_.

```
⌜ite⌝ : ∀{b A}{u v : St.⟦ A ⟧T} → ⌜ if b then u else v ⌝ ≡ I.ite {A} ⌜ b ⌝ ⌜ u ⌝ ⌜ v ⌝
⌜ite⌝ {tt}{A} = I.iteβ₁ ⁻¹
⌜ite⌝ {ff}{A} = I.iteβ₂ ⁻¹

⌜isZero⌝ : ∀{n} → ⌜ iteℕ tt (λ _ → ff) n ⌝ ≡ I.isZero ⌜ n ⌝
⌜isZero⌝ {zero}  = I.isZeroβ₁ ⁻¹
⌜isZero⌝ {suc n} = I.isZeroβ₂ ⁻¹

⌜+⌝ : ∀{m n} → ⌜ m + n ⌝ ≡ ⌜ m ⌝ I.+o ⌜ n ⌝
⌜+⌝ {m}{n} = I.+β ⁻¹
```

Now we can prove completeness of normalisation by induction on terms.

```
Comp : DepModel
Comp = record
  { Ty•       = λ _ → Lift ⊤
  ; Tm•       = λ _ t → Lift (⌜ St.⟦ t ⟧t ⌝ ≡ t)
  ; Nat•      = _
  ; Bool•     = _
  ; num•      = λ n → mk refl
  ; isZero•   = λ {t} t• → mk (⌜isZero⌝ ■ cong I.isZero (un t•))
  ; _+o•_     = λ {u}{v} u• v• → mk (
                (⌜+⌝ {St.⟦ u ⟧t}{St.⟦ v ⟧t}) ■
                (cong₂ I._+o_ (un u•) (un v•)))
  ; true•     = mk refl
  ; false•    = mk refl
  ; ite•      = λ {A}{A•}{t}{u}{v} t• u• v• → mk (
                (⌜ite⌝ {St.⟦ t ⟧t}{A}{St.⟦ u ⟧t}{St.⟦ v ⟧t}) ■
                (cong₃ I.ite (un t•) (un u•) (un v•)))
  ; iteβ₁•    = refl
  ; iteβ₂•    = refl
  ; isZeroβ₁• = refl
  ; isZeroβ₂• = refl
  ; +β•       = refl
  }
module Comp = DepModel Comp

comp : {A : I.Ty}{t : I.Tm A} → ⌜ norm {A} t ⌝ ≡ t
comp {t = t} = un Comp.⟦ t ⟧t
```

Stability of normalisation says that there is no junk in normal forms, for every normal form there is a term which normalises to it.

```
stab : {A : I.Ty}{t : St.⟦ A ⟧T} → norm {A} ⌜ t ⌝ ≡ t
stab {I.Nat} {t} = refl
stab {I.Bool} {tt} = refl
stab {I.Bool} {ff} = refl
```

We used all the equations of Razor in the completeness proof. If we take out e.g. the rule isZeroβ_2 from the definition of the language, we cannot prove completeness anymore because we

have elements of I.Tm I.Bool which are neither I.true, nor I.false. The following (Razor without isZeroβ_2) model distinguishes true, false and isZero (num 1).

```
module RazorIncomplete where
  Ty       : Set₁
  Tm       : Ty → Set
  Nat      : Ty
  Bool     : Ty
  num      : ℕ → Tm Nat
  isZero   : Tm Nat → Tm Bool
  _+o_     : Tm Nat → Tm Nat → Tm Nat
  true     : Tm Bool
  false    : Tm Bool
  ite      : ∀{A} → Tm Bool → Tm A → Tm A → Tm A
  isZeroβ₁ : isZero (num 0) ≡ true
  +β       : ∀{m n} → num m +o num n ≡ num (m + n)
  iteβ₁    : ∀{A}{u v : Tm A} → ite true  u v ≡ u
  iteβ₂    : ∀{A}{u v : Tm A} → ite false u v ≡ v

  Ty                = Set
  Tm A              = A
  Nat               = ℕ
  Bool              = Maybe 2
  num               = λ n → n
  isZero zero       = just tt
  isZero (suc _)    = nothing
  _+o_              = _+_
  true              = just tt
  false             = just ff
  ite nothing u v   = u
  ite (just tt) u v = u
  ite (just ff) u v = v
  isZeroβ₁          = refl
  +β {m}{n}         = refl
  iteβ₁ {A}{u}{v}   = refl
  iteβ₂ {A}{u}{v}   = refl

  true≠false     : ¬ (true  ≡ false)
  true≠isZero1   : ¬ (true  ≡ isZero (num 1))
  false≠isZero1  : ¬ (false ≡ isZero (num 1))
```

Exercise 1.40. *Define normalisation and prove its completeness using a modified standard model where* Bool = Maybe 2*. Show that stability does not hold.*

Exercise 1.41 (compulsory)**.** *Show that* I.true ≠ I.false*, but there are* t, u, v *such that* I.true = I.ite t u v*. Why doesn't the RazorAST-method for proving disjointness of syntactic operators work?*

Exercise 1.42 (compulsory)**.** *Show that* I.num *is injective, but* I.isZero *is not injective. Why doesn't the RazorAST-method for proving injectivity of syntactic operators work?*

Exercise 1.43 (compulsory)**.** *Show that in the syntax we have* ite t u u = u *for any* t, u*, and construct a model in which this doesn't hold.*

Exercise 1.44 (recommended)**.** *Define a height function on terms which returns the height of the normal form of the term.*

Exercise 1.45 (recommended). *A model of a monoid over* A *has the following components.*

$$
\begin{aligned}
C &: Set \\
f &: A \to C \\
\circ &: C \to C \to C \\
id &: C \\
ass &: (a \circ b) \circ c \equiv a \circ (b \circ c) \\
idl &: id \circ a \equiv a \\
idr &: a \circ id \equiv a
\end{aligned}
$$

The initial model is called the free monoid over A.

Define normalisation and prove its completeness and stability for the free monoid over a set A. *Normal forms are simply lists of* A.

1.5.3 Transition relation

The traditional way of defining equality of syntactic terms is using transition relations on syntactic RazorAST terms (see e.g. [11]).

```
module SmallStep where
  import RazorAST
  module I' = RazorAST.I
  data _↦_ : I'.Tm → I'.Tm → Prop where
    isZeroβ₁     :              I'.isZero (I'.num 0) ↦ I'.true
    isZeroβ₂     : ∀{n} →       I'.isZero (I'.num (1 + n)) ↦ I'.false
    +β           : ∀{m n} → (I'.num m I'.+o I'.num n) ↦ I'.num (m + n)
    iteβ₁        : ∀{u v} → I'.ite I'.true  u v ↦ u
    iteβ₂        : ∀{u v} → I'.ite I'.false u v ↦ v

    isZero-cong : ∀{t t'}      → t ↦ t' → I'.isZero t   ↦ I'.isZero t'
    +-cong₁      : ∀{u v u'}   → u ↦ u' → (u I'.+o v)   ↦ (u' I'.+o v)
    +-cong₂      : ∀{u v v'}   → v ↦ v' → (u I'.+o v)   ↦ (u I'.+o v')
    ite-cong     : ∀{t t' u v} → t ↦ t' → (I'.ite t u v) ↦ (I'.ite t' u v)
```

The idea is that if there is a $\mathsf{p} : \mathsf{t} \mapsto \mathsf{t'}$, this means that the program t evaluates to $\mathsf{t'}$ in one step. Running a program means that we have a sequence of evaluation steps: $\mathsf{t} \mapsto \mathsf{t'} \mapsto \mathsf{t''} \mapsto \cdots$

There is a rewriting rule corresponding to each equation in the notion of Razor-model. Then there are congruence rules (order rules) which express which parameters and when should be evaluated. In our example rewriting system above rewriting can happen anywhere except for I'.ite it only happens in the first parameter.

Taking the reflexive transitive closure of $\mapsto$ we obtain the multi-step rewriting relation $\mapsto^*$, taking the symmetric closure of $\mapsto^*$ we get the conversion relation (definitional equality) $\sim$ which in the case of Razor, corresponds to equality of syntactic terms.

Exercise 1.46. *Prove that* $\mathsf{t} \equiv \mathsf{t'}$ *is logically equivalent to* $\mathsf{t} \sim \mathsf{t'}$ *for all* t *and* $\mathsf{t'}$.

For other languages, this is not necessarily the case. In general, the transition system contains more information than the description with equations. Transition is directed and is not necessarily a congruence. In these course notes we only present languages where there is no such additional information.

1.5.4 Another example of normalisation: integers

In this subsection, we define integers as the initial model of an algebraic theory. We illustrate the concepts of model, syntax, iterator, dependent model, induction, normalisation, completeness and stability of normalisation. The representation of integers is from [3].

An integer model has five components, a set, a zero element, a successor operation, a predecessor operation and two equalities saying that successor after or before predecessor is the identity.

```
record Model {ℓ} : Set (lsuc ℓ) where
  field
    Z       : Set ℓ
    Zero    : Z
    Suc     : Z → Z
    Pred    : Z → Z
    SucPred : (i : Z) → Suc (Pred i) ≡ i
    PredSuc : (i : Z) → Pred (Suc i) ≡ i
```

We have an iterator, that is a homomorphism from the initial model to any model.

```
postulate
  ⟦_⟧     : I.Z → Z
  ⟦Zero⟧  :          ⟦ I.Zero   ⟧ ≈ Zero
  ⟦Suc⟧   : ∀{i} → ⟦ I.Suc i  ⟧ ≈ Suc ⟦ i ⟧
  ⟦Pred⟧  : ∀{i} → ⟦ I.Pred i ⟧ ≈ Pred ⟦ i ⟧
```

Integers themselves are given by the initial integer model.

```
ℤ = I.Z
```

There are multiple representations of each number, e.g. −1 is given by Pred Zero, Pred (Suc (Pred Zero)), and so on.

Examples of equalities which hold in any integer model.

```
module examples {ℓ}(M : Model {ℓ}) where
  open Model M

  eq1 : Pred (Suc (Pred Zero)) ≡ Pred Zero
  eq1 = PredSuc (Pred Zero)

  eq2 : Pred (Suc (Suc (Pred Zero))) ≡ Zero
  eq2 = Pred (Suc (Suc (Pred Zero)))
                                    ≡⟨ PredSuc (Suc (Pred Zero)) ⟩
        Suc (Pred Zero)
                                    ≡⟨ SucPred Zero ⟩
        Zero                        ∎
```

Dependent models and induction.

The model M i where Zero is i, everything else comes from the syntax.

```
M : I.Z → Model
M i = record
    { Z       = I.Z
    ; Zero    = i
    ; Suc     = I.Suc
    ; Pred    = I.Pred
    ; SucPred = I.SucPred
    ; PredSuc = I.PredSuc
    }
module M i = Model (M i)
```

Now we have addition j +ℤ i which replaces I.Zero inside j by i:

```
_+ℤ_ : I.Z → I.Z → I.Z
j +ℤ i = M.⟦_⟧ i j
```

```
test+ : {i : I.Z} → I.Suc (I.Suc (I.Pred I.Zero)) +ℤ i ≡ I.Suc (I.Suc (I.Pred i))
test+ {i} = refl

test+' : {i : I.Z} → I.Pred (I.Suc (I.Suc (I.Pred I.Zero))) +ℤ i ≡ I.Pred (I.Suc (I.Suc (I.Pred i)))
test+' {i} = refl
```

Integers were defined using a language with equations. Normal forms of integers are defined by another language without equations. Normalisation means that the syntaxes of these two languages are in bijection: there is a bijection between I.Z and Nf (we don't use other models of the language of normal forms, we only work with its syntax Nf).

Normal forms and examples.

```
data Nf : Set where
  -Suc  : ℕ → Nf
  Zero  : Nf
  +Suc : ℕ → Nf

minusThree minusFive plusSix : Nf
minusThree = -Suc 2
minusFive  = -Suc 4
plusSix    = +Suc 5

SucNf : Nf → Nf
SucNf (-Suc zero)      = Zero
SucNf (-Suc (suc n))   = -Suc n
SucNf Zero             = +Suc 0
SucNf (+Suc n)         = +Suc (suc n)

PredNf : Nf → Nf
PredNf (-Suc n)        = -Suc (suc n)
PredNf Zero            = -Suc 0
PredNf (+Suc zero)     = Zero
PredNf (+Suc (suc n)) = +Suc n
```

Normalisation.

```
N : Model
N = record
  { Z    = Nf
  ; Zero = Zero
  ; Suc  = SucNf
  ; Pred = PredNf
  ; SucPred = λ { (-Suc zero)      → refl
                ; (-Suc (suc n))   → refl
                ; Zero             → refl
                ; (+Suc zero)      → refl
                ; (+Suc (suc n))   → refl
                }
  ; PredSuc = λ { (-Suc zero)      → refl
                ; (-Suc (suc n))   → refl
                ; Zero             → refl
                ; (+Suc zero)      → refl
                ; (+Suc (suc n))   → refl }
  }
module N = Model N

norm : I.Z → Nf
norm = N.⟦_⟧
```

Quoting normal forms into integers:

```
⌜_⌝ : Nf → I.Z
⌜ -Suc zero     ⌝ = I.Pred I.Zero
⌜ -Suc (suc n)  ⌝ = I.Pred ⌜ -Suc n ⌝
⌜ Zero          ⌝ = I.Zero
⌜ +Suc zero     ⌝ = I.Suc I.Zero
⌜ +Suc (suc n)  ⌝ = I.Suc ⌜ +Suc n ⌝
```

A unit test for normalisation:

```
testnorm : ⌜ norm (I.Pred (I.Pred (I.Suc (I.Pred (I.Pred (I.Pred (I.Suc I.Zero))))))) ⌝ ≡
           I.Pred (I.Pred (I.Pred I.Zero))
testnorm = refl
```

Stability of normalisation:

```
stab : (v : Nf) → norm ⌜ v ⌝ ≡ v
stab (-Suc zero)      = refl
stab (-Suc (suc n))   = cong PredNf (stab (-Suc n))
stab Zero             = refl
stab (+Suc zero)      = refl
stab (+Suc (suc n))   = cong SucNf (stab (+Suc n))
```

Helper lemmas for completeness:

```
⌜Suc⌝ : (v : Nf) → ⌜ SucNf v ⌝ ≡ I.Suc ⌜ v ⌝
⌜Suc⌝ (-Suc zero)      = I.SucPred I.Zero ⁻¹
⌜Suc⌝ (-Suc (suc n))   = I.SucPred ⌜ -Suc n ⌝ ⁻¹
⌜Suc⌝ Zero             = refl
⌜Suc⌝ (+Suc n)         = refl

⌜Pred⌝ : (v : Nf) → ⌜ PredNf v ⌝ ≡ I.Pred ⌜ v ⌝
⌜Pred⌝ (-Suc n)        = refl
⌜Pred⌝ Zero            = refl
⌜Pred⌝ (+Suc zero)     = I.PredSuc I.Zero ⁻¹
⌜Pred⌝ (+Suc (suc n))  = I.PredSuc ⌜ +Suc n ⌝ ⁻¹
```

Completeness of normalisation:

```
Comp : DepModel
Comp = record
  { Z•     = λ i → Lift (⌜ norm i ⌝ ≡ i)
  ; Zero•  = mk refl
  ; Suc•   = λ {i} i• → mk (⌜Suc⌝ N.⟦ i ⟧ ∎ cong I.Suc (un i•))
  ; Pred•  = λ {i} i• → mk (⌜Pred⌝ N.⟦ i ⟧ ∎ cong I.Pred (un i•))
  ; SucPred• = λ _ → refl
  ; PredSuc• = λ _ → refl
  }
module Comp = DepModel Comp

comp : (i : I.Z) → ⌜ norm i ⌝ ≡ i
comp i = un (Comp.⟦ i ⟧)
```

Exercise 1.47. *Prove that integers form a commutative monoid with addition.*

Exercise 1.48. *Define multiplication and prove that integers form a commutative ring with addition and multiplication.*

Chapter 2

Bindings

Learning goals of this chapter

Local definitions. Binding, scope, free and bound variables, De Bruijn indices. Abstract binding trees. Contexts, substitutions. Constructors, destructors, computation and uniqueness rules. Main arguments of a destructor. Neutral terms and normal forms, normalisation by induction on terms. Decidability of equality of terms from normalisation. Injectivity and disjointness of constructors but not destructors.

In this chapter we extend Razor with local definitions. For example, we will be able to write $\mathsf{def\ x := 1 + 2\ in\ x + x}$ which we read as "define x to be $1 + 2$ inside $\mathsf{x + x}$", and it will be equal to $(1 + 2) + (1 + 2) = 3 + 3 = 6$.

In general, names are *bound* by symbols which we call *binders* and each binder has a *scope*. Examples from mathematics: in the expression $\sum_{x=1}^{10} x^2$, the name x is bound by the binder $\sum$ in the scope of the binder which is the expression x^2. In $\lim_{x \mapsto \infty} 2^{-x}$, lim binds x in 2^{-x}. In $\int_0^{10} 2x^2 + x - 1 dx$, $\int \ldots d$ binds x in $2x^2 + x - 1$. In $\forall x, x + 3 = 3 + x$, $\forall$ binds x in $x + 3 = 3 + x$. In `int f(int i) { return(i+1); }`, the function definition binds `i` in the function body (the parts between { and }). In $\mathsf{def\ x := 1 + 2\ in\ x + x}$, x is bound by def in $x + x$.

Variables are sometimes called identifiers or simply names. Note that this notion of variable is not the same as a mutable reference which is another concept sometimes called variable in programming languages.

The most direct way to formalise binding is to reuse the function space of our metatheory and add the following new operation to Razor.

$$\mathsf{def} : \{\mathsf{A\ B} : \mathsf{Ty}\} \to \mathsf{Tm\ A} \to (\mathsf{Tm\ A} \to \mathsf{Tm\ B}) \to \mathsf{Tm\ B}$$

which is more verbosely

$$\mathsf{def} : \{\mathsf{A\ B} : \mathsf{Ty}\}(\mathsf{u} : \mathsf{Tm\ A})(\mathsf{t} : (\mathsf{x} : \mathsf{Tm\ A}) \to \mathsf{Tm\ B}) \to \mathsf{Tm\ B}$$

and using derivation rule style notation

$$\frac{\mathsf{u} : \mathsf{Tm\ A} \qquad \mathsf{x} : \mathsf{Tm\ A} \vdash \mathsf{t\ x} : \mathsf{Tm\ B}}{\mathsf{def\ t\ u} : \mathsf{Tm\ B}}$$

where the horizontal line and $\vdash$ are both notations for metatheoretic function space. We also add the equation

$$\mathsf{def\ u\ t = t\ u} \qquad \text{which is more verbosely} \qquad \mathsf{def\ u\ (\lambda\ x \to t\ x) = t\ u}$$

and says that letting the dependency in t be u is the same as applying the function t to the value u.

Local definitions are good for making programs shorter and easier to read, e.g. instead of $(\mathsf{num\ 1 + num\ 2}) + (\mathsf{num\ 1 + num\ 2})$ we can write the shorter $\mathsf{def\ (num\ 1 + num\ 2)\ (\lambda\ x \to x + x)}$. In general, instead of t u we can write def u t.

This way of formalising bindigs is sometimes called higher-order abstract syntax or logical framework (LF) style. The issue with it is that it does not provide notions of homomorphism and thus syntax. The def operation is a *second-order operation*, and algebraic theories don't allow second-order operations, so the language extended with def is not an algebraic theory, it is only a second-order algebraic theory.

However there are ways to translate second-order algebraic theories to first order ones. The most popular way is adding a new sort of *contexts* to the language which collects the names that the binders introduced so far. Another way is using Schönfinkel's combinators.

2.1 Making bindings first order via contexts

We will show how to introduce contexts for the AST, well-typed and well-typed quotiented description of the language.

2.1.1 Abstract binding trees

At the level of abstract syntax trees, we add names and def expressions:

N ::= x | y | z | ...
T ::= N | def N := T in T | true | false | ite T T T | ...

Multiple binders can be used in one expression:

(def x := t in x + x) + (def y := t' in y + y)

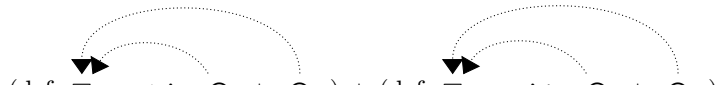
The names of the bound variables have no significance. The above term should be equal to

(def y := t in y + y) + (def x := t' in x + x)

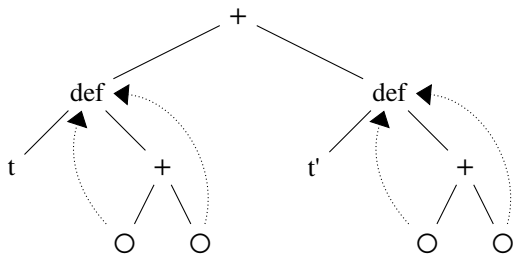
or

(def z := t in z + z) + (def z := t' in z + z)

for any t, t'. These are different terms at the level of abstract syntax trees, however, they are equal at the level of *abstract binding trees* (ABTs). ABT is between levels (3) and (4) of Chapter 1, see Figure 1.1. Names are replaced by pointers to the binding. The above terms become the following.



In tree notation:

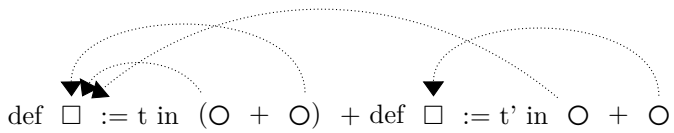


The scope of def is the longest possible, so the scope of both defs end at the end of the term.

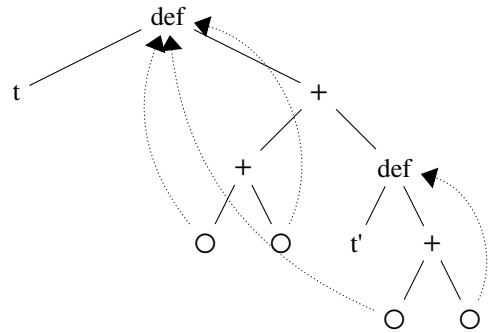
Another example:

def x := t in (x + x) + def y := t' in x + y

Binding structure:



In tree notation:



The term (def x := t in x + x) + (def y := t' in x + y) is different from both of the above terms, it contains a *free* name x in x + y. Free variables are not in the scope of a binder which binds the same name. A term is called *closed* if there are no free variables in it, *open* if it contains at least one free variable. Every open term can be closed by adding enough binders at the front.

One way to make abstract binding trees precise is to add the following equation:

(def x:=t in u) = (def y:=t in u[x↦y])

where u[x↦y] means that we replace all occurrences of x by y in u. However one has to be careful:

(def x:=t in x + y) ≠ (def y:=t in y + y) = (def y:=t in (x + y)[x↦y])

The left-hand side term is open, the right-hand side is closed, so they cannot have the same binding tree. In the equation above it has to be stated that y is fresh for u (u does not contain y). See e.g. [11, Section 1.2] for details on that approach.

Another approach to define abstract binding trees takes it seriously that the names of bound variables don't matter and does not write names at all. Instead, we write natural numbers (De Bruijn indices [4]). Examples:

(def x:=t in x + x) + (def y:=t' in y + y)	(def t in v0 + v0) + (def t' in v0 + v0)
def x:=t in (x + x) + def y:=t' in x + y	def t in (v0 + v0) + def t' in v1 + v0

v0 is a reference to the nearest binder, v1 is a reference to the next one, and so on.

Now terms are indexed by natural numbers expressing the maximal number of free variables in them.

- Tm 0 is the set of closed terms (programs).
- Tm 1 is the set of terms which can refer to one free variable.
- Tm 2 is the set of terms which can refer to two different free variables.
- Tm 3 is the set of terms which can refer to three different free variables.
- ...

Binders are those operators which decrease this number: e.g. def takes a Tm (1 + n) in which it binds the variable and returns a Tm n. Examples:

$$\text{if false then num 0 else num 3} : \text{Tm } 0$$

$$\text{def t } \underbrace{(\text{v0} + \text{v0})}_{:\ \text{Tm } 1} + \text{def t' } \underbrace{(\text{v0} + \text{v0})}_{:\ \text{Tm } 1} : \text{Tm } 0$$

$$\underbrace{\text{def t } ((\text{v0} + \text{v0}) + \text{def t' } \underbrace{(\text{v1} + \text{v0})}_{:\ \text{Tm } 2})}_{:\ \text{Tm } 1} : \text{Tm } 0$$

We have a separate set Var n of variables which are included in terms. A DefABT model:

```
record Model {ℓ ℓ'} : Set (lsuc (ℓ ⊔ ℓ')) where
  infixl 7 _+o_
  field
    Var : ℕ → Set ℓ
    vz : ∀{n} → Var (suc n)
    vs : ∀{n} → Var n → Var (suc n)

    Tm    : ℕ → Set ℓ'
    var   : ∀{n} → Var n → Tm n
    def   : ∀{n} → Tm n → Tm (suc n) → Tm n
    true  : ∀{n} → Tm n
    false : ∀{n} → Tm n
    ite   : ∀{n} → Tm n → Tm n → Tm n → Tm n
    num   : ∀{n} → ℕ → Tm n
    isZero : ∀{n} → Tm n → Tm n
    _+o_  : ∀{n} → Tm n → Tm n → Tm n
```

We have two iterators, one for each sort:

```
⟦_⟧v : ∀{n} → I.Var n → Var n
⟦ I.vz ⟧v = vz
⟦ I.vs v ⟧v = vs ⟦ v ⟧v

⟦_⟧t : ∀{n} → I.Tm n → Tm n
⟦ I.var x ⟧t = var ⟦ x ⟧v
⟦ I.def x t ⟧t = def ⟦ x ⟧t ⟦ t ⟧t
⟦ I.true ⟧t = true
⟦ I.false ⟧t = false
⟦ I.ite t tr fa ⟧t = ite ⟦ t ⟧t ⟦ tr ⟧t ⟦ fa ⟧t
⟦ I.num x ⟧t = num x
⟦ I.isZero t ⟧t = isZero ⟦ t ⟧t
⟦ l I.+o r ⟧t = ⟦ l ⟧t +o ⟦ r ⟧t
```

Some abbreviations:

```
v0 : {n : ℕ} → Tm (1 + n)
v0 = var vz
v1 : {n : ℕ} → Tm (2 + n)
v1 = var (vs vz)
v2 : {n : ℕ} → Tm (3 + n)
v2 = var (vs (vs vz))
v3 : {n : ℕ} → Tm (4 + n)
v3 = var (vs (vs (vs vz)))
```

Formal version of the term (def x:=1+2 in x+x) + (def y:=3+4 in y+y).

```
t = def (num 1 +o num 2) (v0 +o v0) +o def (num 3 +o num 4) (v0 +o v0)
```

Formal version of the term def x:=1+2 in ((x+x) + def y:=3+4 in x+y).

```
t' = def (num 1 +o num 2) ((v0 +o v0) +o def (num 3 +o num 4) (v1 +o v0))
```

Note that in our formal syntax we don't have variable names, so there is no function which extracts the names of the free variables from a term.

Exercise 2.1 (compulsory)**.** *In the following ABT-level syntactic terms, circle the free variables. From bound variables, draw a pointer to the binder.*

x + y

x + def x := 3 in y + x

x + def x := y in y + x

x + def x := x' + (x + def x' := z in x' + x)

3 + def x := x + (x' + def x' := 2 in x' + x)

Exercise 2.2 (compulsory). *Decide whether the following ABT-level syntactic terms are equal.*

x + y $\stackrel{?}{=}$ x + z
(x + def x := 3 in x + y) $\stackrel{?}{=}$ (x + def y := 3 in y + y)
(x + def x := 3 in x + y) $\stackrel{?}{=}$ (x + def y := 3 in y + x)
(x + def x := 3 in x + y) $\stackrel{?}{=}$ (x + def x' := 3 in x' + y)
(x + def x := 3 in def y := 4 in x + y) $\stackrel{?}{=}$ (x + def y := 3 in def x := 4 in x + y)
(x + def x := 3 in def y := 4 in x + y) $\stackrel{?}{=}$ (x + def y := 3 in def x := 4 in y + x)

Exercise 2.3 (compulsory). *Decide whether the following ABT-level syntactic terms are open or closed.*

def x := 3 in x + x
def x := y in x + x
def x := y in x + y
def y := x in x + y
def y := x in x + x

Exercise 2.4 (compulsory). *Rewrite the following (closed) terms with De Bruijn notation.*

def x:=1 in x + def y:=x+1 in y + def z:=x+y in (x+z)+(y+x)
(def x:=1 in x) + def y:=1 in y + def z:=1+y in z+(y+1)
(def x:=1 in x + def y:=x+1 in y) + def z:=1 in z+z
(def x:=1 in x) + (def y:=1 in y) + def z:=1 in z+z

Exercise 2.5 (compulsory). *Rewrite the following (closed) terms with variable name notation.*

```
t1 = def true (v0 +o def v0 (v0 +o v1))
t2 = def true (def false (ite v0 v0 v1))
t3 = true +o def true (false +o def v0 (v1 +o v0))
t4 = def true (def false (def true (def false ((v0 +o v1) +o (v2 +o v3)))))
```

Exercise 2.6 (recommended). *Write a function which returns the number of occurrences of each variable in a term:*

```
countVars : {n : ℕ} → Tm n → Vec ℕ n
```

For example it should work as follows:

```
countVarsTest1 : countVars {1} ((v0 +o v0) +o v0) ≡ 3 :: []
countVarsTest2 : countVars {2} ((v1 +o v1) +o v0) ≡ 1 :: 2 :: []
countVarsTest3 : countVars {2} ((v0 +o v0) +o v1) ≡ 2 :: 1 :: []
countVarsTest4 : countVars {3} ((v2 +o v0) +o v1) ≡ 1 :: 1 :: 1 :: []
```

2.1.2 Well-typed syntax

As a warmup for the real description of the language Def, we look at its well-typed description which does not include equations.

To obtain well-typed terms, we have to keep track not only of the maximal number of free variables that can occur in a term but also of their types. Thus we have a new sort of *contexts* which is a list of types. The empty context $\diamond$ denotes no free variables, the context $\diamond \triangleright \mathsf{A} \triangleright \mathsf{B} \triangleright \mathsf{C}$ has length three and the variables v0, v1, v2 have types C, B and A, respectively. Variables (and terms) are now indexed by their context and their type. The well-typed De Bruijn indices are recovered as follows.

```
v0 : {Γ : Con}{A : Ty}          → Tm (Γ ▷ A) A
v0 = v 0
v1 : {Γ : Con}{A B : Ty}        → Tm (Γ ▷ A ▷ B) A
v1 = v 1
v2 : {Γ : Con}{A B C : Ty}      → Tm (Γ ▷ A ▷ B ▷ C) A
v2 = v 2
v3 : {Γ : Con}{A B C D : Ty} → Tm (Γ ▷ A ▷ B ▷ C ▷ D) A
v3 = var (vs (vs (vs vz)))
v4 : {Γ : Con}{A B C D E : Ty} → Tm (Γ ▷ A ▷ B ▷ C ▷ D ▷ E) A
v4 = var (vs (vs (vs (vs vz))))
```

A DefWT model has the following components.

```
record Model {i j k l} : Set (lsuc (i ⊔ j ⊔ k ⊔ l)) where
  infixl 5 _▷_
  infixl 6 _+o_
  field
    Ty     : Set i
    Nat    : Ty
    Bool   : Ty

    Con    : Set j
    ◇      : Con
    _▷_    : Con → Ty → Con

    Var    : Con → Ty → Set k
    vz     : ∀{Γ A} → Var (Γ ▷ A) A
    vs     : ∀{Γ A B} → Var Γ A → Var (Γ ▷ B) A

    Tm     : Con → Ty → Set l
    var    : ∀{Γ A}     → Var Γ A → Tm Γ A
    def    : ∀{Γ A B} → Tm Γ A → Tm (Γ ▷ A) B → Tm Γ B
    true   : ∀{Γ}       → Tm Γ Bool
    false  : ∀{Γ}       → Tm Γ Bool
    ite    : ∀{Γ A}     → Tm Γ Bool → Tm Γ A → Tm Γ A → Tm Γ A
    num    : ∀{Γ}       → ℕ → Tm Γ Nat
    isZero : ∀{Γ}       → Tm Γ Nat → Tm Γ Bool
    _+o_   : ∀{Γ}       → Tm Γ Nat → Tm Γ Nat → Tm Γ Nat
```

The derivation rule style presentation of the new operators:

$$\frac{}{\mathsf{vz} : \mathsf{Var}\ (\Gamma \triangleright \mathsf{A})\ \mathsf{A}} \qquad \frac{\mathsf{x} : \mathsf{Var}\ \Gamma\ \mathsf{A}}{\mathsf{vs}\ \mathsf{x} : \mathsf{Var}\ (\Gamma \triangleright \mathsf{B})\ \mathsf{A}} \qquad \frac{\mathsf{x} : \mathsf{Var}\ \Gamma\ \mathsf{A}}{\mathsf{var}\ \mathsf{x} : \mathsf{Tm}\ \Gamma\ \mathsf{A}} \qquad \frac{\mathsf{u} : \mathsf{Tm}\ \Gamma\ \mathsf{A} \qquad \mathsf{t} : \mathsf{Tm}\ (\Gamma \triangleright \mathsf{A})\ \mathsf{B}}{\mathsf{def}\ \mathsf{t}\ \mathsf{u} : \mathsf{Tm}\ \Gamma\ \mathsf{B}}$$

Two example derivations:

$$\frac{\dfrac{\dfrac{}{\mathsf{num}\ 0 : \mathsf{Tm} \diamond \mathsf{Nat}}}{\mathsf{isZero}\ (\mathsf{num}\ 0) : \mathsf{Tm} \diamond \mathsf{Bool}} \qquad \dfrac{\dfrac{\dfrac{}{\mathsf{vz} : \mathsf{Var}\ (\diamond \triangleright \mathsf{Bool})\ \mathsf{Bool}}}{\mathsf{var}\ \mathsf{vz} : \mathsf{Tm}\ (\diamond \triangleright \mathsf{Bool})\ \mathsf{Bool}} \qquad \dfrac{}{\mathsf{num}\ 0 : \mathsf{Tm}\ (\diamond \triangleright \mathsf{Bool})\ \mathsf{Nat}} \qquad \dfrac{}{\mathsf{num}\ 1 : \mathsf{Tm}\ (\diamond \triangleright \mathsf{Bool})\ \mathsf{Nat}}}{\mathsf{ite}\ (\mathsf{var}\ \mathsf{vz})\ (\mathsf{num}\ 0)\ (\mathsf{num}\ 1) : \mathsf{Tm}\ (\diamond \triangleright \mathsf{Bool})\ \mathsf{Nat}}}{\mathsf{def}\ (\mathsf{isZero}\ (\mathsf{num}\ 0))\ (\mathsf{ite}\ (\mathsf{var}\ \mathsf{vz})\ (\mathsf{num}\ 0)\ (\mathsf{num}\ 1)) : \mathsf{Tm} \diamond \mathsf{Nat}}$$

$$
\dfrac{\dfrac{}{\text{num } 2 : \text{Tm} \diamond \text{Nat}} \qquad \dfrac{\dfrac{\dfrac{}{\text{vz} : \text{Var} (\diamond \triangleright \text{Nat}) \text{ Nat}}}{\text{var vz} : \text{Tm} (\diamond \triangleright \text{Nat}) \text{ Nat}} \qquad \dfrac{\dfrac{\dfrac{}{\text{vz} : \text{Var} (\diamond \triangleright \text{Nat} \triangleright \text{Nat}) \text{ Nat}}}{\text{var vz} : \text{Tm} (\diamond \triangleright \text{Nat} \triangleright \text{Nat}) \text{ Nat}} \qquad \dfrac{\dfrac{\dfrac{}{\text{vz} : \text{Var} (\diamond \triangleright \text{Nat}) \text{ Nat}}}{\text{vs vz} : \text{Var} (\diamond \triangleright \text{Nat} \triangleright \text{Nat}) \text{ Nat}}}{\text{var (vs vz)} : \text{Tm} (\diamond \triangleright \text{Nat} \triangleright \text{Nat}) \text{ Nat}}}{\text{var vz} + \text{var (vs vz)} : \text{Tm} (\diamond \triangleright \text{Nat} \triangleright \text{Nat}) \text{ Nat}}}{\text{isZero (var vz} + \text{var (vs vz))} : \text{Tm} (\diamond \triangleright \text{Nat} \triangleright \text{Nat}) \text{ Bool}}}{\text{def (var vz) (isZero (var vz} + \text{var (vs vz))) : Tm} (\diamond \triangleright \text{Nat}) \text{ Bool}}}{\text{def (num 2) (def (var vz) (isZero (var vz} + \text{var (vs vz)))) : Tm} \diamond \text{Bool}}
$$

Exercise 2.7 (compulsory). *Derive* def (var vz) true : Tm ($\diamond \triangleright$ Nat) Bool*!*

Exercise 2.8 (compulsory). *Derive* def (var (vs vz)) (var vz) : Tm ($\diamond \triangleright$ Bool $\triangleright$ Nat) Bool*!*

Exercise 2.9 (compulsory). *Derive* def (num 1) (var vz + var (vs vz)) : Tm ($\diamond \triangleright$ Nat) Nat*!*

Exercise 2.10 (compulsory). *What can be* t *such that* def t (def true (ite v0 v1 v1)) : Tm $\diamond$ Nat*?*

2.1.3 Well-typed syntax with equations

If we have local definitions, then in addition to equations such as isZero (num 0) = true, we need equations like the following.

def u (v0 + v0) = u + u
def u (def u' (v0 + v1)) = u' + u
def u (def u' (v1 + (v0 + (v1 + num 3)))) = u + (u' + (u + num 3))

In general, we would like to have def u t = t [⟨ u ⟩] where t [⟨ u ⟩] is the same as t, but all v0s are replaced by u, all v1s are replaced by v0s, all v2s are replaced by v1s, and so on.

As a first try we add a new operation $_[_]_0$ which instantiates the last variable (v0) with a term:

$$\frac{\text{t} : \text{Tm} (\Gamma \triangleright \text{B}) \text{ A} \qquad \text{w} : \text{Tm } \Gamma \text{ B}}{\text{t [w]}_0 : \text{Tm } \Gamma \text{ A}}$$

and we add the following equations

var vz [w]$_0$ = w
var (vs x) [w]$_0$ = var x
true [w]$_0$ = true
false [w]$_0$ = false
ite t u v [w]$_0$ = ite (t [w]$_0$) (u [w]$_0$) (v [w]$_0$)
num n [w]$_0$ = num n
isZero t [w]$_0$ = isZero (t [w]$_0$)
u +o v [w]$_0$ = (u [w]$_0$) +o (v [w]$_0$)

which are called substitution laws (instantiation is sometimes called substitution). They explain that the operation $_[_]_0$ computes by looking at the shape of the term that is being instantiated. This is like defining instantiation by iteration on the term. But how do we instantiate a term that is a local definition?

$$\frac{\text{t} : \text{Tm} (\Gamma \triangleright \text{C}) \text{ A} \qquad \text{t'} : \text{Tm} (\Gamma \triangleright \text{C} \triangleright \text{A}) \text{ B} \qquad \text{w} : \text{Tm } \Gamma \text{ C}}{\text{def t t' [w]}_0 = \text{def (t [w]}_0) \text{ (t' [?]}_0)}$$

The issue is that in t' we need to instantiate the last but one variable, not the last one. We have to generalise $_[_]_0$.

We introduce a new sort called substitutions Sub Δ Γ for any two context Δ and Γ. Sub Δ Γ depends on Δ just as Tm Δ A, however it provides not only one term of type A, but one term for each type in the context Γ. We use σ, ρ metavariables for substitutions. The first way to create such a substitution is ⟨_⟩ which turns a term Tm Γ A into Sub Γ ($\Gamma \triangleright$ A).

$$\frac{}{\text{Sub} : \text{Con} \to \text{Con} \to \text{Set}} \qquad \frac{\text{t} : \text{Tm } \Gamma \text{ A} \qquad \sigma : \text{Sub } \Delta \ \Gamma}{\text{t [} \sigma \text{] : Tm } \Delta \text{ A}} \qquad \frac{\text{u} : \text{Tm } \Gamma \text{ A}}{\langle \text{ u } \rangle : \text{Sub } \Gamma \ (\Gamma \triangleright \text{A})}$$

Then we need a way to lift a substitution so that it does not touch the last variable:

$$\frac{\sigma : \mathsf{Sub}\ \Delta\ \Gamma}{\sigma^{+} : \mathsf{Sub}\ (\Delta \rhd \mathrm{A})\ (\Gamma \rhd \mathrm{A})}$$

Now we can explain how to instantiate all terms:

true [σ] = true
false [σ] = false
ite t u v [σ] = ite (t [σ]) (u [σ]) (v [σ])
num n [σ] = num n
isZero t [σ] = isZero (t [σ])
u +o v [σ] = (u [σ]) +o (v [σ])
def t t' [σ] = def (t [σ]) (t' [σ^{+}])

In the last case we used lifting when instantiating t'. When instantiating variables, we have to check which way the substitution was built.

var vz [⟨ w ⟩] = w
var vz [σ^{+}] = var vz
var (vs x) [⟨ w ⟩] = var x
var (vs x) [σ^{+}] = ?

In the last case we have the following types.

x : Var Γ A
vs x : Var (Γ ▷ B) A
σ : Sub Δ Γ
σ^{+} : Sub (Δ ▷ B) (Γ ▷ B)
var x [σ] : Tm Δ A
var (vs x) [σ^{+}] : Tm (Δ ▷ B) A

So we need to *weaken* var x [σ] which is in context Δ to obtain a term in context Δ ▷ B. Weakening means that we add one to every De Bruijn index. For this purpose we introduce a new substitution p which we can now use to compute var (vs x) [σ^{+}] and then again we have to say what happens if we instantiate a variable with p, but that is easy.

$$\frac{}{\mathrm{p} : \mathsf{Sub}\ (\Gamma \rhd \mathrm{A})\ \Gamma} \qquad \frac{}{\mathsf{var}\ (\mathsf{vs}\ \mathrm{x})\ [\ \sigma^{+}\ ] = \mathsf{var}\ \mathrm{x}\ [\ \sigma\ ]\ [\ \mathrm{p}\ ]} \qquad \frac{}{\mathsf{var}\ \mathrm{x}\ [\ \mathrm{p}\ ] = \mathsf{var}\ (\mathsf{vs}\ \mathrm{x})}$$

In summary, a Def-model consists of the following components. We list them in groups.

- We have the following sorts.

```
Ty  : Set i
Con : Set i
Var : Con → Ty → Set j
Tm  : Con → Ty → Set j
Sub : Con → Con → Set j
```

- The rules of the *substitution calculus* are the general rules concerning contexts, substitutions and variables. They do not mention any specific types.

```
◇     : Con
_▷_   : Con → Ty → Con

p     : ∀{Γ A} → Sub (Γ ▷ A) Γ
⟨_⟩   : ∀{Γ A} → Tm Γ A → Sub Γ (Γ ▷ A)
_⁺    : ∀{Γ Δ A} → (σ : Sub Δ Γ) → Sub (Δ ▷ A) (Γ ▷ A)

vz    : ∀{Γ A} → Var (Γ ▷ A) A
```

```
vs     : ∀{Γ A B} → Var Γ A → Var (Γ ▷ B) A
var    : ∀{Γ A} → Var Γ A → Tm Γ A
_[_]   : ∀{Γ Δ A} → Tm Γ A → Sub Δ Γ → Tm Δ A
[p]    : ∀{Γ A B x} →        var {Γ}{A} x [ p {A = B} ] ≡ var (vs x)
vz[⟨⟩] : ∀{Γ A t} →          var (vz {Γ}{A}) [ ⟨ t ⟩ ] ≡ t
vz[⁺]  : ∀{Γ Δ A σ} →        var (vz {Γ}{A}) [ σ ⁺ ] ≡ var (vz {Δ}{A})
vs[⟨⟩] : ∀{Γ A B x t} →      var (vs {Γ}{A}{B} x) [ ⟨ t ⟩ ] ≡ var x
vs[⁺]  : ∀{Γ Δ A B x σ} → var (vs {Γ}{A}{B} x) [ σ ⁺ ] ≡ var x [ σ ] [ p {Δ} ]
```

- The rules for the type Bool are the following.

```
Bool   : Ty
true   : ∀{Γ} → Tm Γ Bool
false  : ∀{Γ} → Tm Γ Bool
ite    : ∀{Γ A} → Tm Γ Bool → Tm Γ A → Tm Γ A → Tm Γ A
iteβ₁  : ∀{Γ A u v} → ite {Γ}{A} true u v ≡ u
iteβ₂  : ∀{Γ A u v} → ite {Γ}{A} false u v ≡ v
true[] : ∀{Γ Δ σ} → true {Γ} [ σ ] ≡ true {Δ}
false[] : ∀{Γ Δ σ} → false {Γ} [ σ ] ≡ false {Δ}
ite[]  : ∀{Γ Δ A t u v σ} → (ite {Γ}{A} t u v) [ σ ] ≡ ite {Δ} (t [ σ ]) (u [ σ ]) (v [ σ ])
```

- The rules for the type Nat are the following.

```
Nat      : Ty
num      : ∀{Γ} → ℕ → Tm Γ Nat
isZero   : ∀{Γ} → Tm Γ Nat → Tm Γ Bool
_+o_     : ∀{Γ} → Tm Γ Nat → Tm Γ Nat → Tm Γ Nat
isZeroβ₁ : ∀{Γ} → isZero (num 0) ≡ true {Γ = Γ}
isZeroβ₂ : ∀{Γ n} → isZero (num (1 + n)) ≡ false {Γ = Γ}
+β       : ∀{Γ m n} → num m +o num n ≡ num {Γ = Γ} (m + n)
num[]    : ∀{Γ Δ σ n} → num {Γ} n [ σ ] ≡ num {Δ} n
isZero[] : ∀{Γ Δ t σ} → (isZero {Γ} t) [ σ ] ≡ isZero {Δ} (t [ σ ])
+[]      : ∀{Γ Δ u v}{σ : Sub Δ Γ} → (u +o v) [ σ ] ≡ (u [ σ ]) +o (v [ σ ])
```

In fact, we haven't added the operator def to our language. But as it has a simple computation rule, we can just define it as *syntactic sugar* (an abbreviation).

```
def : ∀{Γ A B} → Tm Γ A → Tm (Γ ▷ A) B → Tm Γ B
def u t = t [ ⟨ u ⟩ ]
```

The term let x:= false in if x then true else isZero (if x then (num 0) else (num 1)) is equal to false in any model.

```
def false (ite (var vz) true (isZero (ite (var vz) (num 0) (num 1))))
  ≡⟨ refl ⟩
ite (var vz) true (isZero (ite (var vz) (num 0) (num 1))) [ ⟨ false ⟩ ]
  ≡⟨ ite[] ⟩
ite ((var vz) [ ⟨ false ⟩ ]) (true [ ⟨ false ⟩ ]) (isZero (ite (var vz) (num 0) (num 1)) [ ⟨ false ⟩ ])
  ≡⟨ cong (λ z → ite z (true [ ⟨ false ⟩ ]) (isZero (ite (var vz) (num 0) (num 1)) [ ⟨ false ⟩ ])) vz[⟨⟩] ⟩
ite false (true [ ⟨ false ⟩ ]) (isZero (ite (var vz) (num 0) (num 1)) [ ⟨ false ⟩ ])
  ≡⟨ iteβ₂ ⟩
isZero (ite (var vz) (num 0) (num 1)) [ ⟨ false ⟩ ]
  ≡⟨ isZero[] ⟩
isZero (ite (var vz) (num 0) (num 1) [ ⟨ false ⟩ ])
  ≡⟨ cong isZero ite[] ⟩
isZero (ite (var vz [ ⟨ false ⟩ ]) (num 0 [ ⟨ false ⟩ ]) (num 1 [ ⟨ false ⟩ ]))
```

```
  ≡⟨ cong (λ z → isZero (ite z (num 0 [ ⟨ false ⟩ ]) (num 1 [ ⟨ false ⟩ ]))) vz[⟨⟩] ⟩
isZero (ite false (num 0 [ ⟨ false ⟩ ]) (num 1 [ ⟨ false ⟩ ]))
  ≡⟨ cong isZero iteβ₂ ⟩
isZero (num 1 [ ⟨ false ⟩ ])
  ≡⟨ cong isZero num[] ⟩
isZero (num 1)
  ≡⟨ isZeroβ₂ ⟩
false
  ∎
```

A *program* of type A is an element of I.Tm ⋄ A where I is the initial model as usual. When we work "inside" the language, we can only write programs.

Exercise 2.11 (compulsory)**.** *Prove* (def t u) [σ] ≡ def (t [σ]) (u [σ^+]) *for any* t, u, γ *in the syntax.*

2.1.3.1 Standard model and inequalities in the empty context

In the standard model, types are sets as before, contexts are iterated products of their constituent types, so they are also sets. Terms are not simply elements of their types because they depend on a context. This dependency is modelled using functions: a term of type A in context Γ is a function Γ → A. Substitutions are also functions.

```
St : Model
St = record
  { Ty       = Set
  ; Nat      = ℕ
  ; Bool     = 𝟚
  ; Con      = Set
  ; ⋄        = Lift ⊤
  ; _▷_      = _×_
  ; Var      = λ Γ A → Γ → A
  ; vz       = π₂
  ; vs       = λ σ → σ ∘ π₁
  ; Tm       = λ Γ A → Γ → A
  ; Sub      = λ Δ Γ → Δ → Γ
  ; p        = π₁
  ; ⟨_⟩      = λ t γ → (γ , t γ)
  ; _⁺       = λ σ δa → σ (π₁ δa) , π₂ δa
  ; var      = λ x → x
  ; _[_]     = λ t σ → t ∘ σ
  ; [p]      = refl
  ; vz[⟨⟩]   = refl
  ; vz[⁺]    = refl
  ; vs[⟨⟩]   = refl
  ; vs[⁺]    = refl
  ; true     = λ _ → tt
  ; false    = λ _ → ff
  ; ite      = λ t u v γ → if t γ then u γ else v γ
  ; iteβ₁    = refl
  ; iteβ₂    = refl
  ; true[]   = refl
  ; false[]  = refl
  ; ite[]    = refl
  ; num      = λ n _ → n
  ; isZero   = λ t γ → iteℕ tt (λ _ → ff) (t γ)
  ; _+o_     = λ u v γ → u γ + v γ
```

```
  ; isZeroβ₁ = refl
  ; isZeroβ₂ = refl
  ; +β       = refl
  ; num[]    = refl
  ; isZero[] = refl
  ; +[]      = refl
  }
module St = Model St
```

A term in the empty context reproduces the interpretation of terms in the standard model of Razor, e.g. Tm ⋄ Nat = ⊤ → ℕ which is equivalent to having a natural number. All equations hold by definition in the standard model: we can simply prove them by refl.

Equational consistency can be proved using the standard model just as for Razor. For example, we can show that the following two closed terms are not equal in the syntax. If they were equal, then their interpretations into the standard model would be also equal.

```
import Def.Syntax as I

not-equal : ¬ (
  I.ite (I.isZero (I.num 2)) (I.num 3) (I.num 12) ≡
  I.ite (I.isZero (I.num 0 I.+o I.num 0)) (I.num 11) (I.num {I.⋄} 3))
not-equal e = 12≢11 (cong (λ t → St.⟦ t ⟧t (mk triv)) e)
  where
    12≢11 : ¬ (12 ≡ 11)
    12≢11 ()
```

For open terms, the standard model is not good enough for the same purpose. From normalisation, we will obtain that isZero (q {⋄} +o num 2) is not equal in the syntax to false, but in the standard model they are both interpreted by the constant false function.

```
ex-open : St.⟦ I.isZero (I.var (I.vz {I.⋄}) I.+o I.num 2) ⟧t ≡ St.⟦ I.false {I.⋄ I.▷ I.Nat} ⟧t
ex-open = funext λ { (_ , n) → cong (iteℕ tt (λ _ → ff)) (+comm {n}{2}) }
```

2.1.3.2 Type inference

We define type inference from ABT-level trees to well-typed trees just as for Razor (Section 1.4.1). We define a DefABT model built out of Def syntax. We use an Agda trick called "views" [17] to connect contexts and their lengths.

```
data _hasLength_ : I.Con → ℕ → Set where
  nil  : I.⋄ hasLength 0
  snoc : ∀{Γ n} → Γ hasLength n → (A : I.Ty) → (Γ I.▷ A) hasLength suc n

import DefABT

Inf : DefABT.Model
Inf = record
        { Var = λ n → {Γ : I.Con} → Γ hasLength n → Σ I.Ty λ A → I.Var Γ A
        ; vz = λ { (snoc _ A) → A , I.vz }
        ; vs = λ { infx (snoc Γn A) → π₁ (infx Γn) , I.vs (π₂ (infx Γn)) }
        ; Tm = λ n → {Γ : I.Con} → Γ hasLength n → Maybe (Σ I.Ty λ A → I.Tm Γ A)
        ; var = λ infx Γn → just (π₁ (infx Γn) , I.var (π₂ (infx Γn)))
        ; def = def
        ; true = λ _ → just (I.Bool , I.true)
        ; false = λ _ → just (I.Bool , I.false)
        ; ite = ite
        ; num = λ n _ → just (I.Nat , I.num n)
```

```
        ; isZero = isZero
        ; _+o_ = _+o_
        }
    where
      def : ∀{n} → (∀{Γ} → Γ hasLength n      → Maybe (Σ I.Ty (I.Tm Γ))) →
                   (∀{Γ} → Γ hasLength suc n → Maybe (Σ I.Ty (I.Tm Γ))) →
                   ∀{Γ}  → Γ hasLength n      → Maybe (Σ I.Ty (I.Tm Γ))
      def infu inft Γn with infu Γn
      ... | nothing = nothing
      ... | just (A , u) with inft (snoc Γn A)
      ... | nothing = nothing
      ... | just (B , t) = just (B , t I.[ I.⟨ u ⟩ ])

      ite : ∀{n} →  (∀{Γ} → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))) →
                    (∀{Γ} → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))) →
                    (∀{Γ} → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))) →
                    ∀{Γ}  → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))
      ite inft infu infv Γn with inft Γn | infu Γn | infv Γn
      ... | just (I.Bool , t) | just (I.Bool , u) | just (I.Bool , v) = just (I.Bool , I.ite t u v)
      ... | just (I.Bool , t) | just (I.Nat  , u) | just (I.Nat  , v) = just (I.Nat  , I.ite t u v)
      ... | _                 | _                 | _                 = nothing

      isZero : ∀{n} → (∀{Γ} → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))) →
                      ∀{Γ}  → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))
      isZero inft Γn with inft Γn
      ... | just (I.Nat , t)        = just (I.Bool , I.isZero t)
      ... | _                       = nothing

      _+o_ : ∀{n} → (∀{Γ} → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))) →
                    (∀{Γ} → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))) →
                    ∀{Γ}  → Γ hasLength n → Maybe (Σ I.Ty (I.Tm Γ))
      _+o_ infu infv Γn with infu Γn | infv Γn
      ... | just (I.Nat , u) |       just (I.Nat , v) = just (I.Nat , u I.+o v)
      ... | _                | _                     = nothing
  module Inf = DefABT.Model Inf
```

The final infer function works on terms in the empty context.

```
infer : DefABT.I.Tm 0 → Maybe (Σ I.Ty λ A → I.Tm I.⋄ A)
infer pt = Inf.⟦ pt ⟧t nil
```

2.1.3.3 Normalisation

Before defining normal forms we group the operators specific to Bool and Nat as follows:

- constructors (introduction rules): operators which introduce elements of the type (true and false for Bool; num for Nat),
- destructors (elimination rules, eliminators): operators that can be used to eliminate an element of the type (ite for Bool; isZero and _+o_ for Nat),
- computation (β) rules: equations that explain what happens if a destructor is applied to a constructor (iteβ_1 and iteβ_2 for Bool; isZeroβ_1, isZeroβ_2 and +β for Nat),
- uniqueness (η) rules: equations that explain what happens if a constructor is applied to a destructor (we don't have any),
- substitution rules: explain how instantiation _[_] interacts with the operators (true[], false[], ite[], num[], isZero[], +[]).

Perhaps surprisingly, isZero is a destructor (of Nat) instead of a constructor (of Bool). This is because it computes depending on its input which is a Nat.

Each destructor has some *main arguments* (scrutinees). These are the arguments which are in constructor forms in the computation rule. The main argument of ite is the first one, the single argument of isZero is a main argument, and both arguments of _+o_ are main. Once we know the main arguemnts, we can compute further using the β rule.

Neutral terms and normal forms are defined as mutual inductive types.

```
data Ne (Γ : Con) : Ty → Set
data Nf (Γ : Con) : Ty → Set

data Ne Γ where
  varNe     : ∀{A} → Var Γ A → Ne Γ A
  iteNe     : ∀{A} → Ne Γ Bool → Nf Γ A → Nf Γ A → Ne Γ A
  isZeroNe  : Ne Γ Nat → Ne Γ Bool
  _+NeNe_   : Ne Γ Nat → Ne Γ Nat → Ne Γ Nat
  _+NeNf_   : Ne Γ Nat → ℕ → Ne Γ Nat
  _+NfNe_   : ℕ → Ne Γ Nat → Ne Γ Nat

data Nf Γ where
  neu       : ∀{A} → Ne Γ A → Nf Γ A
  trueNf    : Nf Γ Bool
  falseNf   : Nf Γ Bool
  numNf     : ℕ → Nf Γ Nat
```

Neutral terms are either variables or a destructor with a neutral main argument. For example, ite (var vz) true true is a neutral term because we don't know which computation rule (iteβ_1 or iteβ_2) to apply. Normal forms are either neutral terms or constructors. All non-main arguments of destructors and arguments of constructors have to be normal (we don't have any of the latter). The destructor _+_ has two main arguments, at least one of them has to be neutral. That is why we have three cases: both are neutral, the left is normal (that is, a natural number because that is the only parameter of num), the right is normal. For example, num 3 +o var vz and num 3 +o ite (var vz) (num 1) (num 2) are neutral (and hence normal), but num 3 +o num 2 is not normal because by +β it is equal to num 5.

Normal forms can be quoted back into the syntax.

```
⌜_⌝Ne : ∀{Γ A} → Ne Γ A → Tm Γ A
⌜_⌝Nf : ∀{Γ A} → Nf Γ A → Tm Γ A
⌜ varNe x ⌝Ne = var x
⌜ iteNe t a a' ⌝Ne = ite ⌜ t ⌝Ne ⌜ a ⌝Nf ⌜ a' ⌝Nf
⌜ isZeroNe t ⌝Ne = isZero ⌜ t ⌝Ne
⌜ t +NeNe t' ⌝Ne = ⌜ t ⌝Ne +o ⌜ t' ⌝Ne
⌜ t +NeNf n' ⌝Ne = ⌜ t ⌝Ne +o num n'
⌜ n +NfNe t' ⌝Ne = num n +o ⌜ t' ⌝Ne
⌜ neu t ⌝Nf = ⌜ t ⌝Ne
⌜ trueNf ⌝Nf = true
⌜ falseNf ⌝Nf = false
⌜ numNf n ⌝Nf = num n
```

Exercise 2.12. *What is the normal form of the following terms?*

```
t1 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → □ v □Nf ≡ v0 +o (num 1 +o num 2)
t2 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → □ v □Nf ≡ v0 +o v0
t3 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → □ v □Nf ≡ (num 1 +o num 2) +o v0
t4 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → □ v □Nf ≡ (num 1 +o v0) +o v0
t5 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → □ v □Nf ≡ isZero (num 2)
t6 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → □ v □Nf ≡ isZero (num 1 +o num 2)
```

```
t7 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → □ v □Nf ≡ isZero (num 1 +o v0)
t8 : Σsp (Nf ◇ Bool) λ v → □ v □Nf ≡ def (num 0) (def (isZero v0) (ite v0 false true))
```

In contrast with Razor (Section 1.5.2) where we were able to normalise using the standard model, for Def we have to define a separate model for normalisation. Terms in this model will be normal forms and substitutions will be either weakenings (p with some $_^{+}$s applied to it) or single substitutions (for a t normal, ⟨ t ⟩ with some $_^{+}$s applied to it). We first define these together with their respective quote functions.

```
data Wk  : Con → Con → Set where
  p      : ∀{Γ A} → Wk (Γ ▷ A) Γ
  _⁺Wk   : ∀{Γ Δ A} → Wk Δ Γ → Wk (Δ ▷ A) (Γ ▷ A)

data Sub1 : Con → Con → Set where
  ⟨_⟩Sub1 : ∀{Γ A} → Nf Γ A → Sub1 Γ (Γ ▷ A)
  _⁺Sub1  : ∀{Γ Δ A} → Sub1 Δ Γ → Sub1 (Δ ▷ A) (Γ ▷ A)

⌜_⌝Wk : ∀{Δ Γ} → Wk Δ Γ → Sub Δ Γ
⌜ p      ⌝Wk = p
⌜ σ ⁺Wk ⌝Wk = ⌜ σ ⌝Wk ⁺

⌜_⌝Sub1 : ∀{Δ Γ} → Sub1 Δ Γ → Sub Δ Γ
⌜ ⟨ t ⟩Sub1 ⌝Sub1 = ⟨ ⌜ t ⌝Nf ⟩
⌜ σ ⁺Sub1  ⌝Sub1 = ⌜ σ ⌝Sub1 ⁺
```

Now we define the interpretation of the destructors on normal forms. They return the neutral term when given a neutral argument, and follow the computation rule when given a constructor.

```
iteNf : ∀{Γ A} → Nf Γ Bool → Nf Γ A → Nf Γ A → Nf Γ A
iteNf (neu t) a a' = neu (iteNe t a a')
iteNf trueNf a a' = a
iteNf falseNf a a' = a'

_+NfNf_ : ∀{Γ} → Nf Γ Nat → Nf Γ Nat → Nf Γ Nat
neu tn +NfNf neu tn' = neu (tn +NeNe tn')
neu tn +NfNf numNf n = neu (tn +NeNf n)
numNf n +NfNf neu tn = neu (n +NfNe tn)
numNf n +NfNf numNf n' = numNf (n + n')

isZeroNf : ∀{Γ} → Nf Γ Nat → Nf Γ Bool
isZeroNf (neu n) = neu (isZeroNe n)
isZeroNf (numNf zero) = trueNf
isZeroNf (numNf (suc n)) = falseNf
```

Instantiation of normal forms by weakenings and single normal subsitutions.

```
_[_]VWk : ∀{Γ A} → Var Γ A → ∀{Δ} → Wk Δ Γ → Var Δ A
x    [ p      ]VWk = vs x
vz   [ σ ⁺Wk ]VWk = vz
vs x [ σ ⁺Wk ]VWk = vs (x [ σ ]VWk)

_[_]NeWk : ∀{Γ A} → Ne Γ A → ∀{Δ} → Wk Δ Γ → Ne Δ A
_[_]NfWk : ∀{Γ A} → Nf Γ A → ∀{Δ} → Wk Δ Γ → Nf Δ A

varNe x       [ σ ]NeWk = varNe (x [ σ ]VWk)
iteNe t u v   [ σ ]NeWk = iteNe (t [ σ ]NeWk) (u [ σ ]NfWk) (v [ σ ]NfWk)
isZeroNe t    [ σ ]NeWk = isZeroNe (t [ σ ]NeWk)
(t +NeNe t') [ σ ]NeWk = t [ σ ]NeWk +NeNe t' [ σ ]NeWk
```

```
(t +NeNf n') [ σ ]NeWk = t [ σ ]NeWk +NeNf n'
(n +NfNe t') [ σ ]NeWk = n +NfNe t' [ σ ]NeWk

neu x     [ σ ]NfWk = neu (x [ σ ]NeWk)
trueNf    [ σ ]NfWk = trueNf
falseNf   [ σ ]NfWk = falseNf
numNf n [ σ ]NfWk = numNf n

_[_]VSub1 : ∀{Γ A} → Var Γ A → ∀{Δ} → Sub1 Δ Γ → Nf Δ A
vz    [ ⟨ t ⟩Sub1 ]VSub1 = t
vs x [ ⟨ t ⟩Sub1 ]VSub1 = neu (varNe x)
vz    [ σ ⁺Sub1  ]VSub1 = neu (varNe vz)
vs x [ σ ⁺Sub1  ]VSub1 = x [ σ ]VSub1 [ p ]NfWk

_[_]NeSub1 : ∀{Γ A} → Ne Γ A → ∀{Δ} → Sub1 Δ Γ → Nf Δ A
_[_]NfSub1 : ∀{Γ A} → Nf Γ A → ∀{Δ} → Sub1 Δ Γ → Nf Δ A

varNe x       [ σ ]NeSub1 = x [ σ ]VSub1
iteNe t u v   [ σ ]NeSub1 = iteNf (t [ σ ]NeSub1) (u [ σ ]NfSub1) (v [ σ ]NfSub1)
isZeroNe t    [ σ ]NeSub1 = isZeroNf (t [ σ ]NeSub1)
(t +NeNe t') [ σ ]NeSub1 = t [ σ ]NeSub1 +NfNf t' [ σ ]NeSub1
(t +NeNf n') [ σ ]NeSub1 = t [ σ ]NeSub1 +NfNf numNf n'
(n +NfNe t') [ σ ]NeSub1 = numNf n +NfNf t' [ σ ]NeSub1

neu x     [ σ ]NfSub1 = x [ σ ]NeSub1
trueNf    [ σ ]NfSub1 = trueNf
falseNf   [ σ ]NfSub1 = falseNf
numNf n [ σ ]NfSub1 = numNf n
```

Now we define general normal substitutions: it is either a weakening or a single normal substitution.

```
SubNf : Con → Con → Set
SubNf Δ Γ = Wk Δ Γ ⊎ Sub1 Δ Γ

⌜_⌝SubNf : ∀{Δ Γ} → SubNf Δ Γ → Sub Δ Γ
⌜ ι₁ σ ⌝SubNf = ⌜ σ ⌝Wk
⌜ ι₂ σ ⌝SubNf = ⌜ σ ⌝Sub1

pNf : ∀{Γ A} → SubNf (Γ ▷ A) Γ
pNf = ι₁ p

⟨_⟩Nf : ∀{Γ A} → Nf Γ A → SubNf Γ (Γ ▷ A)
⟨ t ⟩Nf = ι₂ ⟨ t ⟩Sub1

_⁺Nf : ∀{Δ Γ A} → SubNf Δ Γ → SubNf (Δ ▷ A) (Γ ▷ A)
ι₁ σ ⁺Nf = ι₁ (σ ⁺Wk)
ι₂ σ ⁺Nf = ι₂ (σ ⁺Sub1)
```

Instantiation with general normal substitutions.

```
_[_]V : ∀{Γ A} → Var Γ A → ∀{Δ} → SubNf Δ Γ → Nf Δ A
x [ ι₁ σ ]V = neu (varNe (x [ σ ]VWk))
x [ ι₂ σ ]V = x [ σ ]VSub1

_[_]Ne : ∀{Γ A} → Ne Γ A → ∀{Δ} → SubNf Δ Γ → Nf Δ A
t [ ι₁ σ ]Ne = neu (t [ σ ]NeWk)
t [ ι₂ σ ]Ne = t [ σ ]NeSub1

_[_]Nf : ∀{Γ A} → Nf Γ A → ∀{Δ} → SubNf Δ Γ → Nf Δ A
```

```
t [ ι₁ σ ]Nf = t [ σ ]NfWk
t [ ι₂ σ ]Nf = t [ σ ]NfSub1
```

We prove the substitution laws by case analysis on general normal substitutions and the main arguments.

```
vz[⁺]Nf    : ∀{Γ Δ A σ} → neu (varNe (vz {Γ}{A})) [ σ ⁺Nf ]Nf ≡ neu (varNe (vz {Δ}{A}))
vs[⁺]Nf    : ∀{Γ Δ A B x σ} →
             neu (varNe (vs {Γ}{A}{B} x)) [ σ ⁺Nf ]Nf ≡ neu (varNe x) [ σ ]Nf [ pNf {Δ} ]Nf
true[]Nf   : ∀{Γ Δ}{σ : SubNf Δ Γ} → trueNf [ σ ]Nf ≡ trueNf
false[]Nf  : ∀{Γ Δ}{σ : SubNf Δ Γ} → falseNf [ σ ]Nf ≡ falseNf
ite[]Nf    : ∀{Γ Δ A}{σ : SubNf Δ Γ}{t u v} →
             iteNf {A = A} t u v [ σ ]Nf ≡ iteNf (t [ σ ]Nf) (u [ σ ]Nf) (v [ σ ]Nf)
num[]Nf    : ∀{Γ Δ}{σ : SubNf Δ Γ}{n} → numNf n [ σ ]Nf ≡ numNf n
isZero[]Nf : ∀{Γ Δ}{σ : SubNf Δ Γ}{t} → isZeroNf t [ σ ]Nf ≡ isZeroNf (t [ σ ]Nf)
+[]Nf      : ∀{Γ Δ}{σ : SubNf Δ Γ}{u v} →
             (u +NfNf v) [ σ ]Nf ≡ (u [ σ ]Nf) +NfNf (v [ σ ]Nf)
```

Now we build the model for normalisation.

```
open import Def.ParaModel
N : ParaModel
N = record
  { Ty• = Lift ⊤
  ; Nat• = mk triv
  ; Bool• = mk triv
  ; Con• = Lift ⊤
  ; ◇• = _
  ; _▷•_ = _
  ; Var• = λ _ _ → Lift ⊤
  ; vz• = _
  ; vs• = _
  ; Tm• = λ Γ A → Nf (π₁ Γ) (π₁ A)
  ; Sub• = λ Δ Γ → SubNf (π₁ Δ) (π₁ Γ)
  ; p• = pNf
  ; ⟨_⟩• = λ t → ⟨ π₂ t ⟩Nf
  ; _⁺• = λ σ → π₂ σ ⁺Nf
  ; var• = λ x → neu (varNe (π₁ x))
  ; _[_]• = λ t σ → π₂ t [ π₂ σ ]Nf
  ; [p]• = refl
  ; vz[⟨⟩]• = refl
  ; vz[⁺]• = λ where {σ = σ} → vz[⁺]Nf {σ = π₂ σ}
  ; vs[⟨⟩]• = refl
  ; vs[⁺]• = λ where {σ = σ} → vs[⁺]Nf {σ = π₂ σ}
  ; true• = trueNf
  ; false• = falseNf
  ; ite• = λ t a a' → iteNf (π₂ t) (π₂ a) (π₂ a')
  ; iteβ₁• = refl
  ; iteβ₂• = refl
  ; true[]• = λ where {γ = σ} → true[]Nf {σ = π₂ σ}
  ; false[]• = λ where {γ = σ} → false[]Nf {σ = π₂ σ}
  ; ite[]• = λ where {γ = σ} → ite[]Nf {σ = π₂ σ}
  ; num• = numNf
  ; isZero• = λ t → isZeroNf (π₂ t)
  ; _+o•_ = λ t t' → π₂ t +NfNf π₂ t'
  ; isZeroβ₁• = refl
  ; isZeroβ₂• = refl
```

```
    ; +β• = refl
    ; num[]• = λ where {γ = σ} → num[]Nf {σ = π₂ σ}
    ; isZero[]• = λ where {γ = σ} → isZero[]Nf {σ = π₂ σ}
    ; +[]• = λ where {γ = σ} → +[]Nf {σ = π₂ σ}
    }
module N = ParaModel N
```

Normalisation is interpretation into this model.

```
norm : ∀{Γ A} → Tm Γ A → Nf Γ A
norm = N.⟦_⟧t
```

Some examples of normalisation:

```
ex1 : norm (ite (isZero (num 1 +o num 2)) (v 0) false) ≡ falseNf {◇ ▷ Bool}
ex1 = refl

ex2 : norm (def (num 1) ((v 0 +o v 0) +o ite false (v 0) (v 0))) ≡ numNf {◇} 3
ex2 = refl
```

We used the standard model to show that two closed terms are not equal in the syntax. We can also use normalisation for the same purpose:

```
not-equal' : ¬ (
  I.ite (I.isZero (I.num 2)) (I.num 3) (I.num 12) ≡
  I.ite (I.isZero (I.num 0 I.+o I.num 0)) (I.num 11) (I.num {I.◇} 3))
not-equal' e = 12≢11 (cong norm e)
  where
    12≢11 : ¬ (numNf 12 ≡ numNf 11)
    12≢11 ()
```

And unlike the standard interpretation, normalisation can be used to prove inequality of open terms:

```
ex-open' : ¬ (isZero (v 0 {◇ ▷ Nat} +o num 2) ≡ false)
ex-open' e with cong norm e
... | ()
```

For completeness, we first prove that the operations in the normalisation model commute with quotation. These are proven by induction on the variable/neutral term/normal form and the general normal substitution (see the source code for details).

```
⌜+⌝          : ∀{Γ Δ A}{σ : SubNf Δ Γ} → ⌜ _⁺Nf {A = A} σ ⌝SubNf ≡ ⌜ σ ⌝SubNf ⁺
⌜isZeroNf⌝   : ∀{Γ}{t : Nf Γ Nat} → ⌜ isZeroNf t ⌝Nf ≡ isZero ⌜ t ⌝Nf
⌜iteNf⌝      : ∀{Γ A}{t : Nf Γ Bool}{u v : Nf Γ A} →
               ⌜ iteNf t u v ⌝Nf ≡ ite ⌜ t ⌝Nf ⌜ u ⌝Nf ⌜ v ⌝Nf
⌜+NfNf⌝      : ∀{Γ}{t t' : Nf Γ Nat} → ⌜ t +NfNf t' ⌝Nf ≡ ⌜ t ⌝Nf +o ⌜ t' ⌝Nf
⌜[]VWk⌝      : ∀{Γ A}{x : Var Γ A}{Δ}{σ : Wk Δ Γ} →
               var (x [ σ ]VWk) ≡ var x [ ⌜ σ ⌝Wk ]
⌜[]NeWk⌝     : ∀{Γ A}{t : Ne Γ A}{Δ}{σ : Wk Δ Γ} →
               ⌜ t [ σ ]NeWk ⌝Ne ≡ ⌜ t ⌝Ne [ ⌜ σ ⌝Wk ]
⌜[]NfWk⌝     : ∀{Γ A}{t : Nf Γ A}{Δ}{σ : Wk Δ Γ} →
               ⌜ t [ σ ]NfWk ⌝Nf ≡ ⌜ t ⌝Nf [ ⌜ σ ⌝Wk ]
⌜[]VSub1⌝    : ∀{Γ A}{x : Var Γ A}{Δ}{σ : Sub1 Δ Γ} →
               ⌜ x [ σ ]VSub1 ⌝Nf ≡ var x [ ⌜ σ ⌝Sub1 ]
⌜[]NeSub1⌝   : ∀{Γ A}{t : Ne Γ A}{Δ}{σ : Sub1 Δ Γ} →
               ⌜ t [ σ ]NeSub1 ⌝Nf ≡ ⌜ t ⌝Ne [ ⌜ σ ⌝Sub1 ]
⌜[]NfSub1⌝   : ∀{Γ A}{t : Nf Γ A}{Δ}{σ : Sub1 Δ Γ} →
```

```
                ⌜ t [ σ ]NfSub1 ⌝Nf ≡ ⌜ t ⌝Nf [ ⌜ σ ⌝Sub1 ]
⌜[]V⌝       : ∀{Γ A}{x : Var Γ A}{Δ}{σ : SubNf Δ Γ} →
                ⌜ x [ σ ]V ⌝Nf ≡ var x [ ⌜ σ ⌝SubNf ]
⌜[]Ne⌝      : ∀{Γ A}{t : Ne Γ A}{Δ}{σ : SubNf Δ Γ} →
                ⌜ t [ σ ]Ne ⌝Nf ≡ ⌜ t ⌝Ne [ ⌜ σ ⌝SubNf ]
⌜[]Nf⌝      : ∀{Γ A}{t : Nf Γ A}{Δ}{σ : SubNf Δ Γ} →
                ⌜ t [ σ ]Nf ⌝Nf ≡ ⌜ t ⌝Nf [ ⌜ σ ⌝SubNf ]
```

Now we build the dependent model for completeness: completeness is proven by induction on terms and substitutions.

```
open import Def.DepModel
Comp : DepModel
Comp = record
  { Ty• = λ _ → Lift ⊤
  ; Nat• = _
  ; Bool• = _
  ; Con• = λ _ → Lift ⊤
  ; ◇• = _
  ; _▷•_ = _
  ; Var• = λ _ _ _ → Lift ⊤
  ; vz• = _
  ; vs• = _
  ; Tm• = λ {Γ}{A} _ _ t → Lift (⌜ N.⟦ t ⟧t ⌝Nf ≡ t)
  ; Sub• = λ _ _ σ → Lift (⌜ N.⟦ σ ⟧s ⌝SubNf ≡ σ)
  ; p• = mk refl
  ; ⟨_⟩• = λ e → mk (cong ⟨_⟩ (un e))
  ; _⁺• = λ where {A = A}{σ = σ} e → mk (⌜⁺⌝ {A = A}{N.⟦ σ ⟧s} ■ cong _⁺ (un e))
  ; var• = λ _ → mk refl
  ; _[_]• =
    λ where {t = t}{σ = σ} te σe →
              mk (⌜[]Nf⌝ {t = N.⟦ t ⟧t}{σ = N.⟦ σ ⟧s} ■
                  cong (λ z → π₁ z [ π₂ z ]) (un te ,= un σe))
  ; [p]• = refl
  ; vz[⟨⟩]• = refl
  ; vz[⁺]• = refl
  ; vs[⟨⟩]• = refl
  ; vs[⁺]• = refl
  ; true• = mk refl
  ; false• = mk refl
  ; ite• =
    λ where {t = t}{u = u}{v = v} te ue ve →
              mk (⌜iteNf⌝ {t = N.⟦ t ⟧t}{u = N.⟦ u ⟧t}{v = N.⟦ v ⟧t} ■
                  cong (λ z → ite (π₁ z) (π₁ (π₂ z)) (π₂ (π₂ z))) (un te ,= un ue ,= un ve))
  ; iteβ₁• = refl
  ; iteβ₂• = refl
  ; true[]• = refl
  ; false[]• = refl
  ; ite[]• = refl
  ; num• = λ _ → mk refl
  ; isZero• = λ where {t = t} te → mk (⌜isZeroNf⌝ {t = N.⟦ t ⟧t} ■ cong isZero (un te))
  ; _+o•_ =
    λ where {u = u}{v} ue ve →
              mk (⌜+NfNf⌝ {t = N.⟦ u ⟧t}{N.⟦ v ⟧t} ■
                  cong (λ z → π₁ z +o π₂ z) (un ue ,= un ve))
  ; isZeroβ₁• = refl
  ; isZeroβ₂• = refl
```

```
  ; +β• = refl
  ; num[]• = refl
  ; isZero[]• = refl
  ; +[]• = refl
  }
module Comp = DepModel Comp

compl : ∀{Γ A}(t : Tm Γ A) → ⌜ norm t ⌝Nf ≡ t
compl t = un Comp.⟦ t ⟧t
```

We can use completeness to show that two terms are equal without doing all the complex equational reasoning.

```
e4 : ite true (num 3 +o num 2) (num 0) ≡ num 1 +o (num 2 +o (num 1 +o num {◇} 1))
e4 =
  ite true (num 3 +o num 2) (num 0)
                                        ≡⟨ compl (ite true (num 3 +o num 2) (num 0)) ⁻¹ ⟩
  num 5
                                        ≡⟨ compl (num 1 +o (num 2 +o (num 1 +o num 1))) ⟩
  num 1 +o (num 2 +o (num 1 +o num 1))
                                        ∎
```

Stability of normalisation (there is no junk in normal forms) is proved by induction on variables, neutral terms and normal forms.

```
stabNe : ∀{Γ A}(t : Ne Γ A) → norm ⌜ t ⌝Ne ≡ neu t
stabNf : ∀{Γ A}(t : Nf Γ A) → norm ⌜ t ⌝Nf ≡ t
```

Decidability of equality of normal forms. We show how we do it for natural numbers, the rest of the proofs are analogous (see the source code).

```
_≟ℕ_ : (m n : ℕ) → Dec (Lift (m ≡ n))
zero ≟ℕ zero     = ι₁ (mk refl)
zero ≟ℕ suc n    = ι₂ (mk λ { (mk ()) })
suc m ≟ℕ zero    = ι₂ (mk λ { (mk ()) })
suc m ≟ℕ suc n with m ≟ℕ n
... | ι₁ (mk e)  = ι₁ (mk (cong suc e))
... | ι₂ f       = ι₂ (mk λ { (mk refl) → un f (mk refl)})

_≟Ty_ : (A B : Ty) → Dec (Lift (A ≡ B))

_≟Var_ : ∀{Γ A₀ A₁}(x₀ : Var Γ A₀)(x₁ : Var Γ A₁) →
  Dec (Σ (Lift (A₀ ≡ A₁)) λ e → Lift (transp (Var Γ) (un e) x₀ ≡ x₁))

_≟Ne_ : ∀{Γ A₀ A₁}(a₀ : Ne Γ A₀)(a₁ : Ne Γ A₁) →
  Dec (Σ (Lift (A₀ ≡ A₁)) λ e → Lift (transp (Ne Γ) (un e) a₀ ≡ a₁))
_≟Nf_ : ∀{Γ A}(a₀ a₁ : Nf Γ A) → Dec (Lift (a₀ ≡ a₁))
```

We obtain decidability of equality for all terms by checking whether the normal forms are equal. If they are, then by completenes, we have an equality between the two terms. If they are not equal, then if the terms were equal then their normal forms would be also equal, which is a contradiction.

```
_≟Tm_ : ∀{Γ A}(t₀ t₁ : Tm Γ A) → Dec (Lift (t₀ ≡ t₁))
t₀ ≟Tm t₁ with norm t₀ ≟Nf norm t₁
```

```
... | ι₁ (mk e) = ι₁ (mk (compl t₀ ⁻¹ ■ cong ⌜_⌝Nf e ■ compl t₁))
... | ι₂ f = ι₂ (mk λ t₀₁ → un f (mk (cong norm (un t₀₁))))
```

Now it is very easy to prove equalities in the syntax. If we know that a witness of decidability is ι_1 by definition, then we can extract the witness from decidability of equality.

```
ex3 : isZero (v 0 +o num 2) [ ⟨ num {◇} 0 ⟩ ] ≡ false
ex3 = extract (isZero (v 0 +o num 2) [ ⟨ num {◇} 0 ⟩ ] ≟Tm false)
```

If we know that it is ι_2 by definition, we can use extract' to obtain falsity:

```
ex-open'' : ¬ (isZero (var (vz {◇}) +o num 2) ≡ false)
ex-open'' = extract' (isZero (var vz +o num 2) ≟Tm false)
```

From normalisation it follows that constructors are disjoint and injective, but not destructors.

```
true≠false : ∀{Γ} → ¬ (I.true {Γ} ≡ I.false)
true≠false e = trueNf≠falseNf (cong norm e)
  where
    trueNf≠falseNf : ∀{Γ} → ¬ (trueNf {Γ} ≡ falseNf)
    trueNf≠falseNf ()

num-inj : ∀{Γ m n} → I.num {Γ} m ≡ I.num n → m ≡ n
num-inj e = numNf-inj (cong norm e)
  where
    numNf-inj : ∀{Γ m n} → numNf {Γ} m ≡ numNf n → m ≡ n
    numNf-inj refl = refl

isZero=true : ∀{Γ} → I.isZero {Γ} (I.num 0) ≡ I.true
isZero=true = I.isZeroβ₁

isZero-not-inj : ¬ (∀{Γ t t'} → isZero {Γ} t ≡ isZero t' → t ≡ t')
isZero-not-inj H = 1≠2 (num-inj (H {I.◇}{I.num 1}{I.num 2} (I.isZeroβ₂ ■ I.isZeroβ₂ ⁻¹)))
  where
    1≠2 : ¬ (1 ≡ 2)
    1≠2 ()
```

2.1.3.4 A sort of first order functions

The language Def has a separate sort of first-order functions: in context Γ functions from A to B aredenoted Fun A B. We have a lambda and an application operator. All of this is syntactic sugar (just like local definitions).

```
Fun : Con → Ty → Ty → Set
Fun Γ A B = Tm (Γ ▷ A) B

lam : ∀{Γ A B} → Tm (Γ ▷ A) B → Fun Γ A B
lam t = t

_$_ : ∀{Γ A B} → Fun Γ A B → Tm Γ A → Tm Γ B
t $ u = t [ ⟨ u ⟩ ]
```

2.2 Making bindings first order via combinators

In this section we study Moses Schönfinkel's combinator calculus [21] which provides a way to make local definitions first order via higher order function space. The language described here can do more than Def (Section 2.1.3) as it has function space, so it is closer to the language STT (Chapter

3). Lambda terms can be converted to combinatory terms which have some extra equations that we don't describe here, see [2].

The combinator calculus was the earliest description of universal computation, more than ten years before Church's lambda calculus [6] and Turing machines [23]. It avoids the usage of contexts and variables which makes its definition much simpler, compare it with the substitution calculus of Section 2.1.3 corresponding to lambda calculus. However writing (and reading!) programs in SK is much harder than using lambda calculus. Here we present the simply typed variant of combinator calculus with one base type ι.

An SK model consists of two sorts (types and term), two type operations, three term operations and two term equations.

```
record Model {i j} : Set (lsuc (i ⊔ j)) where
  infixr 5 _⇒_
  infixl 5 _$_

  field
    Ty  : Set i
    ι   : Ty
    _⇒_ : Ty → Ty → Ty
    Tm  : Ty → Set j
    _$_ : ∀{A B}     → Tm (A ⇒ B) → Tm A → Tm B
    K   : ∀{A B}     → Tm (A ⇒ B ⇒ A)
    S   : ∀{A B C} → Tm ((A ⇒ B ⇒ C) ⇒ (A ⇒ B) ⇒ A ⇒ C)
    Kβ  : ∀{A B}{t : Tm A}{u : Tm B} → K $ t $ u ≡ t
    Sβ  : ∀{A B C}{t : Tm (A ⇒ B ⇒ C)}{u : Tm (A ⇒ B)}{v : Tm A} →
          S $ t $ u $ v ≡ t $ v $ (u $ v)
```

The I combinator can be derived (here we call it I' to distinguish from the initial model).

```
I' : ∀{A} → Tm (A ⇒ A)
I' {A} = S $ K $ K {A}{ι}

Iβ : ∀{A}{u : Tm A} → I' $ u ≡ u
Iβ {A}{u} =
  I' $ u
                  ≡⟨ refl ⟩
  S $ K $ K $ u
                  ≡⟨ Sβ ⟩
  K $ u $ (K $ u)
                  ≡⟨ Kβ ⟩
  u
                  ∎
```

Another famous combinator is B (which we call B').

```
B' : ∀{A B C} → Tm ((B ⇒ C) ⇒ (A ⇒ B) ⇒ A ⇒ C)
B' = S $ (K $ S) $ K

Bβ : ∀{A B C}{t : Tm (B ⇒ C)}{u : Tm (A ⇒ B)}{v : Tm A} → B' $ t $ u $ v ≡ t $ (u $ v)
Bβ {t = t}{u = u}{v = v} =
  B' $ t $ u $ v
                          ≡⟨ refl ⟩
  S $ (K $ S) $ K $ t $ u $ v
                          ≡⟨ cong {A = Tm _} (λ x → x $ u $ v) Sβ ⟩
  K $ S $ t $ (K $ t) $ u $ v
                          ≡⟨ cong {A = Tm _} (λ x → x $ (K $ t) $ u $ v) Kβ ⟩
  S $ (K $ t) $ u $ v
                          ≡⟨ Sβ ⟩
```

```
  K $ t $ v $ (u $ v)
                                  ≡⟨ cong {A = Tm _} (λ x → x $ (u $ v)) Kβ ⟩
  t $ (u $ v)
                                  ∎
```

The standard model.

```
St : Model
St = record
  { Ty  = Set
  ; Tm  = λ A → A
  ; ι   = Lift ⊤
  ; _⇒_ = λ A B → A → B
  ; _$_ = λ f x → f x
  ; K   = λ x y → x
  ; S   = λ x y z → x z (y z)
  ; Kβ  = refl
  ; Sβ  = refl
  }
module St = Model St
```

Normal forms are given as an inductive predicate over terms (as opposed to Subsection 2.1.3.3 where they are defined as sets together with a functions into terms – the two representations are related by the family–map correspondance, see [5, p. 221]).

A term is in normal form if it is a partial application of either K or S. A full application of K has two arguments, a full application of S has three arguments (and they are equal to some other term using the equations Kβ and Sβ).

```
open I
data Nf : (A : I.Ty) → I.Tm A → Set where
  K₀ : ∀{A B} → Nf (A ⇒ B ⇒ A) K
  K₁ : ∀{A B t} → Nf A t → Nf (B ⇒ A) (K $ t)
  S₀ : ∀{A B C} → Nf ((A ⇒ B ⇒ C) ⇒ (A ⇒ B) ⇒ A ⇒ C) S
  S₁ : ∀{A B C t} → Nf (A ⇒ B ⇒ C) t → Nf ((A ⇒ B) ⇒ A ⇒ C) (S $ t)
  S₂ : ∀{A B C t u} → Nf (A ⇒ B ⇒ C) t → Nf (A ⇒ B) u → Nf (A ⇒ C) (S $ t $ u)
```

The recursive normalisation in Section 2.1.3 does not work, because we cannot define application _$_ on normal forms: we would need something like S_2 t u $ v = t $ v $ (u $ v), but we only have t $ u and u $ v in the model.

The solution is Tait's method of *logical predicates* [22]: we define predicates on terms by induction on types, together with a function called *reification* wich says that if a predicate holds for a term, then it has a normal form.

For ι, the predicate is always false. For a function t the predicate says that if the predicate holds for an input u then it holds for the output t I.$ u, and t has to be in normal form.

```
Norm : DepModel
Norm = record
  { Ty•   = λ A → Σ (I.Tm A → Set) λ P → {t : I.Tm A} → P t → Nf A t
  ; Tm•   = π₁
  ; ι•    = (λ _ → Lift ⊥) , λ where ()
  ; _⇒•_  = λ {A}{B} A• B• →
            (λ t → ({u : I.Tm A} → π₁ A• u → π₁ B• (t I.$ u)) × Nf (A I.⇒ B) t) , π₂
  ; _$•_  = λ t u → π₁ t u
  ; K•    = λ {A}{B}{A•}{B•} →
            (λ t → (λ u → transp (π₁ A•) (I.Kβ ⁻¹) t) , K₁ (π₂ A• t)) , K₀
  ; S•    = λ {_}{_}{_}{_}{_}{C•} →
            (λ t → (λ u → (λ v → transp (π₁ C•) (I.Sβ ⁻¹) (π₁ (π₁ t v) (π₁ u v))
```

```
                                ) , S₂ (π₂ t) (π₂ u)
                            ) , S₁ (π₂ t)
                        ) , S₀
    ; Kβ•   = λ {A}{B}{A•}{B•}{t}{u}{t•}{u•} → transptransp (π₁ A•) (Kβ ⁻¹) {Kβ}
    ; Sβ•   = λ {A}{B}{C}{A•}{B•}{C•}{t}{u}{v}{t•}{u•}{v•} → transptransp (π₁ C•)(Sβ ⁻¹){Sβ}
    }
```

Note that Norm makes use of all the five ways to form normal forms and both equations in our language. To normalise a term of type A, we first interpret the type in the Norm dependent model. This results in a pair where the second component is the reification function which produces a normal form from any term for which the predicate holds. Interpreting the term in Norm gives us a witness of the predicate. We put together these as follows.

```
norm : ∀{A}(t : I.Tm A) → Nf A t
norm {A} t = π₂ Norm.⟦ A ⟧T Norm.⟦ t ⟧t

test : ∀{A B}(u : I.Tm A) → norm (I.K {A}{B} I.$ u) ≡ K₁ (norm u)
test u = refl
```

We prove completeness by another induction. Stability is proven by induction on normal forms.

```
stab : ∀{A t}(n : Nf A t) → norm t ≡ n
stab K₀          = refl
stab (K₁ n)      = cong K₁ (stab n)
stab S₀          = refl
stab (S₁ n)      = cong S₁ (stab n)
stab (S₂ n n')   = cong (λ w → S₂ (π₁ w) (π₂ w)) (stab n ,× stab n')
```

Decidability of equality of normal forms is proven by double-induction on types and normal forms.

```
_≟Ty_ : (A B : Ty) → Dec (Lift (A ≡ B))
_≟_   : ∀{A₀ A₁ t₀ t₁}(v₀ : Nf A₀ t₀)(v₁ : Nf A₁ t₁) →
  Dec (Lift (_≡_ {A = Σ Ty λ A → Σ (Tm A) (Nf A)} (A₀ , t₀ , v₀) (A₁ , t₁ , v₁)))
```

We obtain decidability of equality for all terms by checking whether the normal forms are equal.

```
_≟Tm_ : ∀{A}(t₀ t₁ : Tm A) → Dec (Lift (t₀ ≡ t₁))
t₀ ≟Tm t₁ with norm t₀ ≟ norm t₁
... | ι₁ e = ι₁ (mk (congd (λ z → π₁ (π₂ z)) (un e)))
... | ι₂ ne = ι₂ (mk λ e → un ne (mk (cong (λ z → (_ , z , norm z)) (un e))))
```

TODO: examples of normalisation, sometimes there is an infinite loop (?)

Part II

Type systems

Chapter 3

Higher order function space

Learning goals of this chapter

Rules for function space. Uniqueness rule. Currying. Relationship of metatheoretic and object theoretic function space. Normal forms when we have η rule. Bidirectional type-checking.

We saw in Subsection 2.1.3.4 that the language Def already has first order function space, but this is not first class, for example there is no type of first order functions. Now we extend Def with arbitrary higher order functions which are first class, e.g. they can be passed around as arguments to functions. An STT-model is a Def-model extended with the following components:

```
_⇒_   : Ty → Ty → Ty
lam   : ∀{Γ A B} → Tm (Γ ▷ A) B → Tm Γ (A ⇒ B)
_$_   : ∀{Γ A B} → Tm Γ (A ⇒ B) → Tm Γ A → Tm Γ B
⇒β    : ∀{Γ A B}{t : Tm (Γ ▷ A) B}{u : Tm Γ A} → lam t $ u ≡ t [ ⟨ u ⟩ ]
⇒η    : ∀{Γ A B}{t : Tm Γ (A ⇒ B)} → t ≡ lam (t [ p ] $ var vz)
lam[] : ∀{Γ A B}{t : Tm (Γ ▷ A) B}{Δ}{σ : Sub Δ Γ} → lam t [ σ ] ≡ lam (t [ σ ⁺ ])
$[]   : ∀{Γ A B}{t : Tm Γ (A ⇒ B)}{u : Tm Γ A}{Δ}{σ : Sub Δ Γ} →
        (t $ u) [ σ ] ≡ (t [ σ ]) $ (u [ σ ])
```

In STT (simple type theory), there are three ways to form types: Bool, Nat, and for any two types A and B, we also have a type A ⇒ B. So types are binary trees with Nat or Bool at the leaves. lam is a binder, it binds a variable (of type A) in its first (and only) explicit argument. The operator _⇒_ is right-associative, e.g. A ⇒ B ⇒ C = A ⇒ (B ⇒ C). To match this, _$_ is left-associative, e.g. t $ u $ v = (t $ u) $ v

We group the operators and equations for _⇒_:

- type introduction: _⇒_,
- constructor (called abstraction): lam,
- destructor (called application): _$_,
- computation: ⇒β,
- uniqueness: ⇒η,

A concise way to state the rules for function space is the following natural isomorphism, the right to left direction can be defined using _$_.

lam : Tm (Γ ▷ A) B ≅ Tm Γ (A ⇒ B) : app

An example function with an example on how it computes:

```
add2 : Tm ⋄ (Nat ⇒ Nat)
add2 = lam (v0 +o num 2)

add2$3 : add2 $ num 3 ≡ num 5
add2$3 =
  add2 $ num 3
                                        ≡⟨ refl ⟩
  lam (v0 +o num 2) $ num 3
                                        ≡⟨ ⇒β ⟩
  (v0 +o num 2) [ ⟨ num 3 ⟩ ]
                                        ≡⟨ +[] ⟩
  (v0 [ ⟨ num 3 ⟩ ]) +o (num 2 [ ⟨ num 3 ⟩ ])
                                        ≡⟨ cong (_+o (num 2 [ ⟨ num 3 ⟩ ])) vz[⟨⟩] ⟩
  num 3 +o (num 2 [ ⟨ num 3 ⟩ ])
                                        ≡⟨ cong (num 3 +o_) num[] ⟩
  num 3 +o num 2
                                        ≡⟨ +β ⟩
  num 5
                                        ∎
```

Exercise 3.1 (compulsory)**.** *Derive the typing of*

```
apply2x : Tm ⋄ ((Nat ⇒ Nat) ⇒ Nat ⇒ Nat)
apply2x = lam (lam (v1 $ (v1 $ v0)))
```

Exercise 3.2 (compulsory)**.** *What is the type of the following terms?*

```
lam (isZero (ite v0 (num 1) (num 0)))
lam (lam (ite v0 (isZero v1) false))
lam (lam (v1 +o (v0 $ v1)))
lam (lam (num 1 +o (v1 $ v0)))
```

Exercise 3.3 (compulsory)**.** *Prove*

```
apply2x-example : apply2x $ add2 $ num 3 ≡ num 7
```

Representing two-parameter functions as iterated function space as in the following exercise is named after Haskell Curry and is called *currying*.

Exercise 3.4 (compulsory)**.** *Define the following functions on booleans.*

```
and : Tm ⋄ (Bool ⇒ Bool ⇒ Bool)
or  : Tm ⋄ (Bool ⇒ Bool ⇒ Bool)
neg : Tm ⋄ (Bool ⇒ Bool)
```

Exercise 3.5 (compulsory)**.** *Define the identity function and function composition, and show the following equations relating them.*

```
id   : ∀{Γ A} → Tm Γ (A ⇒ A)
comp : ∀{Γ A B C} → Tm Γ ((B ⇒ C) ⇒ (A ⇒ B) ⇒ A ⇒ C)
idl  : ∀{Γ A B}{t : Tm Γ (A ⇒ B)} → comp $ id $ t ≡ t
idr  : ∀{Γ A B}{t : Tm Γ (A ⇒ B)} → comp $ t $ id ≡ t
ass  : ∀{Γ A B C D}{t : Tm Γ (A ⇒ B)}{t' : Tm Γ (B ⇒ C)}{t'' : Tm Γ (C ⇒ D)} →
       comp $ (comp $ t'' $ t') $ t ≡ comp $ t'' $ (comp $ t' $ t)
```

Exercise 3.6 (recommended). *Define Moses Schönfinkel's* K *and* S *combinators and show their computation rules (see also Section 2.2).*

```
K  : ∀{Γ A B} → Tm Γ (A ⇒ B ⇒ A)
S  : ∀{Γ A B C} → Tm Γ ((A ⇒ B ⇒ C) ⇒ (A ⇒ B) ⇒ A ⇒ C)
Kβ : ∀{Γ A B}{u : Tm Γ A}{v : Tm Γ B} → K $ u $ v ≡ u
Sβ : ∀{Γ A B C}{t : Tm Γ (A ⇒ B ⇒ C)}{u : Tm Γ (A ⇒ B)}{v : Tm Γ A} →
     S $ t $ u $ v ≡ t $ v $ (u $ v)
```

Exercise 3.7 (recommended). *Derive the "categorical application operator"* ap*, show that it is the inverse of* lam*. Also show its substitution rule* app[].

```
app   : ∀{Γ A B} → Tm Γ (A ⇒ B) → Tm (Γ ▷ A) B
appβ  : ∀{Γ A B}{t : Tm (Γ ▷ A) B} → app (lam t) ≡ t
appη  : ∀{Γ A B}{t : Tm Γ (A ⇒ B)} → lam (app t) ≡ t
app[] : ∀{Γ A B}{t : Tm Γ (A ⇒ B)}{Δ}{σ : Sub Δ Γ} → app t [ σ ⁺ ] ≡ app (t [ σ ])
```

Exercise 3.8 (compulsory). *How many different elements does* Tm ◇ (Bool ⇒ Bool) *have? Define them! (See also normalisation below.)*

Exercise 3.9 (compulsory). *Compare the sets* Tm ◇ (Bool ⇒ Bool) *and* Tm ◇ Bool → Tm ◇ Bool*. Is there a logical equivalence between them?*

Exercise 3.10 (compulsory). *Internalise* isZero*, that is, define* isZero' *as below. Similarly, internalise the functions* _+o_ *and* ite*.*

```
isZero' : ∀{Γ} → Tm Γ (Nat ⇒ Bool)
add'    : ∀{Γ} → Tm Γ (Nat ⇒ Nat ⇒ Nat)
ite'    : ∀{Γ A} → Tm Γ (Bool ⇒ A ⇒ A ⇒ A)
```

Exercise 3.11 (very hard). *Show that for any model,* Tm Γ (A ⇒ B) *is isomorphic to* Σ (∀{Δ}(γ : PSub Δ Γ) → Tm Δ A → Tm Δ B) λ f → ∀{γ a δ} → f γ a [δ]P ≡ f (γ ∘ δ) (a [δ]P)*, where* PSub *is a sort of parallel substitutions which have composition. A* PSub Δ (◇ ▷ A_1 ▷ ... A_n) *is a list of* n *terms all in context* Δ *of types* A_1*, ...,* A_n*, respectively.* _[_]P *is instantiation of terms with parallel substitutions, this can be defined by iterating* _[_].

3.1 Standard model

```
; _⇒_  = λ A B → A → B
; lam  = λ t γ → λ a → t (γ , a)
; _$_  = λ t u γ → t γ (u γ)
; ⇒β   = refl
; ⇒η   = refl
; lam[] = refl
; $[]   = refl
```

3.2 Normalisation

Normal forms are like in Def, but there is a twist.

```
infixl 5 _$Ne_

data Ne : Con → Ty → Set
data Nf : Con → Ty → Set
```

```
data Ne where
  varNe     : ∀{Γ A} → Var Γ A → Ne Γ A
  iteNe     : ∀{Γ A} → Ne Γ Bool → Nf Γ A → Nf Γ A → Ne Γ A
  isZeroNe  : ∀{Γ} → Ne Γ Nat → Ne Γ Bool
  _+NeNe_ : ∀{Γ} → Ne Γ Nat → Ne Γ Nat → Ne Γ Nat
  _+NeNf_ : ∀{Γ} → Ne Γ Nat → ℕ → Ne Γ Nat
  _+NfNe_ : ∀{Γ} → ℕ → Ne Γ Nat → Ne Γ Nat
  _$Ne_     : ∀{Γ A B} → Ne Γ (A ⇒ B) → Nf Γ A → Ne Γ B

data Nf where
  neuBool   : ∀{Γ} → Ne Γ Bool → Nf Γ Bool
  neuNat    : ∀{Γ} → Ne Γ Nat → Nf Γ Nat
  trueNf    : ∀{Γ} → Nf Γ Bool
  falseNf   : ∀{Γ} → Nf Γ Bool
  numNf     : ∀{Γ} → ℕ → Nf Γ Nat
  lamNf     : ∀{Γ A B} → Nf (Γ ▷ A) B → Nf Γ (A ⇒ B)

⌜_⌝Ne : ∀{Γ A} → Ne Γ A → Tm Γ A
⌜_⌝Nf : ∀{Γ A} → Nf Γ A → Tm Γ A
```

Only neutral terms of base types (Bool and Nat) are included in normal forms (this was also true in Def because there were no other types). Without this restriction, we would have two different normal forms for a variabe of a function type: q which is equal to lam (q [p]) $ q. We have to do this whenever we have uniqueness rules.

Exercise 3.12. *What are the normal form of the following terms?*

```
t1 : Σsp (Nf (◇ ▷ Nat ⇒ Bool) (Nat ⇒ Bool)) λ v → ⌜ v ⌝Nf ≡ v0
t2 : Σsp (Nf (◇ ▷ Bool) Bool) λ v → ⌜ v ⌝Nf ≡ v0
t3 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → ⌜ v ⌝Nf ≡ v0
t4 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → ⌜ v ⌝Nf ≡ num 1 +o v0
t5 : Σsp (Nf ◇ Nat) λ v → ⌜ v ⌝Nf ≡ num 1 +o num 2
t6 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → ⌜ v ⌝Nf ≡ isZero (v0 +o num 1)
t7 : Σsp (Nf ◇ Bool) λ v → ⌜ v ⌝Nf ≡ isZero (num 2)
t8 : Σsp (Nf ◇ Bool) λ v → ⌜ v ⌝Nf ≡ lam (isZero v0) $ num 0
t9 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → ⌜ v ⌝Nf ≡ lam (isZero v0) $ v0
t10 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → ⌜ v ⌝Nf ≡ lam (isZero v0) $ (v0 +o num 1)
```

Recursive normalisation as for Def does not work as for STT. This is because when defining normal application we need instantiation and when defining instantiation, we need normal application and it is not obvious that such mutual definition terminates. [1]

```
_$Nf_  : Nf Γ (A ⇒ B) → Nf Γ A → Nf Γ B
_[_]Ne : Ne Γ A → Sub Δ Γ → Nf Δ A
lamNf t $Nf u = t [ ⟨ u ⟩ ]Nf
(t $Ne a) [ γ ]Ne = t [ γ ]Ne $Nf a [ γ ]Ne
```

However we can use the method of logical predicates (like in Section 2.2). Instead of proving normalisation by induction on terms, we do induction on types as well. For each type, we define a predicate on terms of that type. For contexts we define a predicate on lists of terms with types in that context by iterating the predicate for the types. For terms Tm Γ A, we show by induction that they respect the predicate: if the predicate holds for a list of terms with types in Γ, then the predicate at A holds for the term substituted by the list of terms. Similarly, substitutions respect the predicate. Then we show by induction on types that if the predicate holds for a term, then it is in normal form.

TODO: show full normalisation.

TODO: bidirectional type checking

[1] It does terminate and normalisation can be defined this way, this is called hereditary substitution [15].

Chapter 4

Product and sum types

Learning goals of this chapter

Nullary and binary products. Nullary and binary sums. Applications: enumerations, booleans encoded as sums, option type. Types form an exponential rig. More equalities in the standard model than in the syntax.

A Fin model consists of the substitution calculus (Def without Bool and Nat), components for function space and components for the following four new type formers.

- Binary products:

```
_×o_  : Ty → Ty → Ty
_,o_  : ∀{Γ A B} → Tm Γ A → Tm Γ B → Tm Γ (A ×o B)
fst   : ∀{Γ A B} → Tm Γ (A ×o B) → Tm Γ A
snd   : ∀{Γ A B} → Tm Γ (A ×o B) → Tm Γ B
×β₁   : ∀{Γ A B}{u : Tm Γ A}{v : Tm Γ B} → fst (u ,o v) ≡ u
×β₂   : ∀{Γ A B}{u : Tm Γ A}{v : Tm Γ B} → snd (u ,o v) ≡ v
×η    : ∀{Γ A B}{t : Tm Γ (A ×o B)} → t ≡ (fst t ,o snd t)
,[]   : ∀{Γ A B}{u : Tm Γ A}{v : Tm Γ B}{Δ}{γ : Sub Δ Γ} →
        (u ,o v) [ γ ] ≡ (u [ γ ] ,o v [ γ ])
```

A term of the binary product type A ×o B (other notations: A × B, record { a : A , b : B }) is an ordered pair of a term of type A and another of type B. The constructor is pairing, and there are two destructors: first and second projections. The computation rules say what happens if we project out the first or second element of a pair, the uniqueness rule says that any term of the product type is a pair. We only need a substitution rule for the constructor, the substitution rules for the destructors can be proven:

```
fst t [ γ ]
                                  ≡⟨ ×β₁ ⁻¹ ⟩
fst (fst t [ γ ] ,o snd t [ γ ])
                                  ≡⟨ cong fst (,[] ⁻¹) ⟩
fst ((fst t ,o snd t) [ γ ])
                                  ≡⟨ cong (λ z → fst (z [ γ ])) (×η ⁻¹) ⟩
fst (t [ γ ])
                                  ∎
```

The rules for binary products can be summarised by the following isomorphism:

Tm Γ (A ×o B) ≅ Tm Γ A × Tm Γ B

The left to right direction is given by fst and snd, the right to left direction by _,o_, the proofs that the roundtrips are identity are ×β and ×η.

- Nullary products:

```
Unit   : Ty
trivial : ∀{Γ} → Tm Γ Unit
Unitη  : ∀{Γ}{u : Tm Γ Unit} → u ≡ trivial
```

The nullary product is the unit (top, ()) type with only one constructor and no destructor: this shows that there is no information in a term of type Unit, there is no way to use such a term. This is why in some programming languages this type is called void – when a function does not return a value, its return type is unit. The summarising isomorphism: Tm Γ Unit ≅ ⊤.

- Binary sums:

```
_+o_   : Ty → Ty → Ty
inl    : ∀{Γ A B} → Tm Γ A → Tm Γ (A +o B)
inr    : ∀{Γ A B} → Tm Γ B → Tm Γ (A +o B)
caseo  : ∀{Γ A B C} → Tm (Γ ▷ A) C → Tm (Γ ▷ B) C → Tm Γ (A +o B) → Tm Γ C
+β₁    : ∀{Γ A B C}{c₁ : Tm (Γ ▷ A) C}{c₂ : Tm (Γ ▷ B) C}{a : Tm Γ A} →
         caseo c₁ c₂ (inl a) ≡ c₁ [ ⟨ a ⟩ ]
+β₂    : ∀{Γ A B C}{c₁ : Tm (Γ ▷ A) C}{c₂ : Tm (Γ ▷ B) C}{b : Tm Γ B} →
         caseo c₁ c₂ (inr b) ≡ c₂ [ ⟨ b ⟩ ]
+η     : ∀{Γ A B C}{c : Tm (Γ ▷ A +o B) C} →
         c ≡ caseo (c [ p⁺ ] [ ⟨ inl (var vz) ⟩ ] [ p⁺ ])
                   (c [ p⁺ ] [ ⟨ inr (var vz) ⟩ ] [ p⁺ ]) (var vz)
inl[]  : ∀{Γ A B}{a : Tm Γ A}{Δ}{γ : Sub Δ Γ} → inl {B = B} a [ γ ] ≡ inl (a [ γ ])
inr[]  : ∀{Γ A B}{b : Tm Γ B}{Δ}{γ : Sub Δ Γ} → inr {A = A} b [ γ ] ≡ inr (b [ γ ])
case[] : ∀{Γ A B C}{c₁ : Tm (Γ ▷ A) C}{c₂ : Tm (Γ ▷ B) C}{ab : Tm Γ (A +o B)}
         {Δ}{γ : Sub Δ Γ} →
         (caseo c₁ c₂ ab) [ γ ] ≡ caseo (c₁ [ γ⁺ ]) (c₂ [ γ⁺ ]) (ab [ γ ])
```

caseo binds a variable of type A in its first explicit argument and a variable of type B in its second explicit argument. They are specified by the following isomorphism which is natural both in Γ and C (this means that we have substitution laws for both Γ and C, see below).

(_[p⁺][⟨inl v0⟩], _[p⁺][⟨inr v0⟩]) : Tm (Γ ▷ A +o B) C ≅ Tm (Γ ▷ A) C × Tm (Γ ▷ B) C : caseo

- Nullary sums:

```
Empty   : Ty
absurd  : ∀{Γ A} → Tm Γ Empty → Tm Γ A
Emptyη  : ∀{Γ A}{t : Tm (Γ ▷ Empty) A} → t ≡ absurd (var vz)
absurd[] : ∀{Γ A}{t : Tm Γ Empty}{Δ}{γ : Sub Δ Γ} →
           absurd {A = A} t [ γ ] ≡ absurd (t [ γ ])
```

The isomorphism: Tm (Γ ▷ Empty) A ≅ ⊤.

Exercise 4.1 (recommended)**.** *Prove the substitution rule for the other destructor for products.*

```
snd[] : ∀{Γ A B}{t : Tm Γ (A ×o B)}{Δ}{γ : Sub Δ Γ} → snd t [ γ ] ≡ snd (t [ γ ])
```

Exercise 4.2 (compulsory)**.** *Prove the substitution rule for the constructor of the unit type.*

```
trivial[] : ∀{Γ Δ}{γ : Sub Δ Γ} → trivial [ γ ] ≡ trivial
```

We define the components of the isomorphism for sums, prove that the roundrips are identities and the naturality equations.

```
left  : ∀{Γ A B C} → Tm (Γ ▷ A +o B) C → Tm (Γ ▷ A) C
left c = c [ p ⁺ ] [ ⟨ inl v0 ⟩ ]
right : ∀{Γ A B C} → Tm (Γ ▷ A +o B) C → Tm (Γ ▷ B) C
right c = c [ p ⁺ ] [ ⟨ inr v0 ⟩ ]
case' : ∀{Γ A B C} → Tm (Γ ▷ A) C → Tm (Γ ▷ B) C → Tm (Γ ▷ A +o B) C
case' c₁ c₂ = caseo (c₁ [ p ⁺ ]) (c₂ [ p ⁺ ]) v0
+η' : ∀{Γ A B C}{t : Tm (Γ ▷ A +o B) C} →
        t ≡ case' (left t) (right t)
+η' {t = t} =
  t
                                  ≡⟨ +η ⟩
  caseo (t [ p ⁺ ] [ ⟨ inl (var vz) ⟩ ] [ p ⁺ ]) (t [ p ⁺ ] [ ⟨ inr (var vz) ⟩ ] [ p ⁺ ]) (var vz)
                                  ≡⟨ refl ⟩
  case' (left t) (right t)
                                  ∎
+β₁' : ∀{Γ A B C}{u : Tm (Γ ▷ A) C}{v : Tm (Γ ▷ B) C} →
        left (case' u v) ≡ u

[]case : ∀{Γ A B C}{u : Tm (Γ ▷ A) C}{v : Tm (Γ ▷ B) C}{D}{w : Tm (Γ ▷ C) D} →
         w [ p ⁺ ] [ ⟨ case' u v ⟩ ] ≡ case' (w [ p ⁺ ] [ ⟨ u ⟩ ]) (w [ p ⁺ ] [ ⟨ v ⟩ ])

+β₂' : ∀{Γ A B C}{u : Tm (Γ ▷ A) C}{v : Tm (Γ ▷ B) C} →
        right (case' u v) ≡ v

case[]' : ∀{Γ A B C}{u : Tm (Γ ▷ A) C}{v : Tm (Γ ▷ B) C}{Δ}{γ : Sub Δ Γ} →
          (case' u v) [ γ ⁺ ] ≡
          case' (u [ γ ⁺ ]) (v [ γ ⁺ ])

[]left : ∀{Γ A B C}{t : Tm (Γ ▷ A +o B) C}{D}{w : Tm (Γ ▷ C) D} →
         w [ p ⁺ ] [ ⟨ left t ⟩ ] ≡ left (w [ p ⁺ ] [ ⟨ t ⟩ ])
[]left = exercisep

[]right : ∀{Γ A B C}{t : Tm (Γ ▷ A +o B) C}{D}{w : Tm (Γ ▷ C) D} →
          w [ p ⁺ ] [ ⟨ right t ⟩ ] ≡ right (w [ p ⁺ ] [ ⟨ t ⟩ ])

left[] : ∀{Γ A B C}{t : Tm (Γ ▷ A +o B) C}{Δ}{γ : Sub Δ Γ} →
         left t [ γ ⁺ ] ≡ left (t [ γ ⁺ ])
```

4.1 Applications

Enumerations can be defined using the unit and sum types. For example, the type with three elements:

```
Three : Ty
Three = Unit +o (Unit +o Unit)
```

Its three elements:

```
three0 three1 three2 : ∀{Γ} → Tm Γ Three
three0 = inl trivial
three1 = inr (inl trivial)
three2 = inr (inr trivial)
```

Elimination principle of Three.

Exercise 4.3 (recommended)**.** *The second and third rules are exercises.*

```
caseThree : ∀{Γ A} → Tm Γ Three → Tm Γ A → Tm Γ A → Tm Γ A → Tm Γ A
caseThree t u v w = caseo (u [ p ]) (caseo (v [ p ] [ p ]) (w [ p ] [ p ]) v0) t
Threeβ0 : ∀{Γ A}{u v w : Tm Γ A} → caseThree three0 u v w ≡ u
Threeβ1 : ∀{Γ A}{u v w : Tm Γ A} → caseThree three1 u v w ≡ v
Threeβ2 : ∀{Γ A}{u v w : Tm Γ A} → caseThree three2 u v w ≡ w
Threeβ0 {Γ}{a}{u}{v}{w} =
  caseThree three0 u v w
                          ≡⟨ refl ⟩
  caseo (u [ p ]) (caseo (v [ p ] [ p ]) (w [ p ] [ p ]) v0) (inl trivial)
                          ≡⟨ +β₁ ⟩
  u [ p ] [ ⟨ trivial ⟩ ]
                          ≡⟨ [p][⟨⟩] ⟩
  u
                          ∎
```

Addition modulo 3:

```
plus3 : Tm ◇ (Three ⇒ Three ⇒ Three)
plus3 = lam (lam (caseThree v1
  {- v1=0 -} v0
  {- v1=1 -} (caseThree v0 three1 three2 three0)
  {- v1=2 -} (caseThree v0 three2 three0 three1)))
```

Exercise 4.4. *Prove the computation rules for* plus3*.*

```
plus3test00 : plus3 $ three0 $ three0 ≡ three0
plus3test11 : plus3 $ three1 $ three1 ≡ three2
plus3test12 : plus3 $ three1 $ three2 ≡ three0
```

We define booleans. The usual eliminator (if-then-else) is defined using case.

```
Bool : Ty
Bool = Unit +o Unit

true : ∀{Γ} → Tm Γ Bool
true = inl trivial

false : ∀{Γ} → Tm Γ Bool
false = inr trivial

ite : ∀{Γ A} → Tm Γ Bool → Tm Γ A → Tm Γ A → Tm Γ A
ite b u v = caseo (u [ p ]) (v [ p ]) b
Boolβ₁ : ∀{Γ A}{u v : Tm Γ A} → ite true u v ≡ u
Boolβ₁ {Γ}{A}{u}{v} =
  ite true u v
    ≡⟨ refl ⟩
  caseo (u [ p ]) (v [ p ]) (inl trivial)
    ≡⟨ +β₁ ⟩
  u [ p ] [ ⟨ trivial ⟩ ]
    ≡⟨ [p][⟨⟩] ⟩
  u
    ∎
```

Exercise 4.5. *Prove the other computation rule.*

```
Boolβ₂ : ∀{Γ A}{u v : Tm Γ A} → ite false u v ≡ v
```

Exercise 4.6. *Prove the following uniqueness rule for booleans.*

```
Boolη : ∀{Γ A}{t : Tm (Γ ▷ Bool) A} → t ≡ ite v0 (t [ ⟨ true ⟩ ] [ p ]) (t [ ⟨ false ⟩ ] [ p ])
```

Exercise 4.7. *With the help of* Boolη, *show that the following two implementations of negation are equal.*

```
ors= : lam {◇} (ite v0 false true) ≡ lam (ite v0 false (ite v0 true true))
```

Exercise 4.8 (hard)**.** *Show that there are only four functions in* Tm ◇ (Bool ⇒ Bool)*!*

With Bool and binary products we can simulate homogeneous binary sums:

```
2*_ : Ty → Ty
2* A = Bool ×o A
inl' inr' : ∀{Γ A} → Tm Γ A → Tm Γ (2* A)
inl' t = (true ,o t)
inr' t = (false ,o t)
case2 : ∀{Γ A C} → Tm Γ (2* A) → Tm (Γ ▷ A) C → Tm (Γ ▷ A) C → Tm Γ C
case2 t u v = ite (fst t) (u [ ⟨ snd t ⟩ ]) (v [ ⟨ snd t ⟩ ])
2*β₁ : ∀{Γ A C}{t : Tm Γ A}{u v : Tm (Γ ▷ A) C} → case2 (inl' t) u v ≡ u [ ⟨ t ⟩ ]
2*β₁ {Γ}{A}{C}{t}{u}{v} =
  case2 (inl' t) u v
    ≡⟨ refl ⟩
  ite (fst (true ,o t)) (u [ ⟨ snd (inl' t) ⟩ ]) (v [ ⟨ snd (inl' t) ⟩ ])
    ≡⟨ cong (λ x → ite x (u [ ⟨ snd (inl' t) ⟩ ]) (v [ ⟨ snd (inl' t) ⟩ ])) ×β₁ ⟩
  ite true (u [ ⟨ snd (inl' t) ⟩ ]) (v [ ⟨ snd (inl' t) ⟩ ])
    ≡⟨ Boolβ₁ ⟩
  u [ ⟨ snd (inl' t) ⟩ ]
    ≡⟨ refl ⟩
  u [ ⟨ snd (true ,o t) ⟩ ]
    ≡⟨ cong (λ x → u [ ⟨ x ⟩ ]) ×β₂ ⟩
  u [ ⟨ t ⟩ ]
    ∎
```

Exercise 4.9. *Prove the other* β *rule.*

```
2*β₂ : ∀{Γ A C}{t : Tm Γ A}{u v : Tm (Γ ▷ A) C} → case2 (inr' t) u v ≡ v [ ⟨ t ⟩ ]
```

The option type former (sometimes called maybe) adds an extra element to a type:

```
Opt : Ty → Ty
Opt A = A +o Unit
none : ∀{Γ A} → Tm Γ (Opt A)
none = inr trivial
some : ∀{Γ A} → Tm Γ A → Tm Γ (Opt A)
some t = inl t
```

A type with n elements:

```
Fino : ℕ → Ty
Fino zero = Unit
Fino (suc n) = Unit +o Fino n
```

Exercise 4.10. *For each* n, *define the elimination principle of* Fino n.

For a natural number n, we define n-ary products which are parameterised by n types.

```
Tys : ℕ → Set
Tys zero    = Raise (Lift ⊤)
Tys (suc n) = Ty × Tys n

Prod : {n : ℕ} → Tys n → Ty
Prod {zero} _           = Unit
Prod {suc n} (A , As) = A ×o Prod As
```

The constructor takes n terms and turns them into a term.

```
Tms : Con → {n : ℕ} → Tys n → Set
Tms Γ {zero} _           = Raise (Lift ⊤)
Tms Γ {suc n} (A , As) = Tm Γ A × Tms Γ As

tpl : ∀{Γ n}{As : Tys n} → Tms Γ As → Tm Γ (Prod As)
tpl {Γ} {zero}  _       = trivial
tpl {Γ} {suc n} (t , ts) = (t ,o tpl ts)
```

We have n different projections.

```
_!_ : ∀{n} → Tys n → Fin n → Ty
(A , As) ! zero  = A
(A , As) ! suc i = As ! i

prj : ∀{Γ n}{As : Tys n}(i : Fin n) → Tm Γ (Prod As) → Tm Γ (As ! i)
prj zero    t = fst t
prj (suc i) t = prj i (snd t)
```

Exercise 4.11. *State and prove the computation rules and the substitution laws.*

Exercise 4.12. *Define* n-*ary coproducts.*

4.2 Algebraic properties of types

Types form a commutative exponential semiring (commutative exponential rig) up to isomorphism. Isomorphism of two types is defined as follows.

```
record _≅_ (A B : Ty) : Set where
  field
    f : Tm (◇ ▷ A) B
    g : Tm (◇ ▷ B) A
    fg : f [ p ⁺ ] [ ⟨ g ⟩ ] ≡ v0
    gf : g [ p ⁺ ] [ ⟨ f ⟩ ] ≡ v0
open _≅_
infix 4 _≅_
```

Exercise 4.13 (compulsory)**.** *Show that for any type* A, A ×o Unit *is isormophic to* A.

```
×idr : ∀{A} → A ×o Unit ≅ A
```

Exercise 4.14 (compulsory)**.** *Show that for any two types* A, B, *there is an isomorphism between* A +o B *and* B +o A.

```
+ocomm : ∀{A B} → A +o B ≅ B +o A
```

The following exercise is a type theoretic version of the algebraic equation $(a+b)^2 = a^2 + 2*a*b + b^2$.

Exercise 4.15 (compulsory)**.** *Show that for any two types* A, B, *there is an isomorphism between* (Unit +o Unit) ⇒ (A +o B) *and* ((Unit +o Unit) ⇒ A +o (Unit +o Unit) ×o A ×o B) +o (Unit +o Unit) ⇒ B. *At least define the first two components (*f *and* g*).*

Exercise 4.16 (some parts are compulsory)**.** *Show that up to isomorphism* _≅_*, types form a commutative exponential semiring (exponential rig).*

4.3 Standard model

The new components:

```
; _×o_      = _×_
; _,o_      = λ a b γ → a γ , b γ
; fst       = λ ab γ → π₁ (ab γ)
; snd       = λ ab γ → π₂ (ab γ)
; ×β₁       = refl
; ×β₂       = refl
; ×η        = refl
; ,[]       = refl
; Unit      = Lift ⊤
; trivial   = _
; Unitη     = refl
; _+o_      = _⊎_
; inl       = λ a γ → ι₁ (a γ)
; inr       = λ b γ → ι₂ (b γ)
; caseo     = λ c₁ c₂ ab γ → case (ab γ) (λ a → c₁ (γ , a)) (λ b → c₂ (γ , b))
; +β₁       = refl
; +β₂       = refl
; +η        = funext λ { (_ , ι₁ _) → refl ; (_ , ι₂ _) → refl }
; inl[]     = refl
; inr[]     = refl
; case[]    = refl
; Empty     = Lift ⊥
; absurd    = λ t γ → exfalso (un (t γ))
; Emptyη    = funext λ ()
; absurd[]  = refl
```

Some equalities of the standard model are not definitional: those which are not implemented in the metatheory (Agda).

Some equalities which hold in the standard model are not present in the syntax, for example:

```
e1 : St.Con ≡ St.Ty
e2 : ∀{Γ Δ} → St.Sub Δ Γ ≡ St.Tm Δ Γ
e4 : ∀{Γ A Δ} → _≡_ {A = St.Sub Δ (Γ St.▷ A) → St.Tm Δ A} (St.var St.vz St.[_]) St.snd
e5 : ∀{A} → St.Tm St.◇ (St.Unit St.×o A) ≡ St.Sub St.◇ (St.◇ St.▷ A)
```

TODO: normalisation. what are normal forms?

Chapter 5

Inductive types

Learning goals of this chapter

Inductive types by examples: natural numbers, unit, lists, binary trees. Type formers, constructors, iterator, recursor, defining functions by iteration. Inductive types don't have η rules. Definability, examples of definable and non definable functions.

TODO: example: syntax as an inductive type.

TODO: add mutual inductive types, e.g. Odd and Even.

An inductive type is specified by its constructors. Its destructor and computation rules are determined by the constructors: the destructor says that for any type C (called motive) and elements of that type which have the shape of the constructors (these are called methods), there is a function (called iterator, fold, catamorphism) from the inductive type to C. The computation rules say that if we apply the iterator to a constructor, we obtain the corresponding method.

We have already seen some inductive types: Bool had two constructors true and false, and its iterator was called if-then-else, it said that for any type C and two elements of C, there is a function from Bool to C. For any two types A and B, the sum A +o B was an inductive type with constructors inl and inr. Its iterator was called caseo, it said that for any type C and elements A → C and B → C, there is a function from A +o B to C. In fact, sums also came with an η rule which is not usual for inductive types.

Natural numbers are also an inductive type: the constructors are zero and suc, the latter can be written in several different equivalent ways:

suc : Tm Γ Nat → Tm Γ Nat
suc : Tm Γ (Nat ⇒ Nat)
suc : Tm (Γ ▷ Nat) Nat

When specifying the arguments of the iterator, we use the last version which does not refer to metatheoretic or object theoretic function space. So the iterator says that given a type C, a term in Tm Γ C and a term in Tm (Γ ▷ C) C, we get a function from Tm Γ Nat to Tm Γ C.

A language has natural numbers if it extends the substitution calculus with the following operators and equations:

```
Nat      : Ty
zeroo    : ∀{Γ} → Tm Γ Nat
suco     : ∀{Γ} → Tm Γ Nat → Tm Γ Nat
iteNat   : ∀{Γ A} → Tm Γ A → Tm (Γ ▷ A) A → Tm Γ Nat → Tm Γ A
Natβ₁    : ∀{Γ A}{u : Tm Γ A}{v : Tm (Γ ▷ A) A} → iteNat u v zeroo ≡ u
Natβ₂    : ∀{Γ A}{u : Tm Γ A}{v : Tm (Γ ▷ A) A}{t : Tm Γ Nat} →
           iteNat u v (suco t) ≡ v [ ⟨ iteNat u v t ⟩ ]
zero[]   : ∀{Γ Δ}{γ : Sub Δ Γ} → zeroo [ γ ] ≡ zeroo
suc[]    : ∀{Γ}{t : Tm Γ Nat}{Δ}{γ : Sub Δ Γ} → (suco t) [ γ ] ≡ suco (t [ γ ])
```

```
iteNat[] : ∀{Γ A}{u : Tm Γ A}{v : Tm (Γ ▷ A) A}{t : Tm Γ Nat}{Δ}{γ : Sub Δ Γ} →
           iteNat u v t [ γ ] ≡ iteNat (u [ γ ]) (v [ γ ⁺ ]) (t [ γ ])
```

iteNat binds a variable of type A in its second argument. The computation rules Natβ$_1$ and Natβ$_2$ express that iteNat u v t works as follows: it replaces all sucos in t by what is specified by v, and replaces zeroo by u. For example, iteNat u v (suco (suco (suco zero))) is equal to v [⟨ v [⟨ v [⟨ u ⟩] ⟩] ⟩] (which can be thought about as v (v (v u)) but we use substitution to express the applications).

Gödel's System T is the name of the language which has function space ⇒ and natural numbers as above.

The simplest inductive type is that with only one constructor trivial. Note that this is not the same as the Unit type in the previous chapter, where it had an η rule.

```
Unit     : Ty
trivial  : ∀{Γ} → Tm Γ Unit
iteUnit  : ∀{Γ A} → Tm Γ A → Tm Γ Unit → Tm Γ A
Unitβ    : ∀{Γ A t} → iteUnit {Γ}{A} t trivial ≡ t
trivial[] : ∀{Γ Δ}{γ : Sub Δ Γ} → trivial [ γ ] ≡ trivial
iteUnit[] : ∀{Γ A t u Δ}{γ : Sub Δ Γ} →
            iteUnit {Γ}{A} u t [ γ ] ≡ iteUnit (u [ γ ]) (t [ γ ])
```

Another inductive type is that of lists. For any type A, we have a type List A. It has two constructors nil : Tm Γ (List A) and cons which again can be expressed in three different ways. We will use the first one when specifying the constructor and the third one when specifying the arguments of the iterator.

cons : Tm Γ A → Tm Γ (List A) → Tm Γ (List A)
cons : Tm Γ (A ⇒ List A ⇒ List A)
cons : Tm (Γ ▷ A ▷ List A) (List A)

A language has lists if it extends the substitution calculus with the following operators and equations:

```
List     : Ty → Ty
nil      : ∀{Γ A} → Tm Γ (List A)
cons     : ∀{Γ A} → Tm Γ A → Tm Γ (List A) → Tm Γ (List A)
iteList  : ∀{Γ A B} → Tm Γ B → Tm (Γ ▷ A ▷ B) B → Tm Γ (List A) → Tm Γ B
Listβ₁   : ∀{Γ A B}{u : Tm Γ B}{v : Tm (Γ ▷ A ▷ B) B} → iteList u v nil ≡ u
Listβ₂   : ∀{Γ A B}{u : Tm Γ B}{v : Tm (Γ ▷ A ▷ B) B}{t₁ : Tm Γ A}{t : Tm Γ (List A)} →
           iteList u v (cons t₁ t) ≡ (v [ ⟨ t₁ ⟩ ⁺ ] [ ⟨ iteList u v t ⟩ ])
nil[]    : ∀{Γ A Δ}{γ : Sub Δ Γ} → nil {Γ}{A} [ γ ] ≡ nil {Δ}{A}
cons[]   : ∀{Γ A}{t₁ : Tm Γ A}{t : Tm Γ (List A)}{Δ}{γ : Sub Δ Γ} →
           (cons t₁ t) [ γ ] ≡ cons (t₁ [ γ ]) (t [ γ ])
iteList[] : ∀{Γ A B}{u : Tm Γ B}{v : Tm (Γ ▷ A ▷ B) B}{t : Tm Γ (List A)}{Δ}{γ : Sub Δ Γ} →
            iteList u v t [ γ ] ≡ iteList (u [ γ ]) (v [ γ ⁺⁺ ]) (t [ γ ])
```

iteList binds a variable of type A and another variable of type B in its second explicit argument. In iteList u v t, u expresses what to do when t = nil t', v expresses what to do when t = cons t' t''.

Another inductive type is that of binary trees with information at the leaves. For any type A, Ty A is a type, the constructors are leaf and node. A language has these binary trees if it has the following operators and equations:

```
Tree     : Ty → Ty
leaf     : ∀{Γ A} → Tm Γ A → Tm Γ (Tree A)
node     : ∀{Γ A} → Tm Γ (Tree A) → Tm Γ (Tree A) → Tm Γ (Tree A)
iteTree  : ∀{Γ A B} → Tm (Γ ▷ A) B → Tm (Γ ▷ B ▷ B) B → Tm Γ (Tree A) → Tm Γ B
Treeβ₁   : ∀{Γ A B}{l : Tm (Γ ▷ A) B}{n : Tm (Γ ▷ B ▷ B) B}{a : Tm Γ A} →
           iteTree l n (leaf a) ≡ l [ ⟨ a ⟩ ]
Treeβ₂   : ∀{Γ A B}{l : Tm (Γ ▷ A) B}{n : Tm (Γ ▷ B ▷ B) B}{ll rr : Tm Γ (Tree A)} →
```

```
            iteTree l n (node ll rr) ≡ n [ ⟨ iteTree l n ll ⟩ ⁺ ] [ ⟨ iteTree l n rr ⟩ ]
leaf[]    : ∀{Γ A}{a : Tm Γ A}{Δ}{γ : Sub Δ Γ} → (leaf a) [ γ ] ≡ leaf (a [ γ ])
node[]    : ∀{Γ A}{ll rr : Tm Γ (Tree A)}{Δ}{γ : Sub Δ Γ} →
            (node ll rr) [ γ ] ≡ node (ll [ γ ]) (rr [ γ ])
iteTree[] : ∀{Γ A B}{l : Tm (Γ ▷ A) B}{n : Tm (Γ ▷ B ▷ B) B}{t : Tm Γ (Tree A)}
            {Δ}{γ : Sub Δ Γ} →
            iteTree l n t [ γ ] ≡ iteTree (l [ γ ⁺ ]) (n [ γ ⁺⁺ ]) (t [ γ ])
```

iteTree binds a variable of type A in its first explicit argument and two variables of type B in its second explicit argument.

The functions defined by iterators are all terminating. Even though they are recursive functions, infinite recursion is not possible, because the target of the iteration decreases at each recursive call.

The recursor (primitive recursor, paramorphism [25]) for natural numbers does not only receive the result of the recursive call, but also the number.

```
recNat      : ∀{Γ A} → Tm Γ A → Tm (Γ ▷ Nat ▷ A) A → Tm Γ Nat → Tm Γ A
recNatβ₁    : ∀{Γ A u v} → recNat {Γ}{A} u v zeroo ≡ u
¬recNatβ₂ : ¬ (∀{Γ A u v t} → recNat {Γ}{A} u v (suco t) ≡ v [ ⟨ t ⟩ ⁺ ] [ ⟨ recNat u v t ⟩ ])
```

For example, with the iterator, it is not possible to define a function pred : Tm (◇ ▷ Nat) Nat s.t. pred [⟨ suco t ⟩] ≡ t for any t (note that t might be a variable e.g. q) [18].

Exercise 5.1 (recommended)**.** *Define the recursor for natural numbers with the help of the iterator and show that it satisfies* recNatβ_1*.*

Exercise 5.2 (very hard)**.** *Show that* recNatβ_2 *does not hold in the syntax (you need a counter-model).*

The iterator is like a for loop where the body of the loop cannot refer to the loop variable. Recursor is like a general for loop. A fixpoint combinator is like while loop. A fixpoint combinator is defined by the following operator and equations:

```
fix    : Tm (Γ ▷ A) A → Tm Γ A
fixβ   : fix t ≡ t [ ⟨ fix t ⟩ ]
fix[]  : fix t [ γ ] ≡ fix (t [ γ ⁺ ])
```

In the case of Bool (or other non-recursive inductive types), there is no difference between the iterator and the recursor.

Our previous definition of Nat (in Section 2.1.3) was not an inductive type because it did not have an iterator or recursor, only two special cases of the iterator: addition and isZero. Both of these can be defined using the iterator.

With the iterator, we can define the following functions on natural numbers. Addition with variable names is written plus = λ x y . iteNat y (z . suc z) x which means that we have a function with two inputs x and y, we do recursion on the first input (x), and in the input, we replace zeros by y, and sucs by suc. The same information is expressed in languages with pattern matching as follows.

```
plus zero    y = y
plus (suc x') y = suc (plus x' y)
```

The first line corresponds to the first argument of iteNat (i.e. y), the second line corresponds to the second argument in which z is bound (z . suc z) where z refers to the recursive call which we write (plus x' y) using pattern matching notation.

In our formal syntax, we write this as follows.

```
plus : Tm ◇ (Nat ⇒ Nat ⇒ Nat)
plus = lam (lam (iteNat v0 (suco v0) v1))
```

The isZero operation can be defined as follows.

```
isZero : Tm ◇ (Nat ⇒ Bool)
isZero = lam (iteNat true false v0)
```

The zero case is simply true, the successor case is false regardless of the result of the recursive call.

Exercise 5.3 (compulsory). *Define* num *as syntactic sugar, and show that* plus *and* isZero *satisfy the same computation rules as in Razor!*

```
num : ∀{Γ} → ℕ → Tm Γ Nat
plusβ : ∀{n m} → plus $ num n $ num m ≡ num (n + m)
isZeroβ₁ : isZero $ num 0 ≡ true
isZeroβ₂ : ∀{n} → isZero $ num (1 + n) ≡ false
```

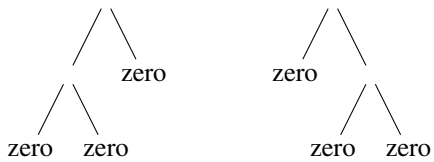
Examples function definitions for lists:

```
isnil : {A : Ty} → Tm ⋄ (List A ⇒ Bool)
isnil = lam (iteList true false v0)

concat : {A : Ty} → Tm ⋄ (List A ⇒ List A ⇒ List A)
concat = lam (lam (iteList v0 (cons v1 v0) v1))
```

isnil and concat are polymorphic functions in the sense that they work for any type A but the polymorphism happens in our metalanguage and not in our object language.

The following two trees in Tm ⋄ (Tree Nat)



are defined as

```
treeEx1 = node (node (leaf zeroo) (leaf zeroo)) (leaf zeroo)
treeEx2 = node (leaf zeroo) (node (leaf zeroo) (leaf zeroo))
```

The following function adds all the numbers in the leaves of a tree.

```
sum : Tm ⋄ (Tree Nat ⇒ Nat)
sum = lam (iteTree v0 (plus [ p ] [ p ] [ p ] $ v1 $ v0) v0)
```

With variable names, we would write this as sum = λ x . iteTree (z . z) (z_1 z_2 . plus \$ z_1 \$ z_2) x. In the formal syntax, we had to weaken plus three times by _[p] because it was defined in the empty context above and now we used it in a context with three free variables (one bound by lam, the other two by iteTree).

The following function turns a binary tree into a list:

```
flatten : ∀{A} → Tm ⋄ (Tree A ⇒ List A)
flatten = lam (iteTree (cons v0 nil) (concat [ p ] [ p ] [ p ] $ v1 $ v0) v0)
```

Exercise 5.4 (compulsory). *List the rules for inductively defined trees with ternary branching, a boolean at each node and a natural number at each leaf.*

Exercise 5.5 (compulsory). *List the rules for inductively defined trees with two kinds of nullary, one kind of binary and three kinds of ternary branching. There is no extra information at leaves or nodes.*

Exercise 5.6 (compulsory). *List the rules for inductively defined trees no information at the leaves and infinity (*Nat*-ary) branching.*

Exercise 5.7 (compulsory). *For types* A, B, *list the rules for the inductive type with only leaves (and no nodes) that contain an* A *and a* B. *Derive all its rules from binary products of the previous section. Which rule of binary products cannot be derived from this inductive type?*

Exercise 5.8 (compulsory)**.** *For types* A, B*, list the rules for the inductive type with two kinds of leaves and no nodes. One kind of leaf contains an* A*, the other kind of leaf contains a* B*. Derive all the rules from binary sums of the previous section. Which rule of binary sums cannot be derived from this inductive type?*

Exercise 5.9 (compulsory)**.** *How many elements are in the inductive type with only the following constructors?*

```
SpecialTree : Ty
node1 : Tm Γ SpecialTree → Tm Γ SpecialTree
node2 : Tm Γ SpecialTree → Tm Γ SpecialTree → Tm Γ SpecialTree
```

5.1 No uniqueness rules

Inductive types usually don't come with η rules. If natural numbers had an η rule, that would make equality undecidable. Because there is no η, we have many, non-equal implementations of the same function. For example, we list some implementations of the identity function on natural numbers. And there are infinitely many more.

```
id id' id'' : ∀{Γ} → Tm Γ (Nat ⇒ Nat)
id   = lam (var vz)
id'  = lam (iteNat zeroo (suco v0) v0)
id'' = lam (iteNat v0 v0 v0)
```

We can show that these are not equal using normalisation.

Similarly, there are infinitely many boolean negation functions.

Exercise 5.10. *Write down infinitely many different terms in* $\boldsymbol{Tm} \diamond (\boldsymbol{Bool} \Rightarrow \boldsymbol{Bool})$*!*

Exercise 5.11. *How many terms are in* $\boldsymbol{Tm} \diamond (\boldsymbol{Unit} \Rightarrow \boldsymbol{Unit})$ *where* Unit *is the inductively defined unit type without* η*?*

Exercise 5.12. *Show that for each of the above inductive types, we can define a model where there are nonstandard elements of the inductive types.*

5.2 Standard model

We define lists and trees using Agda's inductive types.

```
data List (A : Set) : Set where
  [] : List A
  _::_ : A → List A → List A
infixr 5 _::_
iteList : {A B : Set} → B → (A → B → B) → List A → B
iteList b f [] = b
iteList b f (a :: as) = f a (iteList b f as)

data Tree (A : Set) : Set where
  leaf : A → Tree A
  node : Tree A → Tree A → Tree A
iteTree : {A B : Set} → (A → B) → (B → B → B) → Tree A → B
iteTree f g (leaf a) = f a
iteTree f g (node t t') = g (iteTree f g t) (iteTree f g t')
```

The new components in the standard model:

```
; Nat      = ℕ
; zeroo    = λ _ → zero
```

```
; suco      = λ t γ* → suc (t γ*)
; iteNat    = λ u v t γ* → iteℕ (u γ*) (λ x → v (γ* , x)) (t γ*)
; Natβ₁     = refl
; Natβ₂     = refl
; zero[]    = refl
; suc[]     = refl
; iteNat[]  = refl

; Unit     = Lift ⊤
; trivial  = λ _ → mk triv
; iteUnit = λ z _ → z
; Unitβ   = refl
; trivial[] = refl
; iteUnit[] = refl

; List      = List
; nil       = λ _ → []
; cons      = λ u v γ* → u γ* :: v γ*
; iteList   = λ u v t γ* → iteList (u γ*) (λ x y → v ((γ* , x) , y)) (t γ*)
; Listβ₁    = refl
; Listβ₂    = refl
; nil[]     = refl
; cons[]    = refl
; iteList[] = refl

; Tree      = Tree
; leaf      = λ t γ* → leaf (t γ*)
; node      = λ u v γ* → node (u γ*) (v γ*)
; iteTree   = λ {Γ}{A}{B} u v t γ* → iteTree (λ x → u (γ* , x)) (λ x y → v ((γ* , x) , y)) (t γ*)
; Treeβ₁    = refl
; Treeβ₂    = refl
; leaf[]    = refl
; node[]    = refl
; iteTree[] = refl


t≠suct : ∀{t} → ¬ (t ≡ suco {◇} t)
t≠suct {t} e = n≠sucn (cong (λ z → St.⟦ z ⟧t (mk triv)) e)
  where
    n≠sucn : {n : ℕ} → ¬ (n ≡ suc n)
    n≠sucn ()
```

5.3 Definable functions on natural numbers

The Ackermann function is not definable using first-order primitive recursion (that is, in Def + natural numbers as a Tm (◇ ▷ Nat) Nat). But it is definable in our language because we have higher-order functions [11, Section 9.3]. Here is its definition in Agda:

```
ack : (x y : ℕ) -> ℕ
ack zero n = 1 + n
ack (suc m) zero = ack m 1
ack (suc m) (suc n) = ack m (ack (suc m) n)
```

Natural numbers are included in terms.

```
⌜_⌝ : ℕ → Tm ◇ Nat
⌜ zero ⌝ = zeroo
⌜ suc n ⌝ = suco ⌜ n ⌝
```

A function on natural numbers is definable if there is a term that, when applied to a natural number, gives the same result as the function.

```
Definable : (ℕ → ℕ) → Set
Definable f = Σsp (Tm ◇ (Nat ⇒ Nat)) λ t → (n : ℕ) → t $ ⌜ n ⌝ ≡ ⌜ f n ⌝
```

For example, the times two function is definable.

```
definable2* : Definable (iteℕ zero (λ x → suc (suc x)))
definable2* = (lam (iteNat zeroo (suco (suco v0)) v0)) , λ n →
  lam (iteNat zeroo (suco (suco v0)) v0) $ ⌜ n ⌝
      ≡⟨ ⇒β ⟩
  iteNat zeroo (suco (suco v0)) v0 [ ⟨ ⌜ n ⌝ ⟩ ]
      ≡⟨ iteNat[] ⟩
  iteNat (zeroo [ ⟨ ⌜ n ⌝ ⟩ ]) (suco (suco v0) [ ⟨ ⌜ n ⌝ ⟩ ⁺ ]) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])
      ≡⟨ cong (λ x → iteNat x (suco (suco v0) [ ⟨ ⌜ n ⌝ ⟩ ⁺ ]) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])) zero[] ⟩
  iteNat zeroo (suco (suco v0) [ ⟨ ⌜ n ⌝ ⟩ ⁺ ]) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])
      ≡⟨ cong (λ x → iteNat zeroo x (v0 [ ⟨ ⌜ n ⌝ ⟩ ])) suc[] ⟩
  iteNat zeroo (suco (suco v0 [ ⟨ ⌜ n ⌝ ⟩ ⁺ ])) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])
      ≡⟨ cong (λ x → iteNat zeroo (suco x) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])) suc[] ⟩
  iteNat zeroo (suco (suco (v0 [ ⟨ ⌜ n ⌝ ⟩ ⁺ ]))) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])
      ≡⟨ cong (λ x → iteNat zeroo (suco (suco x)) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])) vz[⁺] ⟩
  iteNat zeroo (suco (suco v0)) (v0 [ ⟨ ⌜ n ⌝ ⟩ ])
      ≡⟨ cong (iteNat zeroo (suco (suco v0))) vz[⟨⟩] ⟩
  iteNat zeroo (suco (suco v0)) ⌜ n ⌝
      ≡⟨ ⌜rec⌝ n ⁻¹ ⟩
    ⌜ iteℕ zero (λ x → suc (suc x)) n ⌝
      ∎
  where
    ⌜rec⌝ : (n : ℕ) → ⌜ iteℕ zero (λ x → suc (suc x)) n ⌝ ≡ iteNat zeroo (suco (suco v0)) ⌜ n ⌝
    ⌜rec⌝ zero = Natβ₁ ⁻¹
    ⌜rec⌝ (suc n) =
      suco (suco ⌜ iteℕ zero (λ x → suc (suc x)) n ⌝)
        ≡⟨ cong {A = Tm _ _}(λ x → suco (suco x)) (⌜rec⌝ n) ⟩
      suco (suco (iteNat zeroo (suco (suco v0)) ⌜ n ⌝))
        ≡⟨ cong {A = Tm _ _}(λ x → suco (suco x)) (vz[⟨⟩] ⁻¹) ⟩
      suco (suco (v0 [ ⟨ iteNat zeroo (suco (suco v0)) ⌜ n ⌝ ⟩ ]))
        ≡⟨ cong suco (suc[] ⁻¹) ⟩
      suco (suco v0 [ ⟨ iteNat zeroo (suco (suco v0)) ⌜ n ⌝ ⟩ ])
        ≡⟨ suc[] ⁻¹ ⟩
      suco (suco v0) [ ⟨ iteNat zeroo (suco (suco v0)) ⌜ n ⌝ ⟩ ]
        ≡⟨ Natβ₂ ⁻¹ ⟩
      iteNat zeroo (suco (suco v0)) (suco ⌜ n ⌝)
        ∎
```

Here is definability for two-argument functions.

```
Definable' : (ℕ → ℕ → ℕ) → Set
Definable' f = Σ (Tm ◇ (Nat ⇒ Nat ⇒ Nat)) λ t → (m n : ℕ) → Lift (t $ ⌜ m ⌝ $ ⌜ n ⌝ ≡ ⌜ f m n ⌝)
```

It is (relatively) easy to encode terms as natural numbers, for now we only assume a code function.

```
postulate
  code : Tm ◇ (Nat ⇒ Nat) → ℕ
```

The evaluator of the language can be defined as interpretation into the standard model, for simplicity we only assume it here. Here we use a special case which takes a code for a Nat-endofunction and

a natural number. TODO: use normalisation and its completeness together with decode-encode iso to define the evaluator and prove its property. Is canonicity enough?

```
eval : ℕ → ℕ → ℕ
evalProp : (u : Tm ⋄ (Nat ⇒ Nat))(n : ℕ) → u $ ⌜ n ⌝ ≡ ⌜ eval (code u) n ⌝
```

The evaluator is not definable. The proof uses Cantor's diagonal argument.

```
evalNotDefinable : Definable' eval → Lift ⊥
evalNotDefinable (t , f) = mk (t≠suct (
  ⌜ eval (code u) (code u) ⌝
    ≡⟨ evalProp u (code u) ⁻¹ ⟩
  u $ ⌜ code u ⌝
    ≡⟨ ⇒β ⟩
  suco (t [ p ] $ v0 $ v0) [ ⟨ ⌜ code u ⌝ ⟩ ]
    ≡⟨ suc[] ⟩
  suco ((t [ p ] $ v0 $ v0) [ ⟨ ⌜ code u ⌝ ⟩ ])
    ≡⟨ cong suco $[] ⟩
  suco (((t [ p ] $ v0) [ ⟨ ⌜ code u ⌝ ⟩ ] $ v0 [ ⟨ ⌜ code u ⌝ ⟩ ]))
    ≡⟨ cong (λ x → suco (x $ v0 [ ⟨ ⌜ code u ⌝ ⟩ ])) $[] ⟩
  suco ((t [ p ] [ ⟨ ⌜ code u ⌝ ⟩ ] $ v0 [ ⟨ ⌜ code u ⌝ ⟩ ] $ v0 [ ⟨ ⌜ code u ⌝ ⟩ ]))
    ≡⟨ cong (λ x → suco ((t [ p ] [ ⟨ ⌜ code u ⌝ ⟩ ] $ x $ x))) vz[⟨⟩] ⟩
  suco ((t [ p ] [ ⟨ ⌜ code u ⌝ ⟩ ] $ ⌜ code u ⌝ $ ⌜ code u ⌝))
    ≡⟨ cong (λ x → suco ((x $ ⌜ code u ⌝ $ ⌜ code u ⌝))) [p][⟨⟩] ⟩
  suco ((t $ ⌜ code u ⌝ $ ⌜ code u ⌝))
    ≡⟨ cong suco (un (f (code u) (code u))) ⟩
  suco ⌜ eval (code u) (code u) ⌝
    ∎))
  where
    u : Tm ⋄ (Nat ⇒ Nat)
    u = lam (suco (t [ p ] $ v0 $ v0))
    open Standard
```

Summary of non-definability: if there is a surjection from ℕ to Tm ⋄ (Nat ⇒ Nat) and the latter is isomorphic to ℕ → ℕ, then there is a surjection from ℕ to ℕ → ℕ.

TODO: using an isomorphism ℕ ≅ ℕ × ℕ and its internal version Nat ≅ Nat × Nat, we can show a ℕ ⇒ ℕ ⇒ ℕ function is definable iff. its ℕ ⇒ ℕ encoded variant is definable.

Exercise 5.13. *Show that the constant* 3 *function is definable!*

5.4 Normalisation

For each inductive type, the constructor becomes a normal form, the destructor becomes a neutral term. Inductive types act as base types, so all neutral terms of a base type are normal forms.

```
infixl 5 _$Ne_

data Ne : Con → Ty → Set
data Nf : Con → Ty → Set
data Ne where
  varNe     : ∀{Γ A} → Var Γ A → Ne Γ A
  _$Ne_     : ∀{Γ A B} → Ne Γ (A ⇒ B) → Nf Γ A → Ne Γ B
  iteBoolNe : ∀{Γ A} → Nf Γ A → Nf Γ A → Ne Γ Bool → Ne Γ A
  iteNatNe  : ∀{Γ C} → Nf Γ C → Nf (Γ ▷ C) C → Ne Γ Nat → Ne Γ C
  iteUnitNe : ∀{Γ A} → Nf Γ A → Ne Γ Unit → Ne Γ A
  iteListNe : ∀{Γ A C} → Nf Γ C → Nf (Γ ▷ A ▷ C) C → Ne Γ (List A) → Ne Γ C
  iteTreeNe : ∀{Γ A C} → Nf (Γ ▷ A) C → Nf (Γ ▷ C ▷ C) C → Ne Γ (Tree A) → Ne Γ C
```

```
data Nf where
  neuBool : ∀{Γ} → Ne Γ Bool → Nf Γ Bool
  neuNat  : ∀{Γ} → Ne Γ Nat → Nf Γ Nat
  neuUnit : ∀{Γ} → Ne Γ Unit → Nf Γ Unit
  neuList : ∀{Γ A} → Ne Γ (List A) → Nf Γ (List A)
  neuTree : ∀{Γ A} → Ne Γ (Tree A) → Nf Γ (Tree A)
  lamNf   : ∀{Γ A B} → Nf (Γ ▷ A) B → Nf Γ (A ⇒ B)
  trueNf  : ∀{Γ} → Nf Γ Bool
  falseNf : ∀{Γ} → Nf Γ Bool
  zeroNf  : ∀{Γ} → Nf Γ Nat
  sucNf   : ∀{Γ} → Nf Γ Nat → Nf Γ Nat
  nilNf   : ∀{Γ A} → Nf Γ (List A)
  consNf  : ∀{Γ A} → Nf Γ A → Nf Γ (List A) → Nf Γ (List A)
  leafNf  : ∀{Γ A} → Nf Γ A → Nf Γ (Tree A)
  nodeNf  : ∀{Γ A} → Nf Γ (Tree A) → Nf Γ (Tree A) → Nf Γ (Tree A)

⌜_⌝Ne : ∀{Γ A} → Ne Γ A → Tm Γ A
⌜_⌝Nf : ∀{Γ A} → Nf Γ A → Tm Γ A
```

Exercise 5.14. *What are the normal form of the following terms?*

```
t1 : Σsp (Nf (◇ ▷ Nat ⇒ Bool) (Nat ⇒ Bool)) λ v → ⌜ v ⌝Nf ≡ v0
t2 : Σsp (Nf (◇ ▷ Bool) Bool) λ v → ⌜ v ⌝Nf ≡ v0
t3 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → ⌜ v ⌝Nf ≡ v0
t4 : Σsp (Nf (◇ ▷ Nat) Nat) λ v → ⌜ v ⌝Nf ≡ iteNat v0 v0 v0
t5 : Σsp (Nf ◇ Nat) λ v → ⌜ v ⌝Nf ≡ iteNat zeroo v0 (suco (suco zeroo))
t6 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → ⌜ v ⌝Nf ≡ iteNat false false v0
t7 : Σsp (Nf ◇ Bool) λ v → ⌜ v ⌝Nf ≡ iteBool false true false
t8 : Σsp (Nf ◇ Bool) λ v → ⌜ v ⌝Nf ≡ lam (iteNat true false v0) $ zeroo
t9 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → ⌜ v ⌝Nf ≡ lam (iteNat true false v0) $ v0
t10 : Σsp (Nf (◇ ▷ Nat) Bool) λ v → ⌜ v ⌝Nf ≡ lam (iteNat true false v0) $ suco v0
```

Chapter 6

Coinductive types

Learning goals of this chapter

Coinductive types by example: streams, machines.

Coinductive types are dual to inductive types. Inductive types are specified by their constructors, coinductive types are specified by their destructors. Their constructor is determined by the destructors: if we have a type C (the seed) together with terms which have the shape of the destructors (but using C instead of the coinductive type), we obtain a function (called corecursor, generator, anamorphism) from the seed to the coinductive type. The seed is called like that because the generator makes the coinductive type "grow" out of it. While elements of inductive type are trees of finite depth, elements of coinductive types are potentially infinitely deep trees. Functions out of inductive types are all terminating, functions producing elements of coinductive types are *productive*.

The usual example is infinite lists or streams. There are two ways to destruct a stream: look at its first element (head), or forget its first element (tail). To generate a stream, we need to fix a seed type C which represents the internal state of the stream; we have to provide an A from any C; we have to be able to step the internal state when forgetting the first element; and finally, we need a state to start with. The computation rules say what happens if a destructor is applied to the constructor (generator): we use the corresponding method with the actual internal state, and corecursively call the generator on the new state, if needed.

```
Stream      : Ty → Ty
head        : ∀{Γ A} → Tm Γ (Stream A) → Tm Γ A
tail        : ∀{Γ A} → Tm Γ (Stream A) → Tm Γ (Stream A)
genStream   : ∀{Γ A C} → Tm (Γ ▷ C) A → Tm (Γ ▷ C) C → Tm Γ C → Tm Γ (Stream A)
Streamβ₁    : ∀{Γ A C}{u : Tm (Γ ▷ C) A}{v : Tm (Γ ▷ C) C}{t : Tm Γ C} →
              head (genStream u v t) ≡ u [ ⟨ t ⟩ ]
Streamβ₂    : ∀{Γ A C}{u : Tm (Γ ▷ C) A}{v : Tm (Γ ▷ C) C}{t : Tm Γ C} →
              tail (genStream u v t) ≡ genStream u v (v [ ⟨ t ⟩ ])
head[]      : ∀{Γ A}{as : Tm Γ (Stream A)}{Δ}{γ : Sub Δ Γ} →
              head as [ γ ] ≡ head (as [ γ ])
tail[]      : ∀{Γ A}{as : Tm Γ (Stream A)}{Δ}{γ : Sub Δ Γ} → tail as [ γ ] ≡ tail (as [ γ ])
genStream[] : ∀{Γ A C}{u : Tm (Γ ▷ C) A}{v : Tm (Γ ▷ C) C}{t : Tm Γ C}{Δ}
              {γ : Sub Δ Γ} →
              genStream u v t [ γ ] ≡ genStream (u [ γ ⁺ ]) (v [ γ ⁺ ]) (t [ γ ])
```

genStream binds a variable of type C in both its first end second explicit arguments.

State machines, virtual machines, Turing machines, servers, operating systems are all coinductive types. An example simple machine has three operations: (i) we can put in a natural number, (ii) we can press a button on it, (iii) we can ask it to output a natural number.

```
Machine     : Ty
```

```
put          : ∀{Γ} → Tm Γ Machine → Tm Γ Nat → Tm Γ Machine
set          : ∀{Γ} → Tm Γ Machine → Tm Γ Machine
get          : ∀{Γ} → Tm Γ Machine → Tm Γ Nat
genMachine   : ∀{Γ C} → Tm (Γ ▷ C ▷ Nat) C → Tm (Γ ▷ C) C → Tm (Γ ▷ C) Nat →
               Tm Γ C → Tm Γ Machine
Machineβ1    : ∀{Γ C}{u : Tm (Γ ▷ C ▷ Nat) C}{v : Tm (Γ ▷ C) C}{w : Tm (Γ ▷ C) Nat}
               {t : Tm Γ C}{t' : Tm Γ Nat} →
               put (genMachine u v w t) t' ≡ genMachine u v w (u [ ⟨ t ⟩ ⁺ ] [ ⟨ t' ⟩ ])
Machineβ2    : ∀{Γ C}{u : Tm (Γ ▷ C ▷ Nat) C}{v : Tm (Γ ▷ C) C}{w : Tm (Γ ▷ C) Nat}
               {t : Tm Γ C} →
               set (genMachine u v w t) ≡ genMachine u v w (v [ ⟨ t ⟩ ])
Machineβ3    : ∀{Γ C}{u : Tm (Γ ▷ C ▷ Nat) C}{v : Tm (Γ ▷ C) C}{w : Tm (Γ ▷ C) Nat}
               {t : Tm Γ C} →
               get (genMachine u v w t) ≡ w [ ⟨ t ⟩ ]
put[]        : ∀{Γ}{t : Tm Γ Machine}{u : Tm Γ Nat}{Δ}{γ : Sub Δ Γ} →
               put t u [ γ ] ≡ put (t [ γ ]) (u [ γ ])
set[]        : ∀{Γ}{t : Tm Γ Machine}{Δ}{γ : Sub Δ Γ} → set t [ γ ] ≡ set (t [ γ ])
get[]        : ∀{Γ}{t : Tm Γ Machine}{Δ}{γ : Sub Δ Γ} → get t [ γ ] ≡ get (t [ γ ])
genMachine[] : ∀{Γ C}{u : Tm (Γ ▷ C ▷ Nat) C}{v : Tm (Γ ▷ C) C}{w : Tm (Γ ▷ C) Nat}
               {t : Tm Γ C}{Δ}{γ : Sub Δ Γ} →
               genMachine u v w t [ γ ] ≡
               genMachine (u [ γ ⁺ ⁺ ]) (v [ γ ⁺ ]) (w [ γ ⁺ ]) (t [ γ ])
```

genMachine binds a variable of type C and a variable of type Nat in its first explicit argument, and binds a variable of type C in its second and third explicit arguments.

The stream which contains the number 0,1,2,...:

```
numbers : Tm ◇ (Stream Nat)
numbers = genStream v0 (suco v0) zeroo

testNumbers0 : head numbers ≡ zeroo
testNumbers0 =
  head numbers
    ≡⟨ refl {A = Tm _ _} ⟩
  head (genStream v0 (suco v0) zeroo)
    ≡⟨ Streamβ1 ⟩
  v0 [ ⟨ zeroo ⟩ ]
    ≡⟨ vz[⟨⟩] ⟩
  zeroo
    ∎
```

Exercise 6.1. *Prove the following equation.*

```
testNumbers2 : head (tail (tail numbers)) ≡ suco (suco zeroo)
```

A machine which adds the numbers that we input unless we press the button, then it adds one when inputting a number. If we toggle the button again, it will start adding the numbers again. The number it outputs is the current sum.

```
aMachine : Tm ◇ Machine
aMachine = genMachine (fst v1 ,o (plus $ snd v1 $ v0))
  (iteBool false true (fst v0) ,o snd v0) (snd v0) (true ,o zeroo)
```

Exercise 6.2 (compulsory)**.** *For types* A, B, *define the coinductive type of machines from which we can get out an* A *and a* B*. List all the rules. Derive these rules from those of binary products.*

Exercise 6.3 (compulsory). *Given a language with machines as in the previous exercise, derive the rules of binary products! Which one is not derivable?*

Exercise 6.4 (compulsory). *For types* A, B, *define the coinductive type of machines which has one operation: it receives an input of type* A, *and outputs something of type* B. *List all the rules. Derive these rules from those of the function space.*

Exercise 6.5 (compulsory). *Given a language with machines as in the previous exercise, derive the rules of function space! Which one is not derivable?*

Exercise 6.6 (compulsory). *Define the coinductive types of machines for which there are two different ways to input a natural number and if we press a button, they output a natural number. Implement a machine which has two numbers in its inner state and outputs one or the other alternating. If we input a number in the first way, the machine adds that to its first inner number. If we input a number in the second way, the machine adds that to its second inner number.*

6.1 Standard model

We define streams and machines using Agda's coinductive types.

```
record Stream (A : Set) : Set where
  coinductive
  field
    head : A
    tail : Stream A
open Stream
genStream : {A C : Set} → (C → A) → (C → C) → C → Stream A
head (genStream f g c) = f c
tail (genStream f g c) = genStream f g (g c)

record Machine : Set where
  coinductive
  field
    put : ℕ → Machine
    set : Machine
    get : ℕ
open Machine
genMachine : {C : Set} → (C → ℕ → C) → (C → C) → (C → ℕ) → C → Machine
put (genMachine f g h c) n = genMachine f g h (f c n)
set (genMachine f g h c) = genMachine f g h (g c)
get (genMachine f g h c) = h c
```

The new components in the standard model:

```
; Stream        = Stream
; head          = λ t γ* → head (t γ*)
; tail          = λ t γ* → tail (t γ*)
; genStream     = λ u v t γ* → genStream (λ x → u (γ* , x)) (λ x → v (γ* , x)) (t γ*)
; Streamβ₁      = refl
; Streamβ₂      = refl
; head[]        = refl
; tail[]        = refl
; genStream[]   = refl

; Machine       = Machine
; put           = λ t u γ* → put (t γ*) (u γ*)
; set           = λ t γ* → set (t γ*)
; get           = λ t γ* → get (t γ*)
```

```
; genMachine    = λ u v w t γ* → genMachine (λ x y → u ((γ* , x) , y))
                   (λ x → v (γ* , x)) (λ x → w (γ* , x)) (t γ*)
; Machineβ1     = refl
; Machineβ2     = refl
; Machineβ3     = refl
; put[]         = refl
; set[]         = refl
; get[]         = refl
; genMachine[]  = refl
```

Chapter 7

Summary

A table with the rules for different type formers of the languages defined in the previous chapters.

type formation	constructor	destructor	computation	uniqueness	
Bool	true, false	ite	iteβ_1, iteβ_2		Chapter 1
Nat	num	isZero, _+o_	isZeroβ_1, isZeroβ_2, +β		
_⇒_	lam	_$_	⇒β	⇒η	Chapter 3
_×o_	_,o_	fst, snd	×β_1, ×β_2	×η	Chapter 4
Unit	trivial			Unitη	
_+o_	inl, inr	caseo	+β_1, +β_2	+η	
Empty		absurd		Emptyη	
Nat	zeroo, suco	iteNat	Natβ_1, Natβ_2		Chapter 5
List	nil, cons	iteList	Listβ_1, Listβ_2		
Tree	leaf, node	iteTree	Treeβ_1 Treeβ_2		
Stream	genStream	head, tail	Streamβ_1, Streamβ_2		Chapter 6
Machine	genMachine	put, set, get	Machineβ_1, Machineβ_2, Machineβ_3		

Bibliography

[1] Andreas Abel. Foetus – termination checker for simple functional programs. Programming Lab Report, 1998.

[2] Thorsten Altenkirch, Ambrus Kaposi, Artjoms Sinkarovs, and Tamás Végh. Combinatory logic and lambda calculus are equal, algebraically. In Marco Gaboardi and Femke van Raamsdonk, editors, *8th International Conference on Formal Structures for Computation and Deduction, FSCD 2023, July 3-6, 2023, Rome, Italy*, volume 260 of *LIPIcs*, pages 24:1–24:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

[3] Thorsten Altenkirch and Luis Scoccola. The integers as a higher inductive type. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science*, LICS '20, page 67–73, New York, NY, USA, 2020. Association for Computing Machinery.

[4] N. G. De Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *INDAG. MATH*, 34:381–392, 1972.

[5] John Cartmell. Generalised algebraic theories and contextual categories. *Annals of Pure and Applied Logic*, 32:209–243, 1986.

[6] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58:345, 1936.

[7] Jesper Cockx and Andreas Abel. Sprinkles of extensionality for your vanilla type theory. In Silvia Ghilezan and Jelena Ivetić, editors, *22nd International Conference on Types for Proofs and Programs, TYPES 2016*. University of Novi Sad, 2016.

[8] Zoltán Csörnyei. *Lambda-kalkulus. A funkcionális programozás alapjai.* Typotex, 2007.

[9] Zoltán Csörnyei. *Bevezetés a típusrendszerek elméletébe.* ELTE Eötvös Kiadó, 2012.

[10] Gaëtan Gilbert, Jesper Cockx, Matthieu Sozeau, and Nicolas Tabareau. Definitional proof-irrelevance without K. *Proc. ACM Program. Lang.*, 3(POPL):3:1–3:28, 2019.

[11] Robert Harper. *Practical Foundations for Programming Languages.* Cambridge University Press, New York, NY, USA, 2nd edition, 2016.

[12] Graham Hutton. Programming language semantics: it's easy as 1,2,3. Draft, 2023.

[13] Ambrus Kaposi. Levels of abstraction when defining type theory in type theory. Talk given at Conference "Celebrating 90 Years of Gödel's Incompleteness Theorems" in Nürtingen, Germany. Slides: https://akaposi.github.io/pres_godel90.pdf. Video: https://youtu.be/8qRrN6ewSYw, July 2021.

[14] Ambrus Kaposi, András Kovács, and Thorsten Altenkirch. Constructing quotient inductive-inductive types. *Proc. ACM Program. Lang.*, 3(POPL):2:1–2:24, January 2019.

[15] Chantal Keller and Thorsten Altenkirch. Hereditary substitutions for simple types, formalized. In Venanzio Capretta and James Chapman, editors, *Proceedings of the 3rd ACM SIGPLAN Workshop on Mathematically Structured Functional Programming, MSFP@ICFP 2010, Baltimore, MD, USA, September 25, 2010*, pages 3–10. ACM, 2010.

[16] Conor McBride. *Dependently typed functional programs and their proofs.* PhD thesis, University of Edinburgh, 1999.

[17] Ulf Norell. Dependently typed programming in agda. In Pieter W. M. Koopman, Rinus Plasmeijer, and S. Doaitse Swierstra, editors, *Advanced Functional Programming, 6th International School, AFP 2008, Heijen, The Netherlands, May 2008, Revised Lectures*, volume 5832 of *Lecture Notes in Computer Science*, pages 230–266. Springer, 2008.

[18] Michel Parigot. On the representation of data in lambda-calculus. In Egon Börger, Hans Kleine Büning, and Michael M. Richter, editors, *CSL '89, 3rd Workshop on Computer Science Logic, Kaiserslautern, Germany, October 2-6, 1989, Proceedings*, volume 440 of *Lecture Notes in Computer Science*, pages 309–321. Springer, 1989.

[19] Benjamin C. Pierce. *Types and Programming Languages.* The MIT Press, 1st edition, 2002.

[20] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hriţcu, Vilhelm Sjöberg, Andrew Tolmach, and Brent Yorgey. *Programming Language Foundations*, volume 2 of *Software Foundations.* Electronic textbook, 2021. Version 6.1, http://softwarefoundations.cis.upenn.edu.

[21] Moses Schönfinkel. Über die bausteine der mathematischen logik. *Mathematische Annalen*, 92(3):305–316, Sep 1924.

[22] William W. Tait. Intensional interpretations of functionals of finite type I. *J. Symb. Log.*, 32(2):198–212, 1967.

[23] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937.

[24] Philip Wadler, Wen Kokke, and Jeremy G. Siek. *Programming Language Foundations in Agda.* July 2020. http://plfa.inf.ed.ac.uk/20.07.

[25] Zhixuan Yang and Nicolas Wu. Fantastic morphisms and where to find them - A guide to recursion schemes. In Ekaterina Komendantskaya, editor, *Mathematics of Program Construction - 14th International Conference, MPC 2022, Tbilisi, Georgia, September 26-28, 2022, Proceedings*, volume 13544 of *Lecture Notes in Computer Science*, pages 222–267. Springer, 2022.